Fakultät Informatik

Exploration and Navigation of Latent Spaces in Generative Diffusion Models with Perceptual Evaluation of Results and Navigation Tools

Bachelorarbeit im Studiengang Medieninformatik

vorgelegt von

David Nguyen

Matrikelnummer 354 8145

Erstgutachter:	Prof. Dr. Ulrich von Zadow
Zweitgutachter:	Prof. Moritz Schwind

© 2026

Dieses Werk einschließlich seiner Teile ist **urheberrechtlich geschützt**. Jede Verwertung außerhalb der engen Grenzen des Urheberrechtgesetzes ist ohne Zustimmung des Autors unzulässig und strafbar. Das gilt insbesondere für Vervielfältigungen, Übersetzungen, Mikroverfilmungen sowie die Einspeicherung und Verarbeitung in elektronischen Systemen.

Kurzdarstellung

Diese Arbeit untersucht, wie sich latente Räume in generativen Diffusionsmodellen systematisch erkunden und navigieren lassen. Obwohl latente Räume eine zentrale Grundlage moderner Bildgenerierung bilden, sind ihre innere Struktur und ihre praktischen Grenzen weiterhin nur schwer nachvollziehbar. Im Mittelpunkt der Untersuchung stehen Navigationsverfahren wie Interpolation und die kontrollierte Bewegung entlang ausgewählter Richtungen, ohne dabei das zugrunde liegende Modell selbst zu verändern.

Zur Unterstützung dieser Untersuchung wurde ein prototypisches Explorationswerkzeug als externe Erweiterung für ComfyUI entwickelt. Das Plugin ermöglicht die Erstellung promptbasierter Datensätze, die Projektion und Clusterung ihrer Embeddings sowie die Ausführung verschiedener Navigationsoperationen innerhalb eines reproduzierbaren Arbeitsablaufs. Dadurch entsteht eine praktische Umgebung, in der sich das Verhalten unterschiedlicher Navigationsverfahren vergleichen und ihre Ergebnisse visuell untersuchen lassen.

Der experimentelle Teil der Arbeit analysiert generierte Bildsequenzen und Navigationsergebnisse im Hinblick auf Kontinuität, semantische Veränderung und Stabilität. Die Ergebnisse zeigen, wie ein werkzeuggestützter Ansatz dazu beitragen kann, das Verhalten latenter Räume zugänglicher zu machen, und verdeutlichen zugleich die Grenzen der Interpretierbarkeit und Steuerbarkeit diffusionsbasierter generativer Systeme.

Abstract

This thesis examines how latent spaces in generative diffusion models can be explored and navigated. Latent spaces are central to modern image generation, but their internal structure and practical limits are still difficult to understand. The work focuses on navigation methods such as interpolation and controlled movement along selected directions without changing the underlying model itself.

To support this analysis, a prototype exploration tool was developed as an external extension for ComfyUI. The plugin makes it possible to create prompt-based datasets, project and cluster their embeddings, and run different navigation operations in a reproducible workflow. This provides a practical environment for comparing how different navigation methods behave and how their results can be inspected visually.

The experimental part of the thesis studies generated image sequences and navigation outputs with regard to continuity, semantic change and stability. The thesis shows how such a

tool-based workflow can help make latent-space behavior more accessible, while also highlighting the limits of interpretability and controllability in diffusion-based generative systems.

Contents

1. Introduction **1**
1.1. Motivation 1
1.2. Goal, Research Questions, and Scope 2
1.3. Thesis Structure 2

2. Foundations **3**
2.1. Diffusion and latent diffusion basics 3
2.2. Conditioning, prompt embeddings, and CLIP-based text representations 4
2.3. Compact embeddings versus real conditioning 5
2.4. Dimensionality reduction with UMAP / Parametric UMAP 6
2.5. Density-based clustering with HDBSCAN 8
2.6. Similarity, interpolation, and direction-based navigation 8

3. Related Work **11**
3.1. Latent space exploration, controllability and navigation in generative models . . 11
3.2. Visual and interactive exploration tools for generative systems 13
3.3. Evaluation strategies for latent navigation and generated outputs 14

4. System Design, Implementation, and Experimental Setup **16**
4.1. Requirements for the exploration environment 16
4.2. Technology and platform selection 17
4.3. NeLs: an external ComfyUI plugin 19
4.4. UI/UX design rationale 20
4.5. Model, encoder, and prompt-source selection 23
4.6. Data acquisition and processing 25
4.7. Runtime session and graph-panel flow 32
4.8. Operation families: implementation, math, and configurable parameters 34
4.9. Design tradeoffs and validity limits 50

5. Experimental Results and Analysis **53**
5.1. Evaluation parameters 53
5.2. Projection and cluster structure 54
5.3. Interpolation behavior: LERP vs SLERP 58
5.4. Direction transfer with Delta / Direction Walk 64

5.5. Semantic Axis Projection . . . 70
5.6. Consistency Check . . . 75
5.7. Latent Probe . . . 78

6. Discussion . . . 83

7. Conclusion and Future Work . . . 85

A. Supplemental Information . . . 86
A.1. DeepWiki Research Examples . . . 86
A.1.1. ComfyUI . . . 86
A.1.2. GitNexus . . . 87
A.2. NELS Graph Panel Specification . . . 88
A.3. LLM System Prompts . . . 97
A.3.1. Dataset annotation (Section 4.6) . . . 97
A.3.2. Semantic Axis Projection (Section 4.8) . . . 98
A.3.3. Consistency Check (Section 4.8) . . . 99
A.3.4. Latent Probe (Section 4.8) . . . 99

List of Figures . . . 101

List of Tables . . . 103

Bibliography . . . 104

Glossary . . . 110

Chapter 1.

Introduction

1.1. Motivation

Text-to-image generation has gained substantial importance in visual and creative practices. The tools that make this technology usable are no longer confined to only research environments, because non-professional users are increasingly relying on them for everyday creative work as well [59]. This broader adoption highlights needs that extend beyond the simple generation of an image from a prompt, since users frequently also want to edit an existing image, explore variations of an idea, or compare and move between different results.

Much of this capability relies on diffusion models, which can achieve high-quality image generation within a strong generative framework [31]. This was made more practical through latent diffusion, which moves the diffusion process into a compressed latent representation and supports flexible conditioning through text prompts [53].

This growing practical use, however, does not make the underlying process easy to understand. It is easy to generate many images, but it is difficult to comprehend why specific changes in prompts, seeds, or latent operations lead to specific visual results. The generative process involves several interacting representation spaces rather than a single unified latent space, each playing a different role in the generation pipeline. Prompt-related representations are of particular interest for semantic control, since text is encoded into embeddings that guide the generation process from start to finish [52, 53]. In everyday use, however, these internal representations remain hidden, even though this is the level at which meaningful variations and transitions are happening.

This matters increasingly for exactly this kind of exploration, since testing directional changes or checking whether a sequence of edits remains smooth or breaks down depends on representations that stay hidden from the user. Recent research suggests that these spaces do not behave uniformly: small alterations can lead to different visual changes, and intermediate results are not always stable or meaningful [56, 47]. Navigation through these spaces should therefore not simply be assumed to work smoothly, but they should be examined

more carefully. This thesis is motivated precisely by this gap between generating images on the one hand, and systematically exploring and inspecting latent spaces on the other.

1.2. Goal, Research Questions, and Scope

Building on this motivation, this thesis aims to make latent-space navigation in generative diffusion models reproducible and empirically grounded. Concretely, existing text-to-image workflows are explored through interpolation, direction-based navigation, and structured comparison of their representations and outputs. To this end, the thesis combines a technical contribution, an external ComfyUI plugin for dataset creation, projection, clustering, and navigation, with a scientific contribution, an empirical analysis of how latent-space navigation behaves with regard to continuity, semantic change, stability, and interpretability.

This goal is pursued through three research questions:

1. To what extent can latent spaces in generative diffusion models be systematically explored through interpolation and targeted navigation without modifying the underlying model?
2. Which structural and perceptual patterns emerge during latent space navigation, and where do discontinuities or instabilities occur?
3. What are the limits of interpretability and controllability of latent spaces?

The work does not retrain or fine-tune the diffusion model itself and does not aim at a complete theory of latent-space geometry. It instead studies a bounded set of navigation operations on an existing generative system and interprets the results as empirical findings tied to the chosen model, prompts, and parameter settings. Broader comparative studies and an extended user study will be part of future work.

1.3. Thesis Structure

This thesis is organized into seven chapters. Following the introduction, Chapters 2 and 3 establish the theoretical background and review related work relevant to this thesis. Chapter 4 presents the design and implementation of the plugin NELS (Navigation and Exploration of Latent Spaces) together with its experimental framework, while Chapter 5 reports the resulting analyses of latent-space structure and navigation behavior. Chapter 6 discusses these findings in relation to the research questions and the limitations of the evaluation, and Chapter 7 concludes the thesis by outlining its main contributions and directions for future work.

Chapter 2.

Foundations

This chapter introduces the technical foundations required for the remainder of the thesis. It first explains how diffusion models generate images, how latent diffusion improves efficiency by operating in a compressed representation rather than directly in pixel space, and how text prompts are encoded to guide the generation process. It then distinguishes between the two representations used throughout this work: a detailed conditioning representation for image generation and a compact embedding for comparison, projection, and analysis. The remaining sections present the methods used to make these compact representations inspectable.

2.1. Diffusion and latent diffusion basics

Generative models aim to learn a data distribution so that new samples can be produced that resemble the observed data. Several different architectural approaches exist for this purpose. For this thesis, diffusion-based models are the most relevant because they currently form a major foundation of modern text-to-image systems.

Diffusion models generate data by reversing a noising process. In the forward process, noise is gradually added to a data sample over many steps until the signal is strongly degraded. In the reverse process, a neural network is trained to predict how this noise should be removed step by step, so that a new sample can be reconstructed from random noise. This makes generation iterative. Instead of producing the final image in one pass, the model refines an intermediate representation through a sequence of denoising updates, as illustrated in Figure 2.1. [31]

This setup has two important consequences. First, diffusion models are flexible and can be conditioned in different ways, for example through text. Second, they are computationally expensive when they operate directly in pixel space, because the model has to process high-dimensional image tensors repeatedly over many steps. This motivates the move from pixel-space diffusion to latent diffusion. [53]

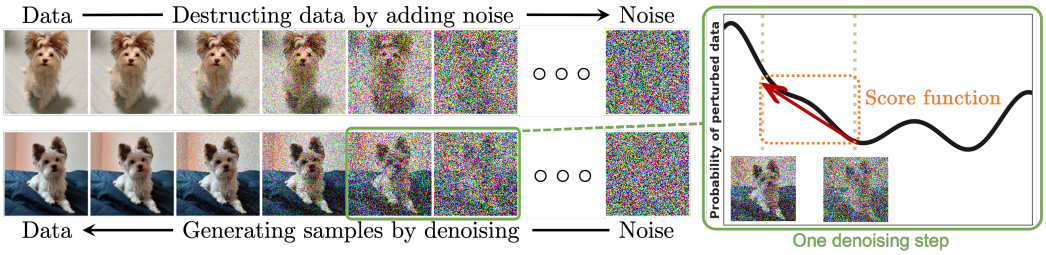


Figure 2.1.: Illustration of the forward noising and reverse denoising process in diffusion models, from [62], p. 6

Latent diffusion models reduce this cost by applying the diffusion process in a compressed latent representation instead of directly on RGB images. The image is first encoded into a lower-dimensional latent space by an autoencoding model, then diffusion is performed in that latent space, and finally the result is decoded back into image space. This setup keeps much of the perceptually relevant structure while making training and sampling more efficient. The underlying autoencoding principle is related to latent-variable models such as variational autoencoders, which learn a continuous latent representation together with a decoder that reconstructs data in the original space. This learned encoder-decoder pair is what makes the reduction to a lower-dimensional space described above possible in the first place, since it must preserve enough information to reconstruct the original data despite the compression. [53, 38]

In text-to-image diffusion models, several distinct representation spaces coexist, making it misleading to refer to a single latent space. During the generation process, these representation spaces include the initial noise or seed space, the timestep-dependent denoising states, the autoencoder's compressed image-latent space, the prompt-embedding space used for conditioning, and the internal feature spaces of the denoising network [53, 56, 47, 39]. These spaces are not interchangeable, because they support different forms of variation, control, and interpretation.

2.2. Conditioning, prompt embeddings, and CLIP-based text representations

The relevant point here is that text-to-image diffusion relies on several interacting representational layers rather than on one undifferentiated latent space. Prompt conditioning is one of the most important space of these layers and therefore needs to be understood before latent-space exploration methods can be described precisely.

Conditioning means that generation is guided by additional information beyond random noise. In text-to-image generation, the most common conditioning input is a text prompt.

The prompt is transformed into a representation that the diffusion model can process during denoising, so that the generated image becomes aligned with the given description. [53]

An important background for such text representations is CLIP (Contrastive Language-Image Pre-training). Radford et al. introduce CLIP as a contrastive model that learns image and text encoders jointly from large numbers of image-text pairs. The text encoder maps natural-language input into an embedding space that is aligned with image representations. This makes CLIP-based text representations useful not only for retrieval or classification, but also as a general semantic representation of prompts. [52]

In text-to-image diffusion systems, prompt conditioning is typically richer than a single global sentence embedding. Latent diffusion models use cross-attention, which lets token-level text information influence the denoising process at multiple spatial locations and multiple steps. In other words, the prompt is not only represented once at the beginning. Its encoded content is repeatedly used while the image is being generated. [53]

Recent work also shows that text embeddings are not merely passive inputs. They can themselves function as a semantically meaningful control space. Yu et al. show that manipulations in text embedding space can support controlled edits and reveal interpretable semantic directions, which means that prompt embeddings are not only technical carriers of text but also structured semantic representations [63]. Na et al. extend this perspective further by showing that a fixed prompt embedding cannot maintain optimal text-image alignment throughout the whole denoising process, since the image evolves at each step and the embedding that best matches it can therefore change over time. In DATE (Diffusion Adaptive Text Embedding), text embeddings are updated during sampling so that the conditioning can adapt to the intermediate image state and maintain this stronger text-image alignment [45].

Consequently, prompt embeddings serve two distinct purposes in this work. First, they form part of the conditioning mechanism that directly guides image generation. Second, they provide an analytical representation that can be reduced or transformed for comparison, projection, clustering, and navigation.

2.3. Compact embeddings versus real conditioning

Building on the double role introduced above, this section defines the two sides of that distinction precisely. Real conditioning refers here to the full prompt-dependent representation that directly participates in image generation, and it is contrasted in this thesis with compact embeddings.

Compact embeddings serve a different purpose. They reduce prompts or intermediate representations to fixed-size vectors that can be stored, compared, searched, projected, clustered, and navigated efficiently. This makes them well suited to exploratory analysis and visualization. However, because this reduction discards part of the information contained in the full conditioning pathway, compact embeddings are not generatively equivalent to real conditioning and should not be treated as direct substitutes for it.

The plugin therefore relies on compact embeddings for comparison and interaction, while the produced images still depend on the model's richer internal conditioning structures. Once such compact representations are available, the next question is how they can be transformed into a form that supports visual inspection and interactive exploration.

2.4. Dimensionality reduction with UMAP / Parametric UMAP

For this purpose, dimensionality reduction is used: a high-dimensional representation is projected into a lower-dimensional space that is easier to inspect, visualize, and analyze.

UMAP (Uniform Manifold Approximation and Projection) is a manifold-learning method for dimensionality reduction. It maps high-dimensional data, such as image or text embeddings, into a lower-dimensional space while aiming to preserve important local relations. The method is based on the assumption that the data lies approximately on a manifold. In a first step, UMAP identifies which points lie close to each other in the original high-dimensional space and represents these local relations as a weighted k-nearest-neighbor graph, whose edge weights describe how strongly points are connected within their local neighborhood. In a second step, UMAP creates a low-dimensional representation of this graph. It arranges the points so that the neighborhood relationships of the original high-dimensional space are preserved as closely as possible. Strongly connected points should therefore remain close in the projection, whereas weakly connected or unrelated points should be positioned farther apart. The final layout is obtained by minimizing the difference between the graph defined in the original space and the graph induced by the low-dimensional representation. [44]

A fitted UMAP model can also be used to project new data points into an existing embedding space. However, this transformation is still an approximation based on the learned neighborhood structure and the existing reference data. Nevertheless, this makes standard UMAP useful for adding new points after fitting, but repeated use in a runtime workflow can involve additional overhead and computation costs, because the transformation remains tied to the fitted structure. This overhead scales poorly as the reference dataset grows, since each new projection must be computed against the entire existing set of points.

Parametric UMAP is based on the same underlying objective as standard UMAP but represents the projection differently. Rather than optimizing the low-dimensional coordinates of individual samples directly, it trains a neural network to approximate the mapping from the original feature space to the reduced embedding space. Once trained, this network can project previously unseen data through a forward pass, making the method particularly suitable for workflows in which new prompts, images, or derived embeddings must be added repeatedly after the initial fit. [54]

To summarize this section, UMAP is used here as the main method for projecting high-dimensional embeddings into lower-dimensional spaces, especially two and three dimensions (Figure 2.2 and Figure 2.3), so that the latent space can be visualized and explored. These projections are useful for examining the mentioned local neighborhoods, cluster tendencies, and relationships between samples. Both standard UMAP and Parametric UMAP projections should nonetheless be interpreted as approximate views of the original structure rather than as exact geometric representations of distances in the high-dimensional embedding space.

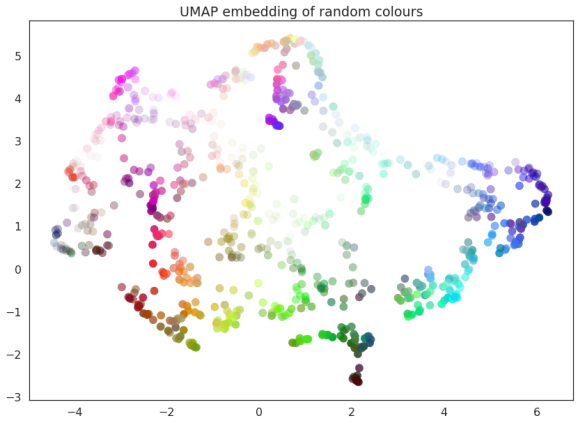


Figure 2.2.: UMAP embedding of random RGBA colors into 2 dimensions; nearby points share similar colors, from [42]

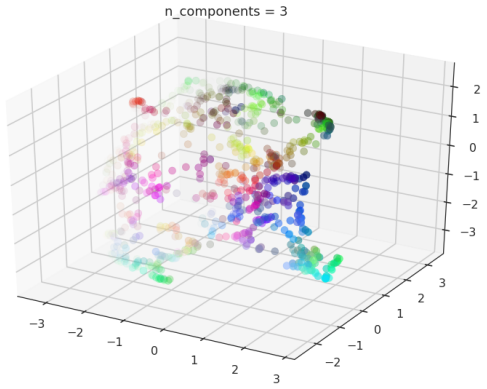


Figure 2.3.: The same random-color data embedded into 3 dimensions instead, from [42]

2.5. Density-based clustering with HDBSCAN

Following dimensionality reduction, clustering is applied to determine whether the embedding space contains dense groups of related samples. Prompt embeddings are not necessarily distributed uniformly and may instead form local structures associated with similar semantic or visual characteristics. HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise) is used to detect these structures without requiring a predefined number of clusters or assuming that clusters have spherical shapes or uniform densities.

HDBSCAN is a hierarchical density-based clustering method that identifies clusters as regions of high point density separated by areas of lower density. Unlike approaches that rely on a single global distance or density threshold, it considers cluster structures across multiple density levels. The method first estimates the local density around each point through its core distance, which is defined as the distance required to reach a specified number of nearest neighbors. These core distances are then combined with the distances between two points to obtain the mutual reachability distance. This transformation reduces the influence of sparse connections, meaning points in dense regions remain closely connected, whereas links through low-density regions receive larger distance values. HDBSCAN uses these distances to construct a minimum spanning tree and derives a hierarchical cluster structure by progressively removing edges in order of decreasing distance, starting with the weakest connections. As the graph separates, groups of points emerge, divide, and disappear at different density levels. The resulting hierarchy therefore represents the persistence of potential clusters across a range of density thresholds. Final clusters are selected according to their stability. A cluster is considered stable when it persists across a broad range of density levels and retains its members consistently throughout that range. Samples that cannot be assigned to a sufficiently stable dense region are labeled as noise. This property is particularly useful for exploratory analysis because points located between clusters or outside clearly structured regions do not have to be forced into an assignment. [5]

The resulting clusters help inspect whether generated samples, prompts, or navigation outputs form coherent local structures.

2.6. Similarity, interpolation, and direction-based navigation

Once a representation space is available, latent-space exploration can be described through three fundamental operations: measuring relationships between points, interpolating between them, and moving along meaningful directions. Together, these operations provide the conceptual basis for navigating and analyzing the space.

The first operation is comparison, which uses similarity or distance measures to quantify the relationship between two representations. Euclidean distance provides a straightforward measure of the direct distance between two vectors. In embedding-based applications, however, cosine similarity is often preferred because it compares normalized vectors and therefore emphasizes their direction in the embedding space rather than their absolute magnitude.

This process, known as L2 normalization, divides a vector by its own Euclidean norm. The resulting unit vector preserves the original direction of v while having a length of one:

$$\hat{v} = \frac{v}{\|v\|}.$$

Cosine similarity is then computed as the dot product of the two L2-normalized vectors:

$$\cos(a, b) = \hat{a} \cdot \hat{b},$$

resulting in a value between -1 and 1 that depends only on the angle between the vectors. CLIP, for example, computes similarities between L2-normalized image and text embeddings and uses their cosine similarity to measure semantic alignment [52].

Once relationships between points can be measured, the next question is how to move between them. Interpolation addresses this problem by constructing intermediate representations along a path connecting two vectors. The simplest method is linear interpolation (LERP), which follows a straight line between the endpoints. Spherical linear interpolation (SLERP) instead follows a path on a hypersphere and is commonly used when preserving vector norms is important.

However, smooth geometric movement does not necessarily produce smooth generative behavior. Samuel et al. demonstrate this for diffusion seed spaces, the space of initial Gaussian noise samples from which reverse diffusion begins, where conventional linear and spherical interpolation can generate poor intermediate states because vector norms influence both image quality and semantic content. Interpolation in generative spaces must therefore account not only for geometry but also for the distribution on which the model was trained [56].

Despite these limitations, LERP and SLERP provide useful baselines for latent-space exploration. Their simplicity and reproducibility make them suitable for examining continuity, semantic drift, and shows which interpolation path begins to produce unstable or incoherent results.

Navigation is not limited to paths between two fixed endpoints. In direction-based navigation, a starting representation is modified by moving it along a selected vector. This vector may encode a semantic attribute, correspond to a locally derived basis direction, or represent the difference between two states. Related approaches appear in several studies

of diffusion models. Kwon et al. identify semantic directions within internal diffusion representations and use them for controlled manipulation, while Park et al. derive local basis directions from the geometry of diffusion latent space and show that traversing these directions can produce interpretable changes [39, 47]. In such methods, both the construction of the direction and the meaning attributed to it shape the resulting navigation behavior.

Together, similarity comparison, interpolation, and direction-based movement form the conceptual foundation for the operations and experiments presented later in this thesis. Similarity enables neighborhood analysis and local comparison, interpolation provides controlled transitions between representations, and direction-based navigation supports targeted movement toward or away from selected properties. However, the literature also indicates that none of these operations can be assumed to behave consistently across all representation spaces, models, or regions of a space.

Chapter 3.

Related Work

Building on the technical foundations introduced in the previous chapter, this chapter surveys existing research relevant to this thesis. It first examines approaches to exploring, controlling, and navigating the latent spaces of generative models, then considers visual and interactive systems designed to make such exploration accessible, and finally covers evaluation strategies for generative systems and their outputs.

3.1. Latent space exploration, controllability and navigation in generative models

Latent space exploration in generative models did not start with diffusion models. Early research examined whether generative models could learn representations in which distinct factors of variation were separated clearly enough to be analyzed individually. Higgins et al. investigate this question in the context of variational autoencoders. They prioritize disentangled representations, where changing one part of the learned representation affects only a single property of the output, over reconstruction quality, the accuracy with which the decoder reproduces the original input. To test this, they vary one part of the representation at a time and observe how the output changes, then analyze the results factor by factor, showing that single dimensions turn out to correspond to changes such as position, scale, or rotation. [30] This establishes a basic exploration pattern that appears again in later work: change one part of the representation in a controlled way and inspect how the output responds.

In research on generative adversarial networks (GANs) [24], this exploration principle became more closely associated with semantic control. A central problem here is no longer whether factors can be separated, but whether meaningful attributes can be manipulated in a predictable way. InterFaceGAN addresses this problem by relating the GAN's latent representation to semantic scores, how strongly an attribute such as age or gender appears in an image measured by a separate classifier, and identifying linear subspaces within that

representation that support controlled editing [57]. SeFa proposes a more unsupervised approach: instead of relying on classifiers and labeled attributes, it factorizes the weights of a pretrained generator directly to find these same kinds of meaningful directions, thereby avoiding manually annotated labels and separately trained classifiers [58]. Together, these methods illustrate a broader development from basic latent-factor traversal toward explicit and interpretable semantic manipulation.

This development is more challenging in diffusion models because, unlike many earlier generative architectures, they do not typically expose an established, semantically organized global latent interface for semantic manipulation. One approach is therefore to identify useful representation spaces within the model itself. Kwon et al. locate such a space in the U-Net bottleneck, the most compressed layer at the center of the denoising network architecture, and show that modifying its representations can produce consistent semantic changes across samples and denoising timesteps [39]. This indicates that control in diffusion models can be understood as navigation through internal representations rather than only as prompt modification.

Other work suggests that semantic manipulation cannot always be reduced to fixed global directions. Meaningful edits may instead depend on the local geometry of the representation space and on the current stage of the denoising process [47]. Following this perspective, Haas et al. study interpretable directions in diffusion-model h-space, the space of the deepest feature maps of the denoiser. They find that these directions tend to capture broad, transferable changes such as pose or gender, whereas an image-specific analysis instead uncovers more localized transformations confined to a particular region, such as the eyes or mouth, which can nonetheless be transferred to edit other images in a similar way [27].

Another part of research focuses less on individual semantic directions and more on the behavior of the complete path between two representations. From this perspective, the key question is not only whether two endpoints can be connected, but whether the intermediate states remain stable, coherent, and meaningful. Studies of diffusion seed space illustrate this problem clearly. Conventional interpolation methods such as Linear Interpolation and Spherical Linear Interpolation may pass through poor intermediate regions in this space, leading to low-quality images with poor semantic content at those intermediate steps [56]. Related instability has also been observed in diffusion latent spaces more generally, where even small additive changes can cause noticeable variations in the generated output [25].

This path-oriented view is also reflected in diffusion editing methods. Lee et al. gradually shift the prompt conditioning from a source prompt embedding to a target prompt embedding during denoising and use the resulting noise difference to edit only the relevant image regions, leaving the rest unchanged [40]. At a larger analytical scale, Zeng et al. examine h-space representation by comparing how it shifts across large, systematically varied collections of prompts and captions, for example when a specific word is added, removed, or

exchanged. Because these shifts are tied directly to the prompt text, the approach scales to uncover broader patterns, such as societal biases the model associates with particular concepts [64]. Taken together, these studies indicate that latent-space navigation should be evaluated not only by its endpoints, but also by the continuity and stability of the entire trajectory.

Controllability can also be achieved without directly traversing a latent space, either by modifying the conditioning signal or by introducing lightweight changes to the model parameters. One line of work treats prompt-related representations themselves as the editing space. Manipulations of text embeddings can already support controlled edits and reveal semantic directions inside the embedding representation [63]. This idea is extended further by adaptive conditioning methods such as DATE, which update the embedding itself throughout the denoising process rather than relying on a fixed one, as already discussed in Chapter 2 [45]. A second line of work intervenes more directly in the model parameters. Gandikota et al. introduce Concept Sliders, which represent concepts as low-rank parameter directions and enable precise, composable control over attributes in images generated by diffusion models [22].

3.2. Visual and interactive exploration tools for generative systems

Research on visual and interactive tools for generative systems has developed largely independently from work on controllability. Instead of focusing on how a model can be steered, these systems examine how users can inspect, understand, and explore generative processes. GAN Lab is an early and influential example. It presents a browser-based interactive environment for learning and experimenting with GANs through visualizations of model structure, sample distributions, and training dynamics. Its contribution is to show that generative models become more accessible when their training process, intermediate behavior, and outputs are exposed through interactive visual representations. [36]

Other works expands this idea from model understanding to the exploration of large output spaces. Dream Lens addresses the problem that generative systems can produce hundreds or thousands of output candidates that are difficult to review systematically. Its interface combines visual browsing, metadata inspection, and example-based ranking to help users navigate a large collection of generated outputs. [41]

More recent tools move closer to direct latent-space interaction. Browsing the Latent Space introduces an interactive environment for exploring latent spaces in a three-dimensional generative setting. It combines dimensionality reduction, interpolation, and visual navigation to make the latent space itself part of the user interface [4]. A related example in diffusion

models is the latent-space explorer developed by Zhong and Huang, which integrates custom latent operations into Stable Diffusion to support conceptual and spatial exploration within a creative workflow [66]. These systems transform latent space from an internal model representation into an explicit surface for interaction.

Together, these tools illustrate a broader development in generative-model interfaces. Earlier systems primarily focused on explaining model behavior and training dynamics, while later systems emphasized the organization and inspection of large output spaces. More recent approaches treat latent representations themselves as interactive environments for exploration, an idea that directly inspired the development of the NELS plugin presented in this thesis.

3.3. Evaluation strategies for latent navigation and generated outputs

This section reviews existing approaches for evaluating generative systems and their outputs and briefly explains what they are meant to measure. It then narrows this broader range of approaches to the metrics and evaluation procedures applied in this thesis.

Evaluation in this area addresses two related but distinct questions. The first concerns the generated images themselves, for instance whether they look plausible or match the prompt. The second concerns how a navigation method behaves across multiple steps, for instance whether a path stays smooth, drifts semantically, or becomes unstable. Since these two aspects capture different qualities of the generation process, no single universal measure covers both, and the literature therefore relies on a range of evaluation approaches.

A standard reference point in generative image evaluation is the Fréchet Inception Distance (FID), introduced by Heusel et al. to compare the distributions of generated and real image sets within a learned feature space. A lower FID score is usually interpreted as a sign of better overall realism and diversity in the generated images. FID is therefore useful when two larger image sets should be compared at a global level. However, it does not directly measure whether an interpolation trajectory is smooth, whether consecutive steps change abruptly, or whether a direction-based walk remains semantically stable. FID thus provides a useful general evaluation measure, but it does not capture the sequential behavior that is central to latent-space navigation. [29]

For pairwise perceptual comparison, the literature commonly relies on feature-based similarity metrics that approximate human perceptual judgment. Zhang et al. show that deep perceptual representations often correspond more closely to human judgments rather than with pixel-wise differences or conventional structural measures. LPIPS (Learned Perceptual Image Patch Similarity) compares two images using feature activations extracted from

multiple intermediate layers of a pretrained neural network. The differences between these activations are combined into a single perceptual distance, so similarity is assessed in terms of visual feature representations rather than exact pixel correspondence [65]. This makes LPIPS particularly suitable for comparing consecutive images in a sequence and estimating the magnitude of visible change between neighboring steps. In latent-space navigation, it can therefore support the analysis of continuity, abrupt transitions, and local instability.

Another class of metrics evaluates semantic alignment between text and image. CLIPScore estimates how well an image corresponds to a textual description by measuring the cross-modal similarity between their embeddings, without requiring a reference image [28]. ImageReward extends this approach by training a reward model on human preference data for text-to-image generation. It combines text and image features from BLIP (Bootstrapping Language-Image Pre-training), a pretrained vision-language model, through cross-attention and maps the resulting representation to a scalar reward score using a multilayer perceptron, with higher values indicating outputs that are more likely to align with human preferences [61]. These metrics are particularly useful when evaluating whether a generated image remains consistent with an intended prompt or semantic concept.

At the same time, the literature also shows that automatic metrics alone are not enough. Otani et al. show that commonly used measures such as FID and CLIPScore may not fully align with human perception and can become increasingly difficult to interpret as model quality improves. They therefore propose a standardized human-evaluation protocol in which individual text-image pairs are assessed according to criteria such as visual fidelity and text-image alignment [46]. Their work highlights the importance of structured qualitative assessment as a complement to automatic metrics, particularly when continuity, artifacts, semantic plausibility, and interpretability must be considered together.

The present thesis does not apply every metric discussed above, as the available evaluation methods cover a broader scope than this work can reasonably address. Its objective is not to benchmark text-to-image models in general, but to evaluate a latent-space exploration plugin and the results produced by its workflow. The evaluation therefore concentrates on methods that correspond directly to the system's outputs and research questions. These include neighborhood- and cluster-based measures, such as k-nearest-neighbor agreement, within-cluster similarity, and the proportion of points classified as noise. Trajectory behavior is assessed using LPIPS together with structured qualitative inspection of generated images and their corresponding graph trajectories. This is a methodological choice for the focused scope of this thesis, because these measures make it possible to assess the plugin's graph structure and navigation outputs directly, whereas broad model-level benchmark scores address a different evaluation objective.

Chapter 4.

System Design, Implementation, and Experimental Setup

Chapter 3 showed that prior research typically treats semantic latent-space navigation, interactive exploration interfaces, and evaluation methods as largely separate areas. Comparatively little work combines reproducible tooling, graph-based exploration, and structured empirical analysis within a single workflow. This chapter presents the system developed to address that gap and describes the components of the exploration environment that were designed and implemented specifically for this thesis, based on the concepts introduced in Chapter 2.

4.1. Requirements for the exploration environment

The gap identified above, leads to four requirements for the exploration environment. First, prompt datasets and their compact embeddings must be created and extended through a reproducible process without requiring manual data management. This ensures that the empirical analyses presented in later chapters are based on a consistent and traceable dataset foundation. Second, the resulting embeddings must support projection and clustering, and both the resulting structure and the outputs of navigation operations must be available for interactive inspection rather than only as static results. Third, the environment must provide multiple navigation methods, including interpolation and direction-based movement, with explicitly defined and recorded parameters. Their outputs must remain traceable to the source points and settings from which they were generated. Fourth, the implementation must neither modify the diffusion model's architecture nor the ComfyUI core. Navigation results must also be transferable back into ComfyUI's standard image-generation pipeline so that they can be rendered as images rather than examined only as numerical representations.

The remainder of this chapter addresses these requirements. Sections 4.2 and 4.3 introduce the selected technologies and component architecture, primarily addressing the fourth requirement, while also touching on the interactive-inspection part of the second requirement.

Section 4.4 presents a deeper understanding of the interaction design derived from the second and third requirements. Section 4.5 defines the model, encoder, and prompt sources used throughout the thesis. Section 4.6 describes the dataset-construction pipeline, including prompt acquisition, encoding, projection, clustering, and storage, thereby addressing the first and second requirements. Section 4.7 explains the runtime session architecture that connects the frontend and backend. Section 4.8 introduces the navigation operations and their integration into the image-generation workflow, addressing the third and fourth requirements. Finally, Section 4.9 discusses the resulting design trade-offs and the validity limitations of the implementation.

4.2. Technology and platform selection

Two requirements identified in the previous section directly influence the technological and platform decisions made in this work. First, the plugin must integrate with ComfyUI's existing image-generation pipeline without retraining the underlying model or modifying ComfyUI's core. Second, it must support interactive exploration of embeddings and navigation results rather than providing only static visualizations. To meet these requirements, Python, TypeScript, ComfyUI, and the supporting libraries were adopted and used as existing technologies.

ComfyUI was selected as the host platform because this thesis examines the latent space of an existing diffusion model rather than introducing a new generative architecture. The open source platform is described by its maintainers as "The most powerful and modular diffusion model GUI, api and backend with a graph/nodes interface" [9]. During the evaluation process, its codebase was examined with the assistance of DeepWiki, an AI-supported tool for exploring software repositories [17]. A representative example of this tool interaction is provided in Appendix 1. A central selection criterion was the availability of a complete image-generation pipeline. The chosen platform needed to provide model loading, conditioning, and sampling functionality so that the implementation could focus on latent-space navigation and analysis rather than reproducing established generation infrastructure. ComfyUI satisfies this requirement and also provides an extension mechanism through which custom nodes and frontend components can be added without modifying its core source code [13]. This allows navigation results to be integrated directly into the same pipeline that ultimately converts them into generated images. ComfyUI also provides an established environment in which the resulting plugin can potentially remain usable and extensible beyond the scope of this thesis. Alternative approaches, including a standalone application with a separate generation backend or integration into another web interface, were considered but not pursued. Both would have required duplicating actively maintained

generation functionality without providing a clear advantage for the navigation and analysis methods investigated in this work.

ComfyUI is available in several distribution formats. The Desktop application provides a launcher for managing one or more ComfyUI installations [10], while the portable distribution is a self-contained package that includes its own Python environment and can be extracted and executed directly [14]. A hosted cloud version is also available, but it is not considered in this thesis because the implementation and testing of the plugin require access to a local extension environment.

Development was carried out on Windows using the portable distribution. This setup provides the needed direct access to the filesystem and, in particular, to the `custom_nodes` directory in which the plugin is installed. The underlying user interface remains the same across the Desktop and portable distributions, but it is presented differently. In the Desktop application, the web interface is embedded within the application window. In the portable setup used for this project, ComfyUI runs as a local web server, and the interface is opened separately in the user's browser through localhost.

Python is used as the primary backend language because ComfyUI's node-execution system is implemented in Python [11]. Implementing the encoding, dataset construction, projection, clustering, and navigation operations in the same language allows the plugin to integrate directly with ComfyUI and it allows the use of PyTorch for tensor-based computation. It also provides access to the Python-based machine-learning ecosystem required by the projection and clustering methods introduced in Chapter 2. The implementation further relies on Hugging Face's `datasets` library for loading prompt corpora and on `transformers` for language models used during cluster annotation [20, 21].

For the interactive frontend, TypeScript was selected because ComfyUI's extension mechanism for custom panels and widgets is based on JavaScript and TypeScript. The source code is compiled with Vite into a JavaScript bundle that can be loaded by the browser. Plain JavaScript was considered as an alternative but rejected because the graph panel manages increasingly complex states, including selected nodes and points, trajectory settings, operation parameters, and data exchanged between the frontend and backend. TypeScript's static type checking helps detect inconsistencies in these data structures during development and reduces the risk of state-management and interface errors as the frontend evolves.

Together, these three technology choices satisfy the requirement of integrating the exploration environment directly into an existing and unmodified image-generation system while also providing an interactive interface for inspecting embeddings and navigation results.

4.3. NeLs: an external ComfyUI plugin

The previous section established that NELS can be integrated into ComfyUI through its extension system without modifying the ComfyUI core. This section describes how the plugin implements this integration and how its internal components are organized.

ComfyUI discovers custom nodes by scanning the `custom_nodes` directory noted above for Python modules that expose a `NODE_CLASS_MAPPINGS` dictionary. When a module is imported successfully and provides this mapping, the registered classes become available as nodes in the workflow editor [12]. NELS follows this convention in its top-level initialization module. The module collects the node classes defined throughout the plugin into a single mapping, registers the HTTP routes required for session management, and exposes the directory containing the frontend assets through `WEB_DIRECTORY` [12]. As a result, the complete plugin can be installed within a single directory under `custom_nodes` and is discovered automatically when ComfyUI starts. No modification of the ComfyUI source code is required.

Below this entry point, the implementation is divided into five components with distinct responsibilities:

- **`nodes/`** contains the classes exposed directly in the ComfyUI workflow editor. These classes define the required input and output interfaces for components such as dataset loading, session handling, and graph operations. They remain intentionally thin and delegate the underlying computation to the reusable implementation in `core/`. From the user's perspective, the classes in this directory behave like standard ComfyUI nodes and can be connected and executed within an ordinary workflow.

- **`core/`** contains the main computational and semantic logic of the plugin. This includes prompt encoding, dataset construction, projection and clustering, and the six operation families introduced in Section 4.8. The code is implemented as ordinary Python functionalities and does not depend on being executed through a ComfyUI node. This separation allows the computational logic to be tested and reused independently of the ComfyUI runtime.

- **`server/`** provides the connection between the frontend and the backend session state. It defines the required HTTP routes and maintains an in-memory session store within ComfyUI's existing server process rather than starting a separate server. This additional communication layer is necessary because the graph panel requires continuous interaction between the browser and the backend beyond the input-output flow of a single node execution. ComfyUI categorizes this type of extension as a connected client-server integration [13].

- `src/` contains the interactive graph-panel frontend, implemented in TypeScript and Preact. Vite compiles these sources into the JavaScript files loaded by the ComfyUI client. The frontend architecture and its interaction model are discussed further in Sections 4.4 and 4.7.
- `scripts/` provides command-line entry points for offline processing. These scripts access the functionality in `core/` directly and do not depend on the node definitions, session server, or a running ComfyUI web client, though they still import ComfyUI's own model-loading and CLIP-encoding classes directly, so a ComfyUI installation must still be present on the machine running them. They are primarily used for batch dataset creation and related preprocessing tasks.

This separation supports the architecture required by Requirement 4. Only the node and server layers interact directly with ComfyUI-specific interfaces, while the central computation remains isolated in a reusable core.

The client-server architecture also introduces a practical limitation. According to the ComfyUI documentation, extensions that depend on direct communication between the browser client and the server are not compatible with ComfyUI's plain API mode [13]. The interactive graph panel therefore requires the standard ComfyUI web client. While individual server-side nodes may be executed headlessly, interactions such as point selection, operation configuration, and live graph updates cannot be controlled solely through the API.

4.4. UI/UX design rationale

This section discusses two central UI/UX design decisions and the interaction features that result from them: first, why the exploration interface is embedded directly within ComfyUI rather than implemented as a separate application, and second, why the dataset is presented as an interactive graph instead of through a form- or table-based interface.

Embedding the panel within ComfyUI follows directly from Requirement 4 and the architecture described in Section 4.3. Since the results of the navigation process must be transferred back into ComfyUI's standard image-generation workflow, placing the exploration interface alongside the node graph avoids switching between a separate analysis tool and the generation environment. The panel is implemented with Preact [50], a lightweight alternative to React that provides a similar component model and broad compatibility with the React API, together with `@preact/signals` [51], a library for reactive state management. It provides separate graph and log views, while a resizable sidebar contains the operation controls, results, and session information. The selected sidebar width is preserved between sessions, allowing the interface to adapt to different tasks without introducing unnecessary interface elements.

The dedicated panel was not the initial interface design. ComfyUI also supports custom widgets rendered directly within a node on the workflow canvas, and this in-node approach was first considered because it would have integrated the exploration interface even more closely with the graph workflow. However, it became unsuitable as the performance and interaction requirements expanded. Displaying a live graph containing hundreds or thousands of points, together with panning, zooming, hovering, multi-selection, drag selection, and point-specific context actions, proved impractical within the limited dimensions, layout constraints, and event-handling environment of a single node. Once the full scope of the required interaction became clear, a dedicated panel offered the more appropriate solution by providing independent screen space, flexible resizing, and event handling separate from the node-graph canvas.

The graph-based interaction model was inspired by GitNexus, a tool that represents the structure of a codebase as a navigable knowledge graph of related code entities [48]. No source code or interface components from GitNexus were reused. Only the broader interaction concept of exploring a large structured space through spatial relationships rather than by scrolling through a linear list was adopted. Before implementing this approach, the GitNexus codebase was examined with the support of DeepWiki. A representative example of this tool interaction is provided in Appendix 1. In NELS, the points represent prompts and their embeddings rather than code entities, but the underlying interaction problem remains comparable: users must navigate a large collection of related items through their spatial proximity and connections.

Since the dataset can be projected into either 2 or 3 dimensions, the panel provides both a two-dimensional and a three-dimensional graph view. The 2D view is implemented with Sigma.js [34], a WebGL library for rendering large graphs in the browser, together with Graphology [49] for the underlying graph data structure, while the 3D view is built with Three.js [60], a general-purpose library for rendering 3D scenes in the browser. Despite relying on different rendering technologies, both views use the same interaction model. Clicking selects a point, modifier-assisted clicking extends the current selection, and shift-dragging creates a selection area that includes all enclosed points. Hovering displays a tooltip with the corresponding point information, while right-clicking opens a contextual action menu. Maintaining the same interaction vocabulary across both views ensures that switching between 2 and 3 dimensions changes only the visual representation, not the way the user operates the interface.

Operation outputs are integrated directly into the existing graph rather than presented in a separate results view. Trajectory steps produced by interpolation or direction-based operations, as well as variant points generated by the Consistency Check, appear alongside the original dataset points. They are distinguished visually through a different texture, a highlight ring, and a subtle animation, but otherwise support the same interactions. This

includes selection, hovering, and context-menu actions. Internally, these temporary points are identified through a dedicated prefix.

Two additional interface elements support interpretation of the displayed structure. The cluster legend lists each detected cluster together with a color indicator, a representative prompt, and the number of associated points. Selecting a legend entry emphasizes the corresponding cluster while dimming the remaining graph. The legend can also report how many generated result points are assigned to a cluster, even when none of that cluster's original samples are included in the currently loaded subset. For Semantic Axis Projection, a separate axis legend displays the human-readable labels of both semantic poles alongside the numerical projection score. This ensures that the resulting values remain connected to their intended semantic interpretation.

The panel includes a purely cosmetic idle animation, namely a subtle and continuous jitter of point positions, so that the graph does not look static and users are encouraged to interact with it rather than treat it as a passive tool. This choice comes with a deliberate limitation: when animation is enabled, points move smoothly between states, but the intermediate positions shown during this movement are not geometrically exact. The interface communicates this limitation explicitly through the notice "animate ON - positions not exact," reducing the risk that a visually smooth transition is interpreted as a precise trajectory.

The resulting interface prioritizes functional interactivity and a consistent interaction model over visual refinement, as shown in Figure 4.1 and Figure 4.2. Since it was developed as a research prototype for investigating latent-space navigation rather than as a production-ready application, its visual design and overall interface polish were not prioritized and remain open for future improvement.

To conclude this section, the frontend was developed with the assistance of Claude Code using the Claude Sonnet 4.6 model. Development was guided by a written specification file that defined the intended architecture, components, and interaction behavior. Individual implementation details were then refined iteratively through follow-up prompts. The specification is included in Appendix 2 to document and support the development process.

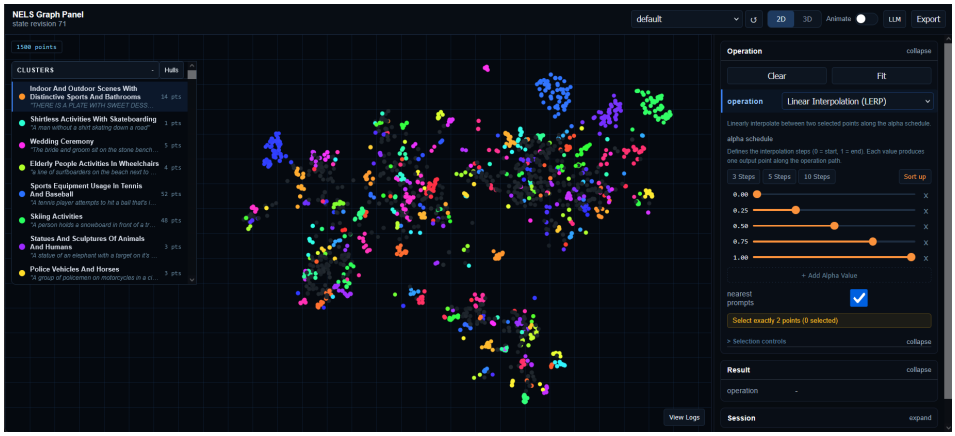


Figure 4.1.: The NELS graph panel in 2D view: 1500 points colored by cluster, the cluster sidebar with legend, and the operation controls

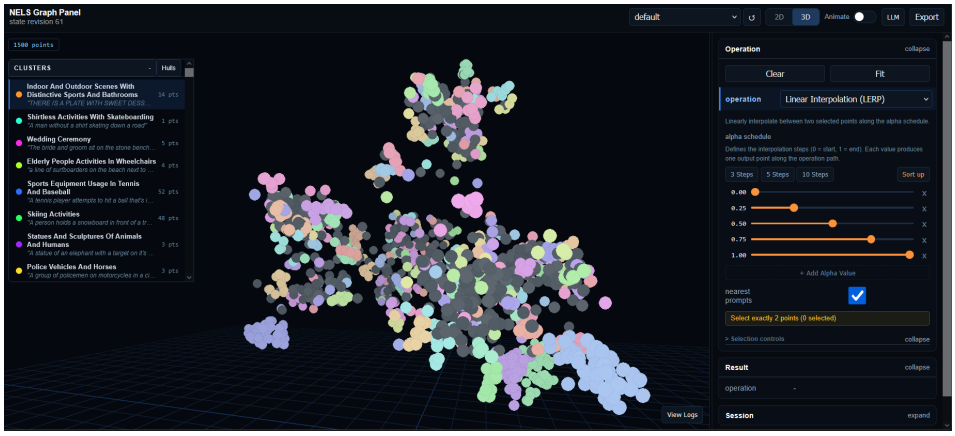


Figure 4.2.: The same session in 3D view

4.5. Model, encoder, and prompt-source selection

Having described how the panel presents results and supports user interaction, the following section introduces the model, encoder, and dataset choices that determine the content displayed within it. All experiments in this thesis use Flux.2 Klein, an open-weight rectified-flow model with approximately four billion parameters that supports both text-to-image generation and image editing. Text conditioning is provided by its associated Qwen-based encoder [3]. Both components are loaded through ComfyUI-compatible `.safetensors` conversions distributed by the ComfyUI community [8].

The availability of open weights was the primary selection criteria for the models, because the investigation requires local access to the model and the ability to inspect and repeatedly execute it without relying on a closed external API. At the beginning of the implementation period, Flux.2 Klein was selected as the most suitable open-weight model available for these

requirements. Since the field develops rapidly, however, newer and more capable open-weight models may become available after the publication of this thesis.

Both the diffusion model and the text encoder are used in quantized form: FP8 (8-bit floating point) for the diffusion model and FP4 (4-bit floating point) for the encoder, which reduces both memory usage and compute during inference. This configuration was selected because it could be executed reliably on the available hardware, namely an NVIDIA GeForce RTX 2070 with 8 GB of VRAM, paired with an AMD Ryzen 5 3600 6-core processor and 32 GB of system RAM. Full-precision checkpoints would have been preferred if sufficient computational resources had been available. Quantization may influence the precise numerical properties of the representations examined in later chapters, although it is not expected to fundamentally alter the qualitative structures investigated in this thesis.

The datasets examined in this thesis comprise four large prompt corpora representing two qualitatively different text distributions, complemented by a small controlled label set for comparing projection strategies. Natural image-caption prompts are drawn from CaptionEmporium's LLaVA-generated short captions for Conceptual Captions CC12M [6], a large-scale corpus of web-sourced image-text pairs [7], and from COCO Captions [35]. Prompts that more closely resemble the descriptive, "AI-art"-style input commonly used in text-to-image systems are taken from daspartho's Stable Diffusion Prompts dataset [15] and Gustavosta's Stable Diffusion Prompts dataset [26]. Including both distributions makes it possible to examine whether the observed navigation behavior remains consistent across different forms of prompt language rather than being specific to a single writing style. A fifth, much smaller dataset is also used: the 1.000 ImageNet-1k class labels [33], selected because they provide a compact, clearly defined, and well-understood label set for the projection-strategy comparison described in Section 4.6, in contrast to the long and heterogeneous natural-language prompts of the four corpora above.

The ImageNet labels were not initially prepared through the pipeline's generic Hugging Face input mechanism because the dataset's label column stores each class as an integer index, through a `ClassLabel` feature, rather than as usable text. The generic loader has no step to resolve that index into its corresponding class name. Instead, a one-off preprocessing script streamed the training split of the ImageNet-1k dataset through the Hugging Face `datasets` library and retrieved the decoded class names from the label feature's list of class names rather than using the stored integer indices. Each class name was then converted into a prompt of the form "a photo of {name}" and written to a local text file containing one prompt per line. This file was subsequently processed through the pipeline's standard local-file input path.

As described in the next section, the dataset-construction pipeline treats prompt sources generically rather than being tied to any specific corpus. Combined with the separation between the node interface and the underlying processing logic discussed in Section 4.3, this

design allows both the prompt source and the model implementation to be exchanged without restructuring the pipeline. The same construction and analysis workflow could therefore be applied to all four large corpora and to the differently formatted ImageNet label file.

4.6. Data acquisition and processing

This section describes the dataset-construction pipeline, including how the plugin collects prompts, encodes and compacts them, projects and clusters the resulting representations, stores the dataset, extends it at a later stage, and assigns labels. The subsection concludes by differentiating between the two representation spaces on which the pipeline operates.

A dataset can be created either through the prompt-encoding node or through the corresponding command-line interface, which executes the same processing pipeline without requiring an active ComfyUI session. Both paths intentionally use the same ComfyUI-native model classes and compatible model weights. As a result, they produce byte-for-byte identical compact embeddings. Using the original model release instead would not affect projection, clustering, or interpolation, since these operations work only with vector representations, but the resulting incompatibility would only become noticeable during image generation. At that stage, the original model's conditioning tensors would not match the shape and conventions expected by ComfyUI's sampling pipeline, preventing the generated conditioning from being processed correctly. Using the ComfyUI-compatible model implementation during dataset construction therefore ensures that every dataset remains compatible with the later image-generation pipeline.

Prompt records are first collected into a single ordered list from as many as two sources: text entered directly in the node and an optional external source. The external input may be either a local text file containing one prompt per line or a streamed Hugging Face dataset selected by repository, split, and configuration where required, optionally combined with a template that merges dataset columns into the prompt text and copies additional metadata into each record. The collected records are then cleaned, filtered by minimum and maximum word count, and optionally deduplicated. Because the external source is capped before deduplication, enabling it can yield fewer records than requested. The final size limit, applied after merging inline and external, then trims excess from the external records first, since inline prompts are retained preferentially.

Each prompt record is encoded exactly once. Tokenization first converts the prompt into a token sequence, after which a single encoder forward pass returns the full sequence-conditioning tensor and, where supported, a pooled representation. From these outputs, one fixed-size compact vector is derived and stored for the prompt. If a pooled representation is available, it is converted to floating-point format. If it is not already one-dimensional,

its leading dimension is removed to produce a one-dimensional vector. For example, a pooled output with shape `[1, D]` becomes a vector of shape `[D]`. If no pooled representation is available, the leading batch dimension of the sequence tensor is removed and the remaining token vectors are averaged:

$$e_{\text{prompt}} = \frac{1}{n} \sum_{i=1}^{n} e_i,$$

where n is the number of token positions and e_i is the representation of the i-th token.

The Qwen-based encoder used throughout this thesis does not provide a pooled output. Consequently, all datasets created for this work use mean pooling over the sequence and record this fallback strategy in their metadata as `embedding_strategy: mean_seq_fallback`. The pooled-output branch remains available for other encoders but is not used by any dataset evaluated here. Mean pooling was chosen for its fixed-size, length-independent representation that lets every token position contribute to the representation, though it is treated as a weaker representation than a dedicated pooling head.

For the encoder described in Section 4.5, the full conditioning tensor of a single prompt has shape `[1, 512, 7680]`, corresponding to 512 token positions with 7.680 features each. Stored as 32-bit floating-point values, this requires roughly 16 MB per prompt and approximately 8 TB for a dataset of about 550.000 prompts. The corresponding compact representation has shape `[7680]`, requires roughly 31 KB per prompt, and occupies approximately 16 GB for the same dataset size. The storage requirement is therefore reduced by a factor of exactly 512, corresponding to the number of token positions collapsed into one vector. This reduction is why dimensionality reduction, clustering, and nearest-neighbor search operate on the compact vectors rather than the full conditioning tensor.

The current pipeline processes prompts sequentially, using one encoder forward pass per prompt since pooled representations are requested individually, and requires every prompt in a dataset to use the same representation strategy. Encoding is stopped with an explicit error if some prompts would use native pooling while others require mean pooling, preventing incompatible representations from being silently combined in the same embedding matrix. This situation does not arise in practice, since pooling availability is a property of the encoder rather than of individual prompts and a dataset build always uses one encoder.

Finally, sample identifiers are assigned in ascending order during dataset creation, ensuring that they remain stable and searchable across later operations. The dataset's metadata, saved into `dataset.json`, describes both how the dataset was built and what its clustering labels are. Saved information includes prompt source, template, and metadata columns used to build the dataset, the encoder and model used to produce the compact vectors, and the projection strategy and umap parameters used to produce the two- and three-dimensional coordinates as well as how the clusters get computed. This metadata is used to inspect the

dataset's construction and to ensure that it can be rebuilt or extended later with the same parameters.

Once compact vectors have been generated for all prompts, the resulting embedding matrix is used to train the projection models. Separate UMAP models are fitted for the two- and three-dimensional views displayed in the graph panel. Both models operate on the same complete compact embedding matrix and use the same UMAP parameter settings stored in the dataset metadata. Fitting the model to the data and computing the display coordinates for the existing samples happen together. The resulting coordinates are stored as static arrays and later used to render the dataset. The fitted UMAP models are retained separately so that they can subsequently project new points and operation trajectories into the same coordinate spaces. How these reusable projection models are stored and applied depends on the selected projection strategy, described later in this section.

Cluster assignment is performed neither in the two- or three-dimensional display space nor directly in the full compact embedding space. HDBSCAN identifies clusters by estimating local density from pairwise distances. In the original 7.680-dimensional space, however, distances tend to concentrate: the relative difference between a point's nearest and farthest neighbors decreases as dimensionality increases, making density estimates less reliable [1]. Density-based clustering methods such as HDBSCAN are therefore also affected by the challenges of high-dimensional data in practice [43]. To address this issue, a third UMAP model is fitted specifically for clustering. Unlike the display projections, which reduce the embeddings to 2 or 3 dimensions, this model reduces them only to a moderate dimensionality, between 10 and 20 components for the datasets used in this thesis. This follows the recommendation in the UMAP documentation to preserve more cluster-relevant structure when dimensionality reduction is used as a preprocessing step for clustering [43]. The resulting coordinates form a separate cluster space that is used exclusively as the input to HDBSCAN.

HDBSCAN is then fitted on the reduced cluster-space coordinates using user defined cluster settings. Each point receives a cluster label, where a value of -1 denotes noise, together with a membership strength that reflects HDBSCAN's confidence in the assigned cluster. The clustering model is also configured to retain the information required for approximate prediction, allowing newly introduced points to be assigned to an existing cluster without refitting the complete model.

The clustering projection, the two display projections, and the HDBSCAN model are each fitted once on the complete dataset, in what NELS refers to as the canonical fit. This fit forms the fixed reference against which later runtime projections and cluster predictions are evaluated. How these fitted components are preserved and subsequently used to position and assign genuinely new points depends on the selected projection strategy, which is described in the following section.

The `classic` strategy stores the fitted cluster-space, two-dimensional, and three-dimensional UMAP models directly, together with the HDBSCAN clusterer, each serialized separately with `joblib` and reused at runtime. When new points must be displayed, the corresponding UMAP model transforms their compact embeddings into the previously learned projection space, yielding approximate coordinates since they are inferred rather than obtained by refitting UMAP with the new points included. For cluster assignment, the new point is transformed into the learned cluster space and assigned via HDBSCAN's approximate prediction, which can only select among clusters from the canonical fit rather than create new ones. The `classic` strategy is the most direct and transparent of the three approaches, reusing the canonical fit's models unchanged rather than introducing a separate runtime approximation. Consequently, runtime projection depends on the fully fitted UMAP model, so transformation time and model size increase with the size of the original dataset, limiting the strategy's scalability.

The `parametric` strategy uses Parametric UMAP for the same three projection tasks: the two-dimensional display space, the three-dimensional display space, and the higher-dimensional cluster space. Instead of storing a non-parametric UMAP model, each projection is represented by a neural network trained with the TensorFlow/Keras backend introduced in Section 4.2. The network learns a direct mapping from the compact embedding space to the respective target space. Each trained projection is stored in a separate directory containing the neural encoder (`encoder.keras`) and the reducer's own saved state (`model.pkl`), while the HDBSCAN model is saved alongside them as `hdbscan_model.joblib`. At runtime, new points are projected by passing their compact embeddings through the trained encoder. Cluster assignment follows the same principle as for the classic strategy. The `parametric` strategy's main disadvantages are the additional dependencies and training complexity introduced by the Parametric UMAP and TensorFlow/Keras stack.

The `two_pass` strategy is the most complex of the three: it performs the same canonical fit as the `classic` strategy, but once the cluster assignments and display coordinates have been produced, the fitted UMAP models are discarded rather than retained. A second training stage then creates smaller projection models specifically for runtime use. By default, these models are fitted on a deterministically selected subset of 20.000 points and are stored separately for the two- and three-dimensional display spaces. The selected samples are also recorded to make the composition of the runtime subset inspectable. Rather than drawing a uniformly random sample from the whole dataset, the procedure ensures that each cluster identified during the canonical fit is represented in the subset in proportion to its size: a cluster that makes up, for instance, 10 percent of all clustered points also contributes about 10 percent of the subset. This also guarantees that even very small clusters contribute at least one point, which a purely random sample could otherwise skip entirely. If the proportional allocation cannot fill the requested subset size, the remaining slots are drawn from noise points instead. Within each cluster, points are sampled across the full range of

membership strengths so that both strongly associated core points and weaker boundary points are represented. New points are subsequently projected using these smaller runtime models instead of the discarded canonical models. Because they were fitted on only a subset of the complete dataset, they require less time and memory for transformation. The resulting coordinates are therefore a more approximate estimate of where the point would have been placed by the full canonical projection.

In the `two_pass` strategy, cluster assignment for a newly introduced point is performed without using a cluster-space UMAP model. Instead, the operation builds a FAISS index over the complete compact embedding matrix and uses cosine similarity to retrieve the 32 most similar labeled samples. FAISS is a library designed for efficient similarity search in high-dimensional vector spaces [18].

Their canonical cluster assignments are then combined through a temperature-scaled softmax vote. For the i-th of the $k = 32$ retrieved neighbors with cosine similarity s_i to the new point, its vote weight is

$$w_i = \frac{\exp(\tau \cdot s_i)}{\sum_{j=1}^{k} \exp(\tau \cdot s_j)},$$

using a temperature of $\tau = 12$. Each neighbor's weight is added to the total weight of its own canonical cluster label, and the cluster with the greatest total weight is selected. The point is classified as noise when the winning label represents noise or when the winning label accounts for less than 45 percent of the total vote weight.

Several alternatives based on cluster-space projection were considered but rejected. One possibility was to project the new point through a hypothetical runtime UMAP cluster model and apply the canonical HDBSCAN model's approximate prediction function to the resulting coordinates. This approach would be invalid because the runtime cluster model and the canonical HDBSCAN cluster-space model were fitted independently and therefore do not represent the same manifold. As discussed in Section 2.4, the geometry of a UMAP embedding depends on the data and configuration used during fitting. Consequently, the decision boundaries learned by the canonical HDBSCAN model cannot be transferred directly to coordinates produced by a separately fitted reducer. A second alternative was to train a new cluster-space projection and HDBSCAN model on the same runtime subset. This was rejected for the same reason: a subset-trained projection would establish a different coordinate system and different cluster boundaries, causing predicted assignments to diverge from the canonical labels already stored with the dataset. A third possibility was to estimate the new point's canonical cluster-space position by averaging the cluster-space coordinates of its nearest neighbors and then applying approximate prediction. However, this would assume that proximity in the compact embedding space corresponds closely enough to proximity in the canonical cluster space for the averaged position to remain within the correct cluster.

The FAISS-based voting procedure avoids these problems by keeping the entire assignment process in the compact embedding space and combining neighbor labels rather than estimating an intermediate position. As a result, the `two_pass` strategy uses two separate approximations: one determines where a new point is displayed, while the other independently estimates the canonical cluster to which it belongs.

Each completed dataset directory contains three core files: `dataset.json` (the metadata), `samples.jsonl` (the line-based sample file), and `compact_embeddings.npy` (the full compact embedding matrix). Each entry in the sample file stores a prompt's identifier, text, source information, optional template or metadata fields, and its `cluster_global`, `is_noise`, and `membership_strength` values. The projection outputs take two fundamentally different forms. Static `umap_2d.npy` and `umap_3d.npy` coordinate arrays, generated when the dataset is created, hold the fixed positions of existing samples and are never transformed again. Serialized projection and clustering models are loaded only when genuinely new points must be projected or assigned to a cluster: `umap_model_2d`, `umap_model_3d`, and `umap_model_cluster` alongside `hdbscan_model.joblib`, or the corresponding `runtime_umap_model_2d.joblib` and `runtime_umap_model_3d.joblib` used by the `two_pass` strategy.

Beyond the coordinate arrays and projection models described above, every dataset also carries descriptive labels for its clusters, produced through two mechanisms of increasing cost and quality. An initial cluster label is generated automatically after dataset creation and whenever new prompts are later added to it. For this purpose, all non-noise samples are grouped by cluster, and their prompt texts are split into individual words. Stop words, very short terms, and overly common words are removed, after which light stemming is applied. The most frequent remaining stem is then selected as the cluster's provisional label. Because this procedure evaluates individual words rather than multi-word expressions, it always produces a single stemmed term rather than a phrase: "golden" and "retriever" are counted as two entirely separate candidates rather than as the joint phrase "golden retriever," and whichever single word occurs most often wins. For example, in a cluster containing prompts such as "a golden retriever running through a park", "retriever puppy playing fetch", and "brown retriever swimming in a lake", the term "retriever" stems to and is counted as "retriev" in all three texts, while "golden" appears in only one of them, so "retriev" is selected as the label rather than "golden retriever" or a description of the scene as a whole. Although this method is computationally inexpensive and always available, it provides only a frequency-based placeholder and does not attempt to summarize the cluster's broader semantic content.

More descriptive labels can be generated in a separate optional annotation step, also after dataset creation or after new prompts have been appended. This process reads the stored prompt texts and cluster metadata directly, excludes noise samples, and prioritizes prompts

with high membership strength as the most representative examples of each cluster. It then constructs a term profile containing frequent terms, distinctive terms identified through a TF-IDF-style weighting scheme, and recurring two-word phrases. The weighting favors terms that occur frequently within one cluster but rarely across the remaining clusters [55]. Shared prompt prefixes are removed beforehand, detected from the dataset's template, a list of known prefixes, or a statistical common-prefix fallback, so that common template text does not dominate the analysis. The resulting term profile and a small set of representative prompts are provided to a language model, which proposes a concise label for each cluster. Several refinement rounds compare the clusters with one another to reduce generic or duplicate labels. The final labels are then stored in the dataset metadata and replace the automatically generated frequency-based placeholders. The system-prompt templates for the initial term-profile labeling step and the refinement round are provided in Appendix 3.

New prompts can be appended to an existing dataset by encoding them with the same representation strategy used during its original creation. Their compact vectors are added to the existing embedding matrix, after which the complete projection and clustering pipeline is fitted again on the combined dataset. This includes the cluster-space projection, HDBSCAN, and both display projections. The previous models are not updated incrementally. Before the newly fitted models are saved, all projection artifacts associated with the earlier dataset state are removed, regardless of the selected projection strategy. They are then replaced with the newly generated versions, preventing outdated models, runtime subsets, or projection files from remaining alongside the updated dataset. This full replacement is intentional because newly added samples can change the density structure on which both UMAP and HDBSCAN depend. Reusing coordinates or cluster boundaries calculated before the new samples were introduced could therefore produce misleading results. Appending data consequently has approximately the same computational cost as rebuilding the dataset from the combined records. The dimensionality of the new compact vectors is checked against the existing embedding matrix, and a mismatch causes the process to stop immediately because it indicates that a different or incompatible encoder was used. Since HDBSCAN is refitted on all existing and newly added samples, cluster identifiers are not guaranteed to remain stable. A cluster assigned a particular numerical identifier before the update may represent a different group afterward. For this reason, the cluster assignment, noise status, and membership strength are recalculated and rewritten for every sample, not only for the newly added records. Existing descriptive cluster labels are also cleared and regenerated from the updated cluster composition, regardless of whether they originated from the frequency-based first pass or a prior LLM annotation, ensuring that the dataset remains consistently labeled after the append operation.

The compact embedding space produced by this pipeline corresponds to the analytical representation introduced in Section 2.3, but is not used for image generation. Whenever

an operation produces images, the original prompt text is re-encoded to obtain the full conditioning tensors required by the diffusion model. Interpolation, directional modification, and axis-based operations are performed directly in this conditioning space. The resulting conditioning is converted into a compact representation only after the operation completes, so that it can be projected and displayed in the graph panel. The compact embedding space therefore serves as the navigation and visualization space, whereas the full conditioning space serves as the generation space. These representations are intentionally kept separate rather than using the compact embedding as an approximate replacement for the original conditioning.

For the experiments in this thesis, each of the four large data sources described in Section 4.5 was processed once using the `two_pass` strategy to avoid the scalability limitations of the `classic` and `parametric` strategies discussed above, and examined in detail further in Section 4.9. Dataset creation was limited to 550.000 prompts, with deduplication enabled and without filtering by word count. The 1.000 ImageNet-1k class labels were processed separately using all three projection strategies: `classic`, `two_pass`, and `parametric`. This setup will later be used in Chapter 5 to evaluate how accurately each strategy places new data relative to the canonical fit. Each strategy was applied to the same set of labels, allowing their results to be compared directly without differences caused by varying source samples.

4.7. Runtime session and graph-panel flow

Having described how a dataset is constructed, saved, and optionally labeled, the following section shifts from offline dataset preparation to the runtime session that loads such a dataset and drives the interactive graph panel.

A session begins by determining which saved samples should be loaded. The available selection strategies include loading the first subset, a specified range, a reproducible random subset, or the complete dataset. The selected samples are then combined into a single embedding collection. Each entry of the collection contains the compact embedding, its saved two- and three-dimensional display coordinates, a shortened display label, and metadata consisting of the complete prompt text, needed by later operations, the stored cluster assignment, noise status, and membership strength.

This embedding collection is cached for the duration of the session as the single source of point data. The cache is keyed by a hash of the dataset-loader configuration, avoiding repeated disk reads of the same dataset files when the source remains unchanged. Whenever the frontend requests data for graph rendering, a JSON-compatible view is generated from

the cached collection, containing everything except the embedding tensors, which are needed only for backend computations.

The raw UMAP coordinates are not displayed directly. Before rendering, the frontend maps all axes to a common range from -1 to 1. It first determines the minimum and maximum values of every currently loaded point along each axis and calculates the center of each range,

$$c_x = \frac{\min_i x_i + \max_i x_i}{2}, \qquad c_y = \frac{\min_i y_i + \max_i y_i}{2},$$

with an equivalent c_z derived the same way for the third axis in the 3D view. A single scale factor is then derived from the largest range across all axes,

$$\text{scale} = \frac{2}{\max(\max_i x_i - \min_i x_i,\ \max_i y_i - \min_i y_i)},$$

again including the z-range when mapping the 3D view, and floored at a small minimum span so the scale factor stays finite if all loaded points happen to share the same coordinate. This scale factor is then applied uniformly to every coordinate,

$$x' = (x - c_x) \cdot \text{scale}.$$

In the two-dimensional view, the vertical axis is additionally inverted to match the screen-coordinate system used by the graph-rendering library Sigma,

$$y' = -\big((y - c_y) \cdot \text{scale}\big).$$

Applying one shared scale factor instead of normalizing each axis independently preserves the relative proportions of the projection. For example, a cluster that is twice as wide as it is tall retains this shape after being mapped to the display space. Independent axis normalization would instead stretch or compress the projection until every axis occupied the same range, thereby distorting its geometry.

The transformation is fully invertible, which is required for the Latent Probe operation described in Section 4.8. The backend therefore always operates on raw UMAP coordinates and does not need to account for the frontend display transformation. This mapping was introduced because UMAP coordinates are not bounded to a fixed range. Without it, a fixed-size graph panel would either clip some points or leave excessive unused space around the projection.

Interacting with the panel, such as selecting points or changing operation settings, does not trigger an execution automatically. Instead, each interface change is sent to an in-memory backend store as an incremental state update. Every update advances a revision counter and is distributed through ComfyUI's existing WebSocket connection.

The current interface state is evaluated only when the user queues a ComfyUI execution. At that point, the backend creates an immutable snapshot containing the loaded embeddings, the current selection, the requested operation, a unique execution identifier, and the latest state revision. This queue-time snapshot ensures that an ongoing execution remains independent of subsequent interface changes. Although further user interactions continue to update the session store, they do not alter the inputs of an execution that has already started.

The graph-operation node reads the frozen session snapshot and executes the requested operation. Before performing the corresponding computation, it applies the operation-specific selection rules described in Section 4.8. Once the computation is complete, all operations follow the same final processing sequence. Any language model loaded for labeling or annotation is released, and results containing tensor objects are converted into a JSON-compatible summary for display. Operations that produce a trajectory, except for the Consistency Check, which creates its own overlay, project that trajectory into two- and three-dimensional display space using the projection models saved with the dataset (Section 4.6): the final compact representation from each trajectory step is combined into a single matrix and transformed using these models. The resulting coordinates remain in the original projection space and are not normalized during this backend step. They are mapped to the shared display range only after reaching the frontend, using the same transformation applied to all other graph points. Finally, this result is then returned by the node and transmitted to the graph panel through ComfyUI's WebSocket connection. This allows the panel to update even though the computation is executed by a queued backend node rather than directly in the browser.

4.8. Operation families: implementation, math, and configurable parameters

Section 4.7 described the common dispatch and post-processing path. The following operation-specific descriptions explain the computation performed by each active operation, its implementation, the parameters exposed to the user, and the information retained for subsequent analysis. Fixed settings used in the experiments for the subsequent analysis are reported in Chapter 5.

Some operations additionally construct a trajectory. This ordered sequence of intermediate states makes a transformation or exploration path inspectable and, where possible, available for downstream image generation. Each intermediate state corresponds to one value in the user-specified alpha schedule and forms a single step of the trajectory. Besides the ordered steps, the trajectory records the identifiers of the source points and the corresponding full prompt texts. It therefore documents, for example, which two prompts served as the endpoints of an interpolation or which three prompts defined a Delta Walk, allowing

every generated step to be traced back to its inputs. For trajectories that can be rendered as images, the full prompt text associated with the source points in the respective operation are retrieved. These texts are re-encoded with the connected CLIP model to obtain real conditioning (cond) and, when available, a pooled CLIP output. Whenever an operation combines two or more real conditioning tensors, their sequence lengths are first aligned by zero-padding shorter tensors to the longest participating sequence. This preserves the existing token representations while providing compatible tensor shapes for subsequent arithmetic and image generation. The operation retains the resulting conditioning values in each computed trajectory step. Simultaneously, a compact embedding is derived from each step, using the representation rule described in Section 4.6, only for projection and visualization in the graph panel. For three of these trajectory-producing operations, LERP, SLERP, and Delta Walk, the user can additionally request a nearest-prompt lookup for every generated step. When the option is enabled, each step's final compact embedding is compared against every currently loaded point using cosine similarity, and the three closest matches are attached to that step and stored in the field "distance". The following sections describe the logic and functionality of each operation in detail. They explain how the respective computation is implemented, which design decisions shape its behaviour, and why these decisions were made.

Nearest Neighbor: Nearest Neighbor identifies prompts from the loaded subset of the dataset the user chooses, that are most similar to one or more selected prompts in compact embedding space. Its purpose is to reveal the local semantic neighbourhood of each selected point visually while excluding the selected points themselves.

The operation requires at least one selected point and a user-specified number of neighbours between 1 and 128, with a default value of 4. Only the points currently loaded into the panel serve as the candidate pool, rather than the complete saved dataset. This keeps the search aligned with the mentioned subset that the user is currently inspecting and with the points that can be highlighted and connected in the graph. It also avoids loading and comparing the complete dataset for each interactive query, which would reduce responsiveness for large datasets. For each selected query point and every loaded candidate, the operation flattens the compact tensor into one feature vector and L2-normalizes it to unit length. The resulting similarity is the dot product

$$\text{cosine}(q, x) = \hat{q} \cdot \hat{x},$$

of the query point and the candidate. This metric emphasizes vector direction rather than raw magnitude. That is appropriate for the compact representation because magnitude can be affected by prompt length and token statistics without directly representing semantic similarity. It also avoids relying on Euclidean distances, which, as mentioned before, become less meaningful in high-dimensional spaces [1].

Nearest Neighbor uses FAISS, introduced in Section 4.6, as its primary similarity search implementation. For candidate pools of up to 200.000 points, the operation builds an exact flat inner-product index and compares the query with every candidate. For larger pools, it instead uses an inverted-file index. This index first partitions the candidate vectors into coarse cells by training a quantizer. The quantizer uses the k-means procedure to repeatedly assign each candidate to a cell's centroid and recompute every centroid as the mean of its assigned candidates, thereby minimizing the average distance between each candidate and its assigned centroid [18]. The operation sets the number of cells to the nearest integer value of $\sqrt{n}$ for n candidates, following FAISS's recommended scaling. Although FAISS normally uses L2 distance for this coarse assignment, Nearest Neighbor passes inner-product similarity explicitly for both the quantizer and the subsequent search, since the resulting centroids are not unit-length like the input vectors, so L2- and inner-product-based assignment would not otherwise agree [18]. After the index has been built, the query is first compared with all coarse centroids. This step is computationally inexpensive because, as noted above, the index contains only approximately $\sqrt{n}$ centroids rather than n individual candidates. The operation then selects the closest 20 percent of the cells, with a minimum of one cell, and searches only the candidates stored in their inverted lists. As a result, the query is compared with a restricted subset of the candidate pool instead of with every candidate. This fraction is kept constant instead of fixing the searched cell count at one absolute number, so the number of cells actually searched still grows together with the total cell count as more candidates are loaded, matching FAISS's own view that the appropriate number of searched cells is inherently dataset-dependent [18]. If FAISS is unavailable or raises an error, the operation falls back to an exact brute-force cosine ranking implemented by the operation with PyTorch.

To compensate for selected points that could otherwise occupy top-ranked positions, the operation over-fetches by requesting the desired neighbor count plus the number of selected points, then discards the selected identifiers and keeps the requested number of remaining entries. Since no second search is issued, fewer entries can be returned when the loaded subset lacks enough unselected candidates. Each result entry identifies the candidate point, its display label, its cosine-similarity score, and the selected query point it belongs to. Results are not deduplicated across queries, so the same candidate can appear once per selected point for which it ranks among the nearest neighbours.

No trajectory is produced. The graph panel renders each result as a dashed connection from its selected query point to the returned neighbour.

LERP and SLERP: LERP and SLERP provide two alternative methods for constructing intermediate states between two selected prompts. LERP follows a direct linear path between the representations, whereas SLERP interpolates their directions along a spherical

path. Comparing the resulting trajectories makes it possible to examine whether the chosen interpolation geometry affects either the projected path or the images generated from the intermediate states.

The graph panel requires the user to select exactly two points. For each, the operation retrieves the complete stored prompt text, falling back to the truncated display label only when no complete text is available, and encodes both once with the connected CLIP model before constructing the trajectory. The resulting conditioning tensors and, where available, pooled CLIP outputs are then reused for every interpolation step.

Each value in the alpha schedule introduced above is converted to a floating-point number between 0 and 1. The operation terminates with an error if no valid values remain after this conversion.

Let A and B denote the aligned real conditioning tensors of the two selected prompts. LERP then calculates every intermediate state as the direct component-wise blend

$$LERP(\alpha) = (1 - \alpha)A + \alpha B.$$

Consequently, $\alpha = 0$ reproduces the start anchor and $\alpha = 1$ reproduces the end anchor, apart from any padding needed to establish the common shape. Intermediate values lie on the straight line between the two tensors.

SLERP uses the same padded inputs but weights them by the angle between their directions rather than blending them in a straight line. Both tensors are first flattened and normalized purely to compute this angle:

$$\theta = \arccos\left(\frac{A \cdot B}{\|A\|\|B\|}\right).$$

The intermediate state is then a direct weighted sum of the original, un-normalized anchor tensors, A and B, themselves:

$$SLERP(\alpha) = \frac{\sin((1 - \alpha)\theta)}{\sin(\theta)}A + \frac{\sin(\alpha\theta)}{\sin(\theta)}B.$$

This construction is also commonly used in community implementations of SLERP for interpolating diffusion-model prompts and seeds [37, 2]. One consequence is that whichever anchor has the larger magnitude also pulls the interpolated direction somewhat toward itself. If an anchor has a near-zero norm, or if the two normalized anchors are nearly collinear in either direction, the angle would be numerically unstable. SLERP then falls back to the corresponding linear interpolation for that step. When both anchors provide a pooled CLIP output, the operation applies LERP or SLERP to these pooled vectors in the same respective manner. If either pooled output is absent, no pooled output is attached to the generated step.

The resulting trajectory follows the common post-processing and image-generation handoff described at the beginning of this section, including the optional nearest-prompt lookup for each step whenever the user enables it. Finally, the graph panel projects the trajectory into the panel. Representative rendered examples are shown in Section 5.3.

Delta / Direction Walk: Unlike interpolation, Direction Walk does not generate intermediate representations between two selected points. Instead, it applies the semantic direction defined by two points, A and B, to an independent starting point, C, constructing new prompt representations by moving from C along the direction of B minus A. This makes it possible to examine whether a semantic transformation remains meaningful when transferred to a different semantic context.

Similar to LERP and SLERP, the operation retrieves the complete stored prompt text for each selected point, falling back to the truncated display label when unavailable, and encodes all three texts once with the connected CLIP model before the walk is computed.

The user additionally provides a schedule of alpha values within the interval from -2 to 4. Positive values apply the A-to-B direction with increasing strength, while negative values move from C in the opposite direction, applying the B-to-A transformation instead.

Following the shared sequence-alignment rule at the start of this section, let A, B, and C denote the three conditioning tensors, and let their unit-normalized forms be $\hat{A}$, $\hat{B}$, and $\hat{C}$. Therefore, the transferred direction is

$$d = \hat{B} - \hat{A}.$$

For each alpha value, Direction Walk adds a scaled version of this direction to the normalized start point, normalizes the resulting tensor again, and restores the original magnitude of C:

$$T(\alpha) = \|C\| \frac{\hat{C} + \alpha d}{\|\hat{C} + \alpha d\|}.$$

A small positive lower bound is used in each normalization to avoid division by zero. This normalized formulation is deliberate. A direct calculation of $C + \alpha(B - A)$ would allow differences in the raw magnitudes of A and B to influence the transferred change. Direction Walk instead derives and applies the relation on the unit sphere, so that the movement is governed by direction. Reapplying the original magnitude of C afterward preserves the scale expected from a normally encoded conditioning tensor by the downstream generation path and by the compact-embedding conversion.

At $\alpha = 0$, the result is C itself. Importantly, $\alpha = 1$ is not generally equal to B: it is a synthetic state obtained by transferring the full A-to-B change to C. Similar to LERP and SLERP,

pooled CLIP outputs are combined with the same normalized direction-transfer procedure when all three source prompts provide one, and omitted from the step otherwise.

The resulting trajectory is processed using the shared post-processing and image-generation handoff described at the beginning of this section. In the graph panel, the semantic relation from A to B is visualized as a coloured line, while the projected synthetic states form the walk trajectory. Representative rendered examples are shown in Section 5.4.

Semantic Axis Projection: Semantic Axis Projection was introduced to investigate whether the loaded compact embedding space contains coherent semantic dimensions that explain more than local proximity and cluster membership alone. It is an exploratory operation for making such potential visual or thematic dimensions inspectable. Any identified axis remains specific to the loaded dataset and the user's selected examples. Furthermore, this operation is one approach to approximating such semantic structure, rather than a universal semantic metric.

The following paragraphs describe how Semantic Axis Projection operationalizes this idea:

First, the user specifies the proposed primary contrast through two groups of loaded points. These group assignments are maintained separately from the ordinary point selection used by other operations, and each group must contain at least one valid point. In the graph panel, assigning points to one group automatically removes them from the other, keeping the two groups disjoint. The operation also accepts an optional request for LLM-generated axis labels and an alpha schedule for intermediate image generation steps. The schedule is restricted to values from 0 to 1 and defaults to [0, 0.25, 0.5, 0.75, 1.0] when it is absent or empty. If the two selected groups have identical compact-space centroids, no direction can be derived and the operation stops with an error.

Semantic Axis Projection determines the position of each loaded dataset point along the axis from group A to group B entirely within the compact embedding space. As established in Section 4.6, compact embeddings are already one-dimensional, so flattening here only ensures a consistent representation for tensor inputs. The operation first computes an arithmetic mean from the unnormalized vectors of group A and another from those of group B. However, these raw centroids are not used to define the analytical axis. Instead, they are retained solely to rank group members according to their representativeness when image-generation anchors are selected at a later stage.

This distinction is intentional. The analytical axis must represent all dataset points and both group centroids within the same direction-based geometry. If the axis direction were derived from raw centroids, differences in vector magnitude could influence its orientation for the same reason already discussed for Nearest Neighbor above, where such differences reflect properties like prompt length or token statistics rather than semantic content. To

avoid this inconsistency, the operation L2-normalizes the compact embedding of every loaded point, since these normalized vectors are also needed later to score the remaining dataset points along the axis. The centroids of the normalized vectors for groups A and B are then calculated as the arithmetic mean of the respective member vectors. Although the individual vectors have unit length, the resulting centroid is not normalized again, because in this case its magnitude still reflects the internal consistency of the group.

Let c_A and c_B denote these centroids for groups A and B, respectively. The primary axis is defined as the unit vector pointing from c_A to c_B:

$$a_1 = \frac{c_B - c_A}{\|c_B - c_A\|}.$$

The midpoint between the two centroids serves as the origin of the axis,

$$o = \frac{c_A + c_B}{2}.$$

while half of their Euclidean distance defines its scale:

$$s = \frac{\|c_B - c_A\|}{2}.$$

For each normalized loaded point x_i, Semantic Axis Projection calculates the score

$$\text{score}_i = \frac{(x_i - o) \cdot a_1}{s}.$$

Centering the projection at the midpoint and scaling it by half the distance between the centroids maps the group centroids to approximately -1 and +1. Consequently, negative scores indicate a position closer to group A along the semantic axis-1, whereas positive scores indicate a position closer to group B. Scores near zero correspond to points located close to the midpoint between the two groups. Each resulting entry contains the point identifier, its display label, and the corresponding projection score, sorted in descending order.

To interpret the two poles, the operation constructs two complementary evidence sets, since this point-wise ranking determines where each loaded point lies along the axis but individual extreme points may be atypical. The first consists of broad support bands, which preserve a sufficiently large population of extreme prompts for the subsequent contrastive axis-term analysis. For each side of axis-1, the target band size is 10 percent of the loaded points, clamped to a minimum of 24 and a maximum of 80 prompts. Consequently, a dataset with fewer points naturally provides only the available entries. The B-side band is taken from the highest scores and the A-side band from the lowest scores. Both are ordered from their respective most extreme pole inward. The second consists of a narrower coherent subset for each side, so that optional LLM labeling is based on mutually representative prompts rather

than on isolated extremes. It considers at most the 40 most extreme entries in the respective support band, forms the mean of their already normalized compact vectors, normalizes this prototype, and retains up to the 12 entries with the greatest cosine similarity to it.

As mentioned before, the operation uses the broad support bands to identify contrastive axis terms for both sides of the semantic axis. The three highest-scoring candidates on each side are used as concise textual labels for the corresponding pole, which will be explained in detail in the following paragraphs.

To prevent concepts shared by both poles from being interpreted as distinctive features, the operation also selects up to 20 prompt texts located near the midpoint of the score distribution as neutral examples. Candidate terms are then extracted from the prompts on both focus sides and from these neutral prompts. The extracted candidates include individual words, adjacent two-word phrases, and clauses consisting of one to three words separated by commas or enclosed in parentheses.

All three candidate types are converted to lowercase and filtered to remove English stop words, such as "the", "and", and "with", as well as common image-generation boilerplate, including terms such as "photo", "image", and "painting", though bigrams use a looser filter that only discards them when both words are stop words. Very short tokens are also discarded. Each prompt contributes at most one occurrence of a given term across all three types, so the resulting counts represent document frequencies rather than repeated occurrences within a single, potentially verbose prompt.

Single-word candidates must also occur in at least two prompts to be considered, whereas multi-word candidates are exempt from this frequency threshold and instead receive an additional phrase bonus. For each candidate term on the current focus side, the contrastive score is calculated as

`focus_count - 0.8 * other_count - 0.5 * neutral_count + phrase_bonus`,

where `phrase_bonus` is set to `0.2` for multi-word candidates and to 0 for single-word candidates. This scoring procedure rewards candidates that occur frequently on the focus side while penalizing candidates that also appear on the opposite side or in the neutral examples. Only candidates with a positive score are retained.

The remaining candidates are sorted first by their contrastive score, then by their frequency on the focus side, and finally alphabetically if score and frequency are tied. Semantic Axis Projection subsequently removes candidates that are fully contained within an earlier retained candidate, thereby reducing redundant labels. The three highest-ranked candidates for each side are returned as a string separated by `"|"` characters and displayed in the graph panel as a compact description of the two semantic poles.

For optional LLM-based labeling, the operation uses the coherent subsets rather than the broader support bands as its primary evidence. From each coherent subset, it samples up to five prompts at even intervals. It also derives up to three distinct axis terms from the coherent subset. These LLM-specific axis terms are used only during label generation and are not returned as separate result fields.

The number of example prompts is deliberately limited to 5 instead of including all 12 points in a coherent subset. This keeps the labeling prompt compact, which is particularly relevant because the procedure may issue up to two LLM calls per axis and therefore up to four calls when both axes are labeled. Keeping a limited number of prompts helps reduce overall compute cost. Since the 12 coherent points have already been selected for their proximity to one another, 5 evenly distributed examples should capture most of the subset's internal variation while reducing near-duplicate evidence.

Axis terms and example prompts serve different purposes within the labeling prompt. The axis terms summarize the features that distinguish one side of the axis from the other, whereas the complete prompts provide tone, context, and concrete visual or stylistic details that cannot be conveyed by isolated axis terms alone. Providing only the raw prompts would require the LLM to determine which of their many shared and differing characteristics define the semantic contrast. Because the axis terms computationally isolate this contrast in advance, the LLM can focus on expressing it as a concise and vivid axis label rather than rediscovering it.

Together with the neutral midpoint examples, this evidence is used to request three candidate labels in the form "X to Y". The labeling instruction asks for concrete visual or thematic contrasts, discourages generic scales, and explicitly excludes concepts that are common to both sides. If multiple candidates remain after validation, a second LLM call selects or lightly refines the candidate that most accurately reflects the axis terms while remaining vivid and clearly differentiated on both sides.

Before a candidate is accepted, formatting is removed and its structure is validated. Each side may contain no more than 4 words or 32 characters and must not include explanatory connective terms such as "because", "while", "with", or "featuring".

If the LLM call fails or produces no valid candidate, the operation falls back to a simpler, less descriptive label or returns none at all. In addition, the label assigned to the second axis is explicitly prevented from duplicating the accepted label of the first axis. The system-prompt templates for candidate generation and refinement are provided in Appendix 3.

After deriving the primary axis, Semantic Axis Projection searches for a second, automatically discovered direction in the variation that remains unexplained. It first centers all

normalized compact embeddings around their global mean $\bar{x}$. For a normalized point x_i, this mean-centered representation is

$$c_i = x_i - \bar{x}.$$

The operation then removes the component that lies along the primary axis:

$$r_i = c_i - (c_i \cdot a_1)a_1.$$

Singular-value decomposition (SVD) of the resulting residual matrix yields up to five candidate directions and their shares of residual variance [63]. A second axis is considered only when the leading candidate explains at least 5 percent of that variance. Since the residual matrix rows are already orthogonal to a_1 by construction, any remaining alignment between a candidate direction and a_1 reflects only floating-point rounding error accumulated during the decomposition, not a substantive second overlap. To correct this, each candidate is made orthogonal to a_1 again by removing the primary-axis component and normalizing:

$$a_2 = \frac{v - (v \cdot a_1)a_1}{\|v - (v \cdot a_1)a_1\|},$$

where v denotes the first candidate whose corrected norm exceeds a small numerical threshold (10^{-8}) rather than falling close to it, which would indicate that the candidate direction aligns almost exactly with the primary axis. In practice, v is almost always the candidate with the strongest residual variance itself, since the procedure only advances to the next-strongest candidate in the rare case that this correction removes nearly all of that candidate's length. Each point is then projected onto this second direction:

$$q_i = r_i \cdot a_2.$$

Unlike the primary-axis score, this value is normalized only for display by the largest absolute value of q across the loaded subset:

$$\text{score}_i = \frac{q_i}{\max_j |q_j|}.$$

The published values therefore range from -1 to +1, but do not express distance relative to two user-defined centroids.

The same residual and SVD procedure is not conceptually limited to a single secondary axis. The cap of five candidate directions is itself a chosen setting of the operation rather than a mathematical limit of the singular-value decomposition, which could in principle return as many components as the rank of the residual matrix permits. Because the operation already computes up to this many candidates, further axes could in principle be derived by repeating the same steps: removing the newly found axis's component from the residual

as well, then re-applying the variance threshold and orthogonalization to the remaining candidates. The current implementation stops after one secondary axis, but the underlying method generalizes to as many additional axes as desired.

Returning to the single secondary axis Semantic Axis Projection currently computes, it has no user-selected groups of its own. Its positive and negative poles are determined by the highest and lowest residual-axis scores. Semantic Axis Projection then applies the same support-band and coherent-subset procedure described for axis-1 to these residual extremes, and the resulting bands additionally form the candidate pools from which the axis-2 image-generation anchors are selected, rather than the original A and B groups used for axis-1.

This is the later stage foreshadowed above, keeping the generated trajectory tied to the semantic contrast the user selected rather than to the automatically identified extremes used for axis-term evidence.

For the first axis, only points from the original groups A and B that contain usable prompt text are considered. Within each group, the points are ranked by their cosine similarity to the group's raw compact centroid. The operation retains up to eight of the most representative points from each side and then reduces both anchor sets to the size of the smaller set, ensuring that the two sides contribute an equal number of anchors.

The anchors for the second axis are selected using the same cosine-similarity criterion. However, because this axis is not based on user-defined groups, its raw centroids are calculated from the total points in the coherent residual subsets described above.

For each axis with valid anchors on both sides, the anchor prompts are re-encoded to obtain their conditioning tensors, which are aligned using the shared sequence-alignment rule and averaged into a single real-conditioning centroid for each side. Pooled outputs are also averaged this way, but only if every anchor on the corresponding side provides one. Then the operation linearly interpolates between them. Figure 4.3 shows an example run of this image-generation trajectory for both axes.

Finally, interpolation steps from the first and second axes are assigned different source-text pair identifiers. This allows the graph panel to display the resulting trajectories as two separate paths rather than combining them into a single line.

Consistency Check: Consistency Check examines whether small textual changes preserve the position and cluster of a prompt in the compact embedding space. It starts from one selected dataset point and generates several variants of its prompt. The operation then measures how far these variants move away from the original prompt relative to the local scale of the saved dataset. It is therefore an operational test of semantic stability: variants

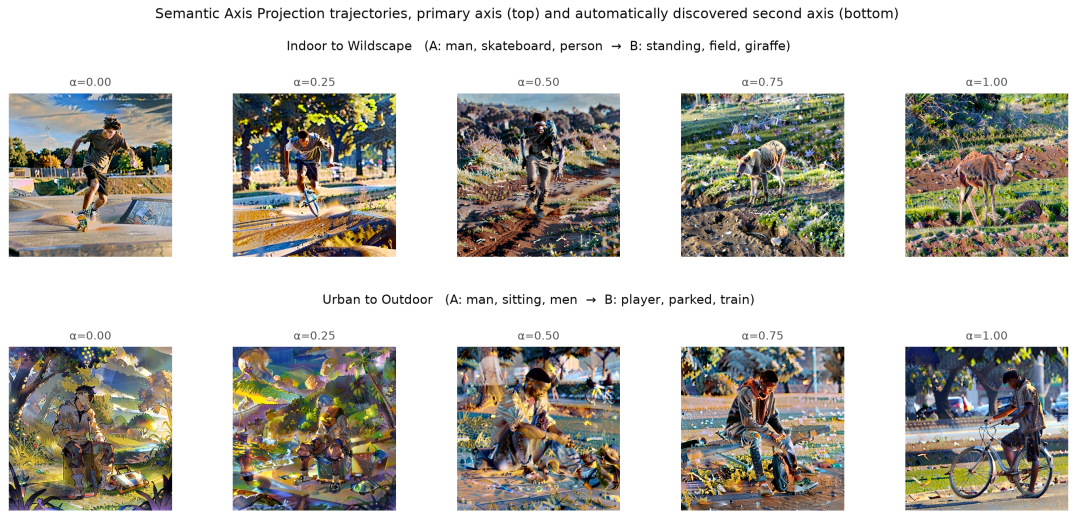


Figure 4.3.: Images generated along the primary axis and the automatically discovered second axis with the LLM-generated axis labels and the three contrastive axis terms

that are intended to remain close in meaning should also remain close to the base prompt in the compact representation.

The operation requires exactly one selected point and an active connection to a CLIP model. It uses the complete original prompt associated with the selected point as the base prompt. The user can configure both the number of generated variants and the variation strength. The number of variants can range from 3 to 20, with a default value of 5, while the variation strength is restricted to the interval from 0.05 to 1.0 and defaults to 0.3. Reproducibility is ensured by the random seed configured in the NELS Dataset Loader node and is passed down the pipeline into the operation. This seed controls the stochastic components of the operation, including the words selected by the word-dropout procedure and the points sampled from the complete saved dataset as background references for the consistency metric.

For strengths from 0.05 to below 0.35, the operation generates prompt variants using the mentioned word dropout method. If the base prompt contains at least two words, the number of words to remove is calculated as follows:

```
max(1, min(number_of_words - 1, round(min(strength, 0.2) * number_of_-
words)))
```

The strength-dependent term is capped at 0.2, meaning that the procedure generally removes no more than 20 percent of the words. The value is further limited to keep at least one word in the variant. Finally, the outer max operation guarantees that at least one word is removed, even if rounding would otherwise produce zero. For very short prompts, this

minimum removal of one word may therefore exceed the nominal 20 percent limit. Each accepted candidate must differ from the base prompt, retain at least one word, and be distinct from all previously accepted candidates. To generate the requested number of variants, the operation evaluates up to 8 times as many candidates as requested. Candidates are added in the order in which they are generated, and the procedure terminates once the requested number of distinct valid variants has been obtained. For example, when 5 variants are requested, Consistency Check retains the first 5 candidates that satisfy these conditions. If the procedure cannot generate enough distinct candidates, the remaining result slots are filled with unchanged copies of the base prompt. Word dropout also serves as a fallback if the LLM-based generation path fails. In this fallback mode, the retained words are additionally shuffled when the variation strength exceeds 0.55.

For strengths of 0.35 or higher, the operation instead requests one LLM paraphrase per desired variant. The instruction changes with the selected strength: values below 0.4 request minor rewording and synonyms only, values from 0.4 to below 0.7 request moderate rephrasing with varied structure, and values of 0.7 or above request substantial changes in phrasing. Each call requests the paraphrased prompt as a single plain-text line and is informed of paraphrases that are already accepted, so that repetitions can be avoided. To keep the variants comparable to the complete base prompt, a paraphrase may contain at most five words more than the base prompt. The response budget of the LLM is derived from this limit, combined with an allowance of 3 tokens per permitted word and a minimum of 24 tokens. Word-dropout variants already satisfy the same upper bound because they can only remove or reorder words. If the LLM produces fewer acceptable distinct paraphrases than requested, Consistency Check also fills the remaining result slots with unchanged copies of the base prompt. An example run and its resulting outputs are shown in Table 4.1. The system-prompt template used for LLM paraphrasing, including the strength-dependent style clause, the maximum-word-count rule, and the list of previously accepted paraphrases, is provided in Appendix 3.

The base prompt and all variants are encoded together in one CLIP pass. For every text, this produces real conditioning, a compact embedding, and, when supplied by the encoder, a pooled output. The operation reuses these values for every later stage: the consistency metric, cluster assignment, graph overlay and trajectory construction.

To turn this idea into a comparable result, Consistency Check measures each variant's compact-embedding drift relative to a reference sample drawn from the whole saved dataset. The consistency metric is computed only from the compact embeddings. As in the nearest-neighbor operation, cosine similarity is used. For every variant, its drift from the base prompt is

$$\text{intra}_i = 1 - \text{cosine}(\text{base}, \text{variant}_i).$$

Table 4.1.: Consistency Check variants for the shared base prompt "a photo of great white shark, white shark, man-eater, man-eating shark, Carcharodon carcharias" (`imagenet1k_labels_classic`), three returned variants per representative strength setting

Variation strength	**Generation strategy**	**Variants generated from the shared base prompt**
0.20	Word dropout	"of great white shark, white shark, man-eater, man-eating Carcharodon carcharias" "a photo of shark, white shark, man-eater, man-eating shark, carcharias" "a white shark, white shark, man-eater, man-eating shark, Carcharodon carcharias"
0.35	LLM: minor rewording and synonyms	"image of great white shark, white shark, man-eater, Carcharodon carcharias" "picture of great white shark, white shark, predator, Carcharodon carcharias" "snapshot of great white shark, white shark, predator, Carcharodon carcharias"
0.50	LLM: moderate rephrasing with varied structure	"great white shark image, man-eater, Carcharodon carcharias" "great white shark image, white shark, man-eater predator" "a photo of great white shark, white shark, man-eater, man-eating shark, Carcharodon carcharias" (unchanged base prompt, fallback)
0.80	LLM: substantial changes in phrasing	"mighty white shark, fierce predator, man's fiercest foe" "mighty white shark, apex predator, human hunter" "vision of mighty white shark, apex predator, human meal, deadly Carcharodon carcharias"

Averaging these values yields `mean intra`, the mean within-set drift. To place this value in context, the operation loads the complete saved compact-embedding matrix for the dataset directly from disk, and samples up to 100 embeddings from it using the dataset loader's random seed. Following this, the same distance metric reference from the base prompt to these samples is computed. Their average, `mean inter`, is the background reference scale. The final score is

$$\text{consistency score} = \frac{\text{mean intra}}{\max(\text{mean inter}, \epsilon)},$$

where ϵ prevents division by zero. A lower score indicates that the generated variants stayed close to the base prompt relative to this background scale, and a higher score indicates greater relative drift in the model's learned representation. In conclusion, the score is a relative stability measure rather than a universal semantic distance metric, independent of which subset happens to be loaded into the graph panel at the time.

To complement the consistency score with a structural view of the embedding space, Consistency Check assigns each generated variant to a cluster through the dataset's saved runtime cluster-prediction mechanism described in Section 4.6. If the variant is classified as noise,

no cluster identifier is reported. This shows whether variants that remain close to the base prompt according to CLIP-space similarity also retain its local cluster context.

Taken together, the result reports for every variant its generated text, its cosine similarity to the base prompt, and the predicted cluster identifier. The graph overlay projects the compact embeddings of the variants through the saved display models. The separate returned trajectory includes the base prompt as its first state, followed by every generated variant. Accordingly, the downstream image-generation node can render the base prompt and all retained variants directly from their attached conditioning, following the common trajectory handoff described at the beginning of this section. A representative rendered example is shown in Section 5.6.

Latent Probe: Latent Probe reverses the usual direction of the graph exploration. The other operations begin with one or more existing prompts and examine their position, neighborhood, or trajectory in the graph. Latent Probe begins with a position in the graph and asks which prompt representation could plausibly correspond to that location. Its purpose is to investigate whether local geometry in the projected embedding space carries usable semantic information about the underlying prompt representations.

Existing work investigates semantic directions in text embeddings and local geometric structure in diffusion representations, but does not establish a unique or exact mapping from a low-dimensional projection coordinate back to one prompt [63, 47, 39]. NELS therefore treats Latent Probe as an exploratory prototype that serves as an approximation for geometric structure and its semantic encoding. It does not claim to recover an original prompt from a graph coordinate exactly.

The operation has two independent phases: first estimating a compact target vector for the clicked position and, optionally, second reconstructing a prompt near that target. The user chooses whether to place the probe in the 2D or 3D graph view and whether to enable text reconstruction. When reconstruction is enabled, the user can configure the number of seed, noise, and negative examples, as well as the maximum search time and number of refinement iterations that guide the operation.

The frontend first converts the clicked position from the [-1, 1] display space back into raw UMAP coordinates using the inverse of the mapping in Section 4.7. The backend therefore always receives and processes raw 2D or 3D UMAP projection coordinates. It then loads the complete saved reference-coordinate array for the selected UMAP view (`umap_2d.npy` or `umap_3d.npy`), rather than only the currently loaded subset, together with the memory-mapped compact-embedding matrix (`compact_embeddings.npy`). From that data it computes the Euclidean distance from the clicked position to every saved reference coordinate. This is appropriate at this stage because the query and reference points both

lie in the same low-dimensional UMAP coordinate system. Thus, the 10 nearest reference points are retained and their compact embeddings are combined into one estimate with inverse-distance weights

$$w_i = \frac{1}{\text{dist}_i + \epsilon},$$

normalized so that $\sum_i w_i = 1$. The estimated target vector is then

$$q = \sum_i w_i \, \text{compact_embedding}_i.$$

Here, ϵ is a small positive constant that prevents division by zero when the clicked position coincides exactly with a saved reference coordinate. Selecting 10 neighbours provides sufficient local coverage for a stable weighted estimate while limiting the influence of points from more distant regions of the projection. Using only one neighbor would make the estimate more sensitive to local projection artifacts, whereas using a substantially larger neighborhood could average away the local structure that the probe is intended to examine. As a result, q is a locally weighted estimate in the compact embedding space. It serves as the target for nearest-prompt retrieval, approximate cluster assignment, and optional text reconstruction.

The estimated vector `q` is passed to the dataset's cluster-prediction mechanism described in Section 4.6. The resulting cluster id, noise flag, and membership strength are approximate runtime assignments. Latent Probe also compares `q` with the compact tensors of the currently loaded points using cosine similarity and returns up to the requested number of nearest existing prompts, defaulting to 15 when no request is defined. Each hit contains an id, a similarity score, the full prompt text, and a display label capped at 80 characters. The graph panel displays up to the five highest-scoring prompts from this result set. If no points are loaded, or none carries a usable tensor, the same lookup falls back to the complete saved dataset and reads only the necessary rows from `samples.jsonl`. Such fallback hits may not correspond to points currently rendered on the canvas and are therefore not necessarily clickable in the graph panel.

Text reconstruction runs only when it is enabled and the probe lookup produced at least one usable hit. Its objective is to find a new prompt whose embedding lies close to q. First, the reconstruction context is assembled from the currently loaded subset. The retrieved nearest prompts described above serve as positive seed examples for the LLM-based search. The least similar loaded points are used as negative examples that define concepts to avoid. Additional points are sampled from the middle of the similarity ranking as noise examples. Building on this, the initial LLM call seeds and noise texts are combined as positive examples near the target, while negative texts specify, as mentioned above, themes to avoid. The result also returns the identifiers and truncated labels of all three categories as an inspectable record of the context supplied to the first LLM call. If the LLM produces no valid initial

candidate, the lookup and cluster result remain available, but no reconstruction trajectory is created.

With the explained reconstruction context above, reconstruction begins with an LLM prompt that requests up to 16 new, self-contained image descriptions of 5 to 20 words. The generated descriptions must not copy phrases from the reference prompts. Each candidate is re-encoded with CLIP, converted into a compact embedding, and scored by cosine similarity to q. The eight highest-scoring candidates form the initial beam. In each subsequent round, the LLM receives this ranked beam, including the current similarity scores, together with the negative examples. It proposes up to 16 refinements, which get combined with the previous beam while removing duplicate prompt texts. Finally eight best candidates are retained. During each completed search step, the graph panel receives a WebSocket progress event containing the current best prompt and its similarity score. Finally, the search stops when it reaches the configured time or iteration limit, when the LLM produces no new candidate, or when the best score fails to improve by more than 0.0007 for four consecutive rounds. To summarize, the reconstruction is trying to match the estimated target vector q in the compact embedding space, to be able to approximate a prompt for the clicked position in the UMAP projection.

The two LLM instruction templates used for the initial candidate generation and the later refinement rounds are provided in Appendix 3.

The resulting trajectory is a compact record of the search progress. Once valid initial candidates exist, Latent Probe always records the first beam state. A later state is recorded only when its best score exceeds that of the preceding snapshot by more than 0.0001. The final state is retained in every case. If its best prompt matches the last recorded prompt, the operation updates that snapshot instead of adding a duplicate. Otherwise, it appends the final state as a new snapshot. Each recorded state contains its final compact tensor and real CLIP conditioning. If the connected encoder provides a pooled output, this value is stored as well. No additional conditioning step is required before image generation. Its associated texts are the prompts generated during the search, and each trajectory step refers to one prompt by its position in that ordered list. Accordingly, the downstream image-generation node can render the recorded states directly from their attached conditioning, following the common trajectory handoff. An example search and its retained snapshots are shown in Figure 4.4.

4.9. Design tradeoffs and validity limits

Several limitations arise directly from the design decisions presented in this chapter rather than from implementation errors. The system uses compact embeddings for analysis and

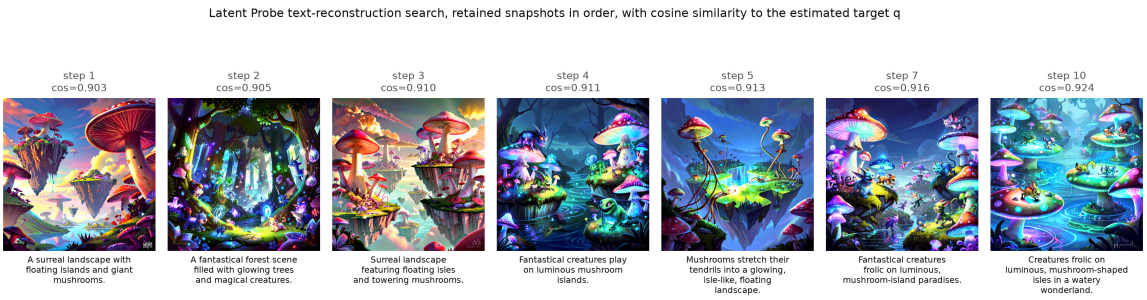


Figure 4.4.: Latent Probe text-reconstruction search, retained snapshots in order, with cosine similarity to the estimated target q and the reconstructed prompt text below each image

navigation, while image generation relies on the original conditioning tensors. This separation keeps the panel responsive and preserves compatibility with the KSampler (ComfyUI's diffusion-sampling node), but it also means that proximity in the graph does not necessarily correspond to proximity in the conditioning space used for generation. Consequently, a point that appears close to its neighbors on the canvas may still differ substantially from them in the full conditioning representation. Moreover, compact embeddings are inherently lossy. Regardless of the pooling method used to create them (Section 4.6), token-level information is reduced to a single fixed-size vector, which may conceal subtle semantic differences that remain present in the complete sequence tensor.

UMAP, HDBSCAN, and the inverse estimation applied by Latent Probe are approximate methods rather than exact geometric procedures. Their projected distances, cluster boundaries, and estimated target embeddings should therefore be interpreted as practical approximations rather than exact results.

Dataset construction introduces additional scalability limitations. Prompt encoding is performed sequentially, requiring one forward pass per prompt because pooled outputs are obtained individually. Furthermore, fitting UMAP and HDBSCAN is CPU-bound. As described in Section 4.6, adding new prompts requires the projection and clustering models to be refitted on the complete dataset rather than updated incrementally. Consequently, extending an existing dataset remains computationally expensive.

The `classic` strategy was the preferred approach for the large-scale operation datasets, since it reuses the canonical fit directly rather than introducing a separate runtime approximation, and later proved to be the most accurate of the three strategies at reproducing the canonical fit (Section 5.2). It was nevertheless not used for the four large-scale operation datasets described in Section 4.5. An exploratory `classic` build at the target size of 550.000 prompts showed that each of the three serialized runtime UMAP models (cluster space, 2D, and 3D) reached approximately 50 GB, since the underlying UMAP implementation stores a copy of the training data together with its nearest-neighbor search index as attributes of

the fitted model object, all of which are serialized into the same file when the model is persisted. Loading and holding three such models simultaneously is not feasible on typical consumer hardware, which motivated the use of the `two_pass` strategy's smaller, subset-trained runtime models for the large-scale datasets instead.

The `parametric` projection strategy introduces further dependencies and training overhead through Parametric UMAP and TensorFlow. Its current implementation is not yet fully robust for very large embedding matrices, where fitting may fail because of resource constraints. For this reason, the four large-scale operation datasets described in Section 4.5 use the `two_pass` strategy, while `parametric` is applied only to the smaller ImageNet-1k comparison dataset. Two concrete instances of this robustness limitation were investigated in more detail below, namely a reproducibility failure at the ImageNet-1k scale, and a hard scalability failure at a substantially larger corpus size.

This reproducibility limitation was examined by rebuilding the `parametric` ImageNet-1k comparison dataset using the same input data, parameters, random seed, and library versions. Despite these identical conditions, the resulting clustering differed substantially. The divergence was traced to non-deterministic floating-point behavior during TensorFlow CPU training with oneDNN acceleration, where small numerical differences accumulated over training and produced different network parameters. After disabling oneDNN through the `TF_ENABLE_ONEDNN_OPTS` setting, repeated runs produced bit-identical outputs.

A more severe scalability failure occurred when constructing a `parametric` dataset from approximately 61.700 prompts. Training terminated before producing a projection because the serialized TensorFlow computation graph exceeded protobuf's fixed 2 GB message-size limit. This restriction originates within the TensorFlow, Keras, and `umap-learn` stack rather than in the NELS implementation itself. Since the failure occurred at approximately one ninth of the size of the large operation datasets used in this thesis, it further supports the use of the `two_pass` strategy instead of `parametric` for large-scale datasets.

These design trade-offs and validity limitations establish the scope within which the implementation can be evaluated. Chapter 5 presents this empirical evaluation and examines how the operations introduced in this chapter behave in practice.

Chapter 5.

Experimental Results and Analysis

This chapter presents the empirical evaluation of the exploration environment described in the previous chapter. It first establishes the shared experimental setup and rationale underlying all experiments, then validates the accuracy of the underlying projection and clustering methods on previously unseen data. The remaining sections evaluate each of the plugin's navigation operations individually. The datasets, analysis scripts, and results referenced throughout this chapter are publicly available in the project repository [16], under the `datasets` and `analysis` directories respectively.

5.1. Evaluation parameters

Following the implementation and its limitations described in the previous chapter, this evaluation focuses on whether the generated trajectories exhibit interpretable, continuous, and reproducible changes under controlled experimental conditions. To make comparisons between operations and parameter settings meaningful, all experiments use a standardized image-generation configuration.

Every image-generation experiment reported in this chapter uses the same sampling parameters. This ensures that observed differences between experimental conditions are primarily due to the navigation operations rather than to changes in the generation setup. All images are generated at a resolution of 512x512 pixels using the Euler sampler, the `simple` scheduler, a classifier-free guidance scale of 8.0, and full denoising with a denoising strength of 1.0. The model and text encoder correspond to the configuration introduced in Section 4.5.

The sampling seed is not configured independently. Instead, it is derived from the same fixed seed used to select the loaded dataset subset, `42`, and remains constant across all experiments in this chapter. This controls the stochastic component of the generation process and prevents newly sampled noise from introducing an additional source of variation when images and consecutive trajectory steps are compared.

The number of denoising steps was selected empirically during preliminary testing, comparing image quality and how well the generated images represented the prompt's semantics

at different steps. Based on these observations, 20 steps were adopted as the fixed setting for all image-generation experiments reported in this chapter, keeping the generation time per image at a minimum while still producing reliable results.

5.2. Projection and cluster structure

Section 4.6 introduced the three projection strategies `classic`, `two_pass`, and `parametric` and compared their canonical fits using ImageNet-1k datasets constructed from the same corpus of 1.000 class-label prompts. However, that comparison does not show how accurately each strategy handles previously unseen points at runtime. The present section therefore evaluates three aspects of runtime performance: the accuracy with which new points are placed in the existing projection, the accuracy of their assigned cluster labels, and whether the same local neighborhood is retained after projecting a new point, regardless of whether its projected coordinates are accurate. For each strategy, a subset of prompts is withheld during dataset construction and subsequently processed using the corresponding runtime projection and cluster-assignment procedures. The resulting coordinates and cluster assignments are then compared with those obtained from a complete canonical fit that includes the held-out prompts to reveal differences.

The 1.000 ImageNet-1k prompts are divided deterministically into 800 training prompts and 200 held-out prompts using a fixed random seed. Because the split is generated from newly reconstructed prompt text rather than from the row order of the existing datasets, the identifiers of the held-out samples are recovered by matching their exact text against the samples stored in the existing `imagenet1k_labels_classic` dataset. A separate 800-prompt training dataset is then created for each projection strategy. All datasets use the same CLIP encoder, canonical UMAP parameters, and clustering configuration as the corresponding full 1.000-point comparison datasets. The number of training samples is therefore the only systematic difference between each reduced training dataset and its full reference dataset.

For the `two_pass` strategy, the runtime projection models are trained on a deterministic subset of the canonical dataset. To preserve the same 50 percent subset proportion used in the existing comparison dataset, 400 of the 800 training points are selected for the primary evaluation. For each strategy, the corresponding existing 1.000-point dataset serves as the reference, since its canonical fit already contains the 200 held-out prompts.

For each strategy, the 200 held-out prompts' already-encoded compact embeddings are reused directly, since they are byte-identical across all three comparison datasets, and are pushed through that strategy's runtime projection and cluster-assignment functions. For `classic`, this means transforming through the saved canonical UMAP models and predicting cluster assignment via the cluster-space UMAP together with HDBSCAN's approximate-

prediction mechanism. `parametric` uses the same two steps through its trained neural encoder, since it maintains no separate runtime model. `two_pass` instead projects positions through its smaller runtime UMAP models, while cluster assignment uses the FAISS-based cosine-similarity vote described in Section 4.6.

Because the runtime and reference projections are produced by independently fitted UMAP models, their coordinates cannot be compared directly. Even when both embeddings preserve a similar structure, their coordinate systems may differ by translation, rotation, reflection, and uniform scaling. Coordinate accuracy is therefore evaluated after a scaled Procrustes alignment [19], a standard procedure for aligning two corresponding point sets. The 800 training prompts serve as landmarks because their coordinates are available in both the runtime projection and the full reference projection. The orthogonal transformation is estimated using `scipy.linalg.orthogonal_procrustes`, while centering and uniform scaling are applied explicitly. For each held-out point, the coordinate error is then defined as the Euclidean distance between its aligned runtime position p_{aligned} and its reference position g:

$$d = \|p_{\text{aligned}} - g\|.$$

The errors of the 200 held-out prompts are summarized by their mean, median, and standard deviation, separately for the two- and three-dimensional projections.

Cluster-assignment accuracy is evaluated using the Adjusted Rand Index (ARI) [32], comparing each held-out point's cluster label predicted by the strategy's runtime procedure against its ground-truth cluster label from the corresponding full 1.000-point dataset. The ARI compares two clusterings by considering all pairs of samples and determines whether each pair is assigned to the same cluster or to different clusters in both labelings. It then subtracts the amount of agreement expected by chance from this score. This makes the measure independent of the numerical identifiers assigned to individual clusters, which is essential because HDBSCAN and the `two_pass` voting procedure may represent the same underlying grouping with different cluster numbers. An ARI of 1 indicates identical cluster assignments, a value near 0 indicates agreement comparable to random labeling, and negative values indicate agreement below chance. Noise points are treated as members of a separate label category. The score is computed with `sklearn.metrics.adjusted_rand_score`. In addition, noise agreement is reported as the proportion of held-out points for which the predicted noise status matches the reference noise status.

The third aspect, neighbor recall, is evaluated next. For each held-out prompt, a reference neighborhood N_i^{raw} is first defined directly in the compact embedding space. It consists of the $k = 32$ most similar samples among the 800 training prompts, measured by cosine similarity. Because this step is performed before dimensionality reduction, the resulting neighborhood is independent of the projection strategy.

Two projected neighborhoods are then compared with the same compact-space reference. The first, labeled `subset` represents the strategy's runtime behavior. The held-out point is projected using the runtime model fitted on the corresponding 800-point training dataset. Where the runtime model defines an independent coordinate system, as in the `two_pass` strategy, its output is first transformed into the coordinate frame of that same 800-point dataset's own stored training-point positions, using the Procrustes alignment described above. The 32 nearest training samples are then identified in the aligned two- or three-dimensional space.

The second variant, labeled `canonical` in the underlying results, referred to here as the **ground-truth** variant, is derived from the corresponding full 1.000-point reference dataset. Since both the held-out prompts and the 800 training prompts were included in its full fit, their stored two- and three-dimensional coordinates can be compared directly. No runtime projection or alignment is required. The resulting neighborhood therefore reflects the structure preserved by the complete UMAP fit itself rather than by an additional runtime approximation.

Neighbor recall at k is defined as

$$\text{recall@}k = \frac{1}{n}\sum_{i=1}^{n}\frac{|N_i^{\text{raw}} \cap N_i^{\text{proj}}|}{k},$$

where $n = 200$ is the number of held-out prompts and N_i^{proj} is the neighborhood obtained from either the runtime or the ground-truth variant. The metric represents the average proportion of compact-space neighbors that remain among the k nearest neighbors after projection. The ground-truth score, derived entirely from the complete 1.000-point canonical fit, measures the neighborhood information lost through dimensionality reduction alone. The difference between the runtime score, computed from the reduced 800-point fit, and the ground-truth score then estimates the additional loss introduced specifically by the strategy's runtime approximation of fitting on fewer points and projecting the held-out point out-of-sample.

Table 5.1.: Held-out accuracy per strategy. Neighbor recall is reported as runtime-model / ground-truth-model

Strategy	Coord. err. 2D (mean/med./SD)	Coord. err. 3D (mean/med./SD)	Cluster ARI	Noise agr.	Recall@32 2D	Recall@32 3D
`classic`	1.228 / 0.798 / 1.056	1.120 / 0.998 / 0.569	0.679	0.985	0.390 / 0.390	0.449 / 0.434
`two_pass`	36.161 / 38.527 / 5.364	50.751 / 51.785 / 4.710	0.679	0.985	0.219 / 0.390	0.287 / 0.434
`parametric`	1.383 / 1.067 / 1.240	1.014 / 0.937 / 0.532	0.222	0.780	0.288 / 0.386	0.370 / 0.421

Because `classic` and `two_pass` share the same reference fit, their ground-truth cluster labels and noise assignments are identical. Their runtime procedures differ, however. As described above, `classic` reuses the original fitted projection and clustering models, whereas `two_-`

`pass` relies on a separately trained runtime projection and a FAISS-based voting procedure for cluster assignment.

Even after Procrustes alignment and scale correction, the coordinate error of `two_pass` remains approximately 30 times larger than that of `classic`, visible in Figure 5.1. Its neighbor recall is also close to half the value obtained from the corresponding ground-truth fit. These findings indicate that the separately fitted runtime UMAP model provides a substantially weaker approximation of the ground-truth projection than the direct model reuse employed by `classic`. By contrast, the FAISS-based cluster-assignment procedure used by `two_pass` reproduces the ground-truth cluster labels with approximately the same accuracy as HDBSCAN's approximate prediction in `classic`. The cluster ARI obtained by `parametric` is considerably lower than that of the other two strategies. This result is consistent with its ground-truth fit producing a substantially different cluster structure from the shared reference fit used by `classic` and `two_pass`.

The comparison is based on a relatively small corpus consisting of 800 training prompts and 200 held-out prompts. Runtime projections based on UMAP may benefit from larger and more representative training sets, meaning that the relative disadvantage observed for `two_pass` cannot necessarily be transferred directly to the substantially larger datasets. A direct evaluation at that scale was not feasible with the available hardware, making this small-corpus experiment the only direct assessment of runtime projection accuracy conducted in this work.

Within this held-out evaluation, `classic` most closely reproduces the ground-truth reference fit. The ground-truth geometry of `parametric` differs most strongly from those of the other two strategies, while `two_pass` provides the least accurate runtime coordinate approximation. Its cluster-assignment procedure nevertheless performs at a level comparable to that of `classic`.

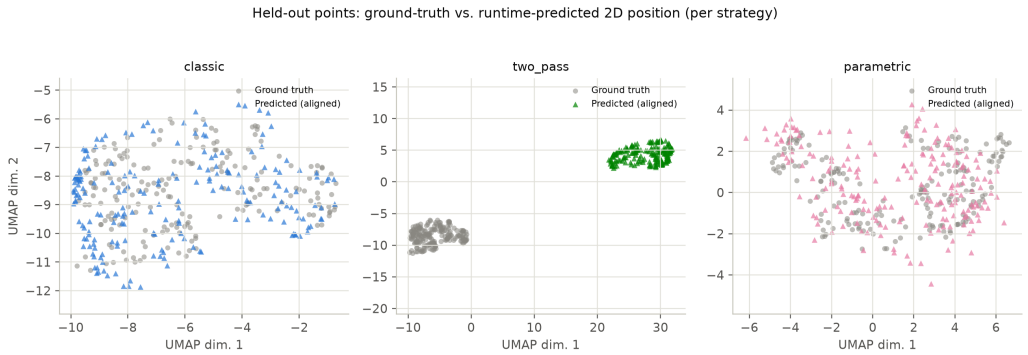


Figure 5.1.: Held-out points' ground-truth position (gray circles) against each strategy's Procrustes-aligned runtime-predicted position (colored triangles)

Figure 5.1 shows this gap directly: `classic`'s and `parametric`'s predicted points fall inside the same region as their ground truth, while `two_pass`'s predicted cluster sits entirely apart

from where the held-out points actually belong, consistent with its far larger coordinate error in Table 5.1.

5.3. Interpolation behavior: LERP vs SLERP

Section 4.8 introduced LERP and SLERP as alternative methods for interpolating between the same pair of source prompts, while Section 2.6 noted that geometric smoothness does not necessarily translate into perceptually smooth image transitions. The present section therefore examines whether the selected interpolation method and its resulting trajectory provide meaningful information about the quality and semantic continuity of the generated images, or whether these properties are influenced more strongly by other factors. LERP and SLERP are compared under identical experimental conditions across prompt pairs representing different levels of source-prompt similarity. Their intermediate states and trajectories are evaluated quantitatively using LPIPS and embedding-based similarity measures, following the evaluation framework established in Section 3.3.

Six prompt pairs were selected from the `gustavosta_sd_prompts_v2_two_pass` dataset, which contains 61.746 prompts and is one of the four large-scale operation datasets generated for this thesis. Pair selection combined a deterministic, algorithmic candidate-generation step with a subsequent manual curation step. The candidate pool was first restricted to 3.647 non-noise prompts of 40 to 140 characters in length. Eight anchor prompts were then drawn from this pool using a fixed random seed, and for each anchor, cosine similarity between compact embeddings was used to determine three tiered candidate partners: the highest-, median-, and lowest-similarity match, yielding 24 candidate pairs in total. From these 24 candidates, two of the eight anchors were dropped manually as less illustrative examples, leaving six anchors. Each of the six remaining anchors then contributed exactly one of its three tiered candidates to the final selection, yielding six pairs, two for each similarity tier: high similarity (0.98-0.99), medium similarity (0.85-0.92), and low similarity (0.72-0.73). The low tier represents the lowest similarity range that could be identified reliably under this procedure. For all eight original anchors, the lowest-similarity partner resolved to the same single prompt, suggesting that this prompt occupies an unusually isolated position in the compact embedding space. The final pair selection is fixed and documented in `pair_manifest.json`.

For controlled execution, a small companion dataset was created containing only the 11 distinct prompts involved in the comparison. One endpoint occurs in both low-similarity pairs, as mentioned before, because it was the only suitable low-similarity partner that could be identified reliably for both anchor prompts. The resulting dataset therefore contains 11 rather than 12 unique prompts. Using a separate companion dataset does not affect the interpolation results, because both interpolation operations re-encode the original

prompt texts to obtain the full conditioning tensors required for generation, as descibed in Section 4.8.

For each pair, LERP and SLERP were executed using the same five interpolation values, (0, 0.25, 0.5, 0.75, 1). All runs used the generation configuration defined in Section 5.1. The experiment comprises three similarity tiers, two prompt pairs per tier and two interpolation methods, resulting in 12 trajectories in total. Each tier contains four trajectories: two produced with LERP and two with SLERP. The generated images and the compact embedding associated with each interpolation step were stored using an implemented save node.

Perceptual change is measured with LPIPS, using the `lpips` package, between each pair of consecutive generated images. Higher LPIPS values indicate stronger visual changes, whereas lower values indicate greater perceptual similarity. In parallel, cosine similarity is calculated between the compact embeddings of consecutive interpolation steps. Higher cosine similarity indicates that neighboring representations remain more similar, while lower values indicate stronger divergence.

For each trajectory, both measures are summarized across its four consecutive transitions. The reported statistics include the mean LPIPS and mean cosine similarity, as well as the most abrupt transition. This transition is identified by the maximum LPIPS value and the minimum cosine similarity. Tier-level results are obtained by averaging the trajectory-level values across the two prompt pairs evaluated with the same interpolation method.

Table 5.2.: Mean, worst-step LPIPS, cosine similarity per similarity tier and operation, averaged over the two pairs per tier

Tier	Operation	LPIPS mean	LPIPS worst-step mean	Cosine-sim. mean	Cosine-sim. worst-step mean
high	lerp	0.571	0.676	0.9991	0.9991
high	slerp	0.569	0.676	0.9991	0.9991
medium	lerp	0.607	0.676	0.9926	0.9919
medium	slerp	0.609	0.678	0.9926	0.9919
low	lerp	0.623	0.685	0.9834	0.9804
low	slerp	0.624	0.688	0.9834	0.9804

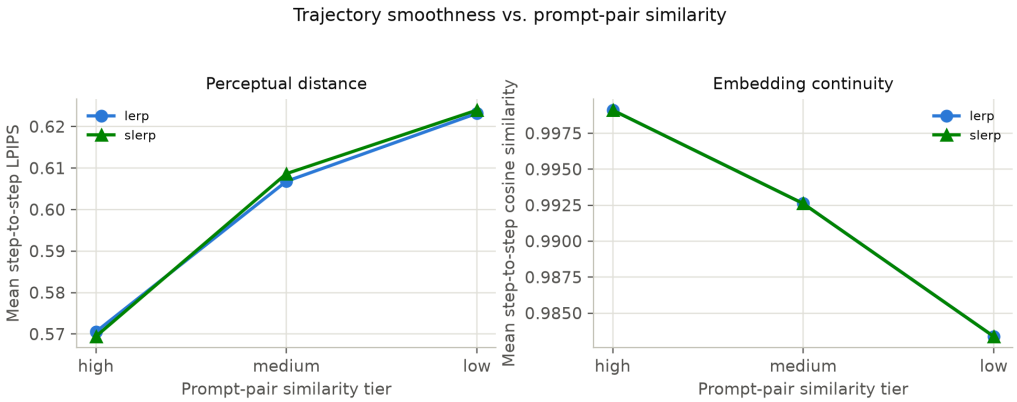


Figure 5.2.: Mean step-to-step LPIPS and cosine similarity across the three similarity tiers

Figure 5.2 and Table 5.2 show the same overall pattern. From the high- to the low-similarity tier, mean LPIPS increases monotonically while mean cosine similarity decreases. This indicates that the similarity of the source prompts, rather than the interpolation method, is the primary factor determining how strongly consecutive trajectory steps differ. Within each tier, the results for LERP and SLERP differ only negligibly for both metrics, and neither method shows a consistent advantage in smoothness.

The largest individual transition within each trajectory shows a different pattern from the mean. Maximum step-wise LPIPS remains close to 0.68 across all three similarity tiers, whereas mean LPIPS rises from approximately 0.57 to 0.62. Thus, even trajectories between highly similar prompts contain at least one substantial perceptual change.

The location of this strongest LPIPS transition is also structured rather than random. In 8 of the 12 trajectories, it occurs between $\alpha = 0.5$ and $\alpha = 0.75$. In the remaining four trajectories, it occurs between $\alpha = 0.25$ and $\alpha = 0.5$. It never appears in either outer interval, and LERP and SLERP identify the same interval for each prompt pair, which is broadly consistent with the tier-averaged curves shown in Figure 5.3 below. By contrast, the minimum step-wise cosine similarity occurs across all four intervals and does not exhibit a similarly consistent positional pattern.

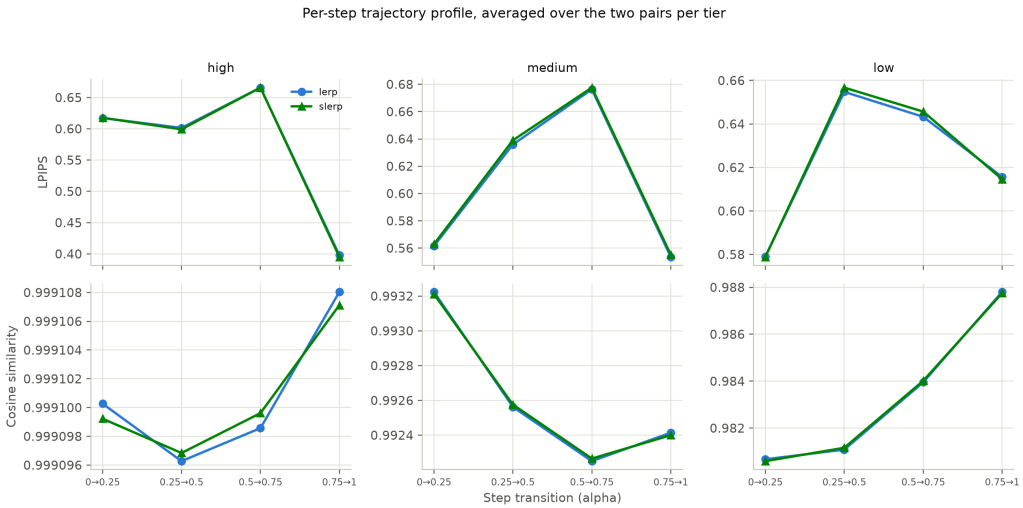


Figure 5.3.: Per-step LPIPS and cosine similarity profile across the four step transitions, averaged over the two pairs per tier

A structured qualitative review examined both endpoints and the transition with the highest LPIPS value for all 12 trajectories. In every case, the images generated at ($\alpha = 0$) and ($\alpha = 1$) were clearly consistent with their respective endpoint prompts, and no visible artifacts or incoherent outputs were observed at any interpolation step. The largest LPIPS changes generally corresponded to substantial shifts in scene or subject matter rather than to a decline in image quality. For example, pair 4 transitions from a bear-like monster to a

nurse portrait at ($\alpha = 0.75$) (Figure 5.5). Pair 5, the least similar pair in the experiment, produced the most distinctive intermediate result: at ($\alpha = 0.75$), the generated image depicts a spacecraft arriving above a medieval settlement, forming a coherent combination of both endpoint concepts rather than an abrupt transition between them (Figure 5.6).

Figure 5.4.: Representative trajectory, high-similarity tier (pair 2): forest fox to cyberpunk-styled fox. A: "a fox fursona, trending on artstation, by kawacy, furry art, digital art". B: "a fox fursona, trending on artstation, by kawacy, furry art, digital art, cyberpunk, high quality, backlighting"

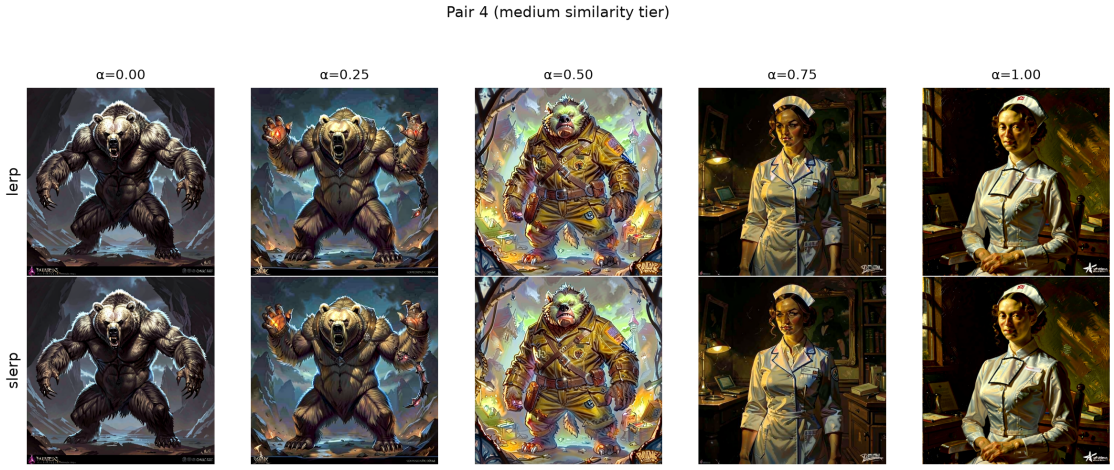


Figure 5.5.: Representative trajectory, medium-similarity tier (pair 4): grizzly bear monster to nurse portrait. A: "a ferocious grizzly bear monster with 4 arms, fantasy concept art, detailed, epic pose". B: "Female in nurse suit. By Rembrandt and artstation trending"

Finally, three representative trajectories were projected into the embedding space of `gustavosta_sd_prompts_v2_two_pass`. Rather than using the UMAP model fitted on the 11-prompt companion dataset, the analysis employed the large dataset's `two_pass` runtime reducer. Since this reducer was trained on a 20,000-point subset of the original dataset, it

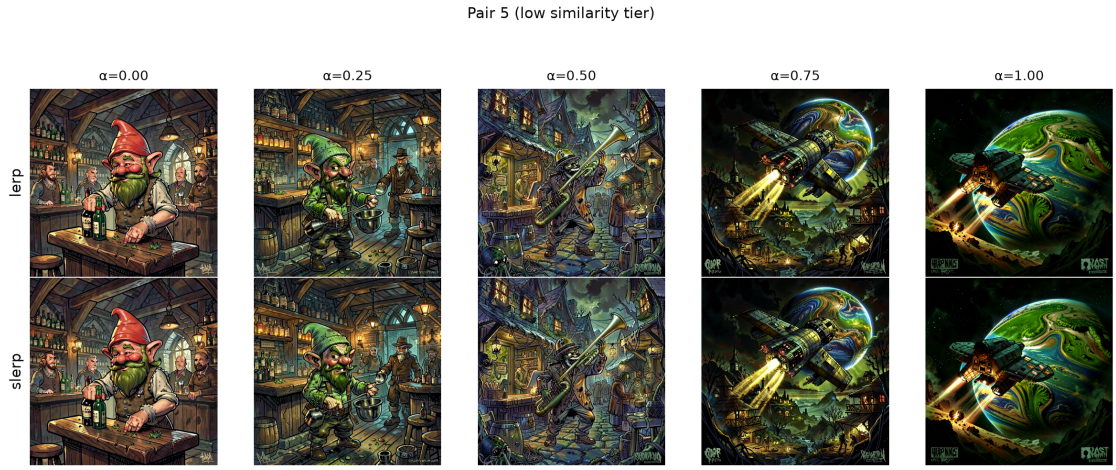
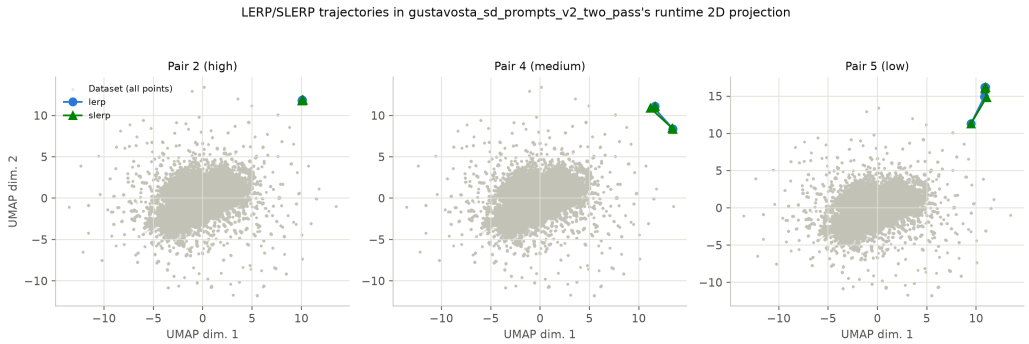


Figure 5.6.: Representative trajectory, low-similarity tier (pair 5): garden gnome in a medieval tavern to spacecraft, including the fusion result at $\alpha = 0.75$. A: "garden gnome bartending in a medieval tavern, concept art, john howe". B: "concept art of spacecraft arriving at the last habitable planet, shot from a horror film"

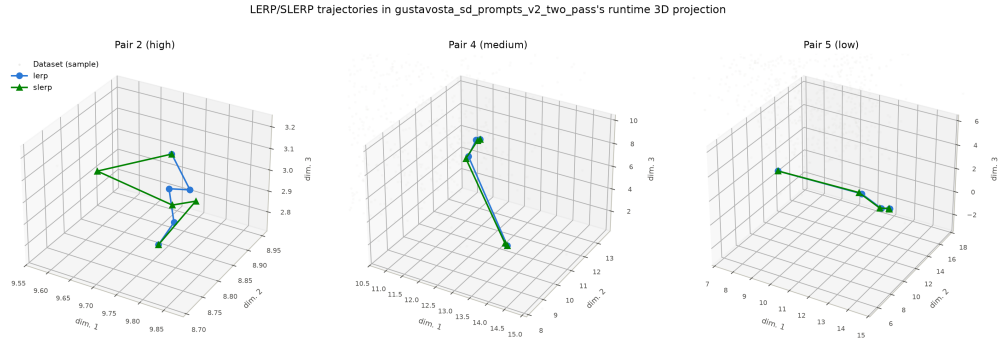
provides a more representative reference space for positioning the trajectory embeddings. The five saved compact embeddings from each trajectory were transformed using this reducer and plotted against the dataset's own canonical positions for context. The 2D figure shows all 61.746 points, while the 3D figure, zoomed into each trajectory's own local neighborhood.

Figure 5.7a shows that all three trajectories lie near the boundary of, or outside, the main density region of the full dataset. This is consistent with the finding in Section 5.2 that the `two_pass` runtime reducer provides a comparatively weak approximation for genuinely unseen points. The trajectory steps represent an even more pronounced out-of-sample case than the held-out prompts evaluated there, since they are derived from newly generated intermediate states rather than from prompts contained in the original corpus. Their position should therefore be interpreted primarily as a limitation of the runtime reducer's generalization rather than as reliable evidence of where these states belong in the underlying compact embedding space.

The projected trajectories also do not always move consistently between their endpoints, although LERP and SLERP follow simple, well-defined paths in the original conditioning space. This is not necessarily contradictory, since UMAP is designed to preserve local neighborhood relationships rather than global distances or straight-line geometry. A linear trajectory in a high-dimensional space is therefore not guaranteed to remain straight or monotonic after dimensionality reduction. In the three-dimensional view shown in Figure 5.7b, pair 2's paths separate visibly at the intermediate steps once the third dimension is included, despite its generated images (Figure 5.4) and two-dimensional trajectories being nearly indistinguishable between LERP and SLERP. By contrast, the LERP and SLERP



(a) Projected through the dataset's runtime 2D reducer, against all 61.746 dataset points



(b) The same three trajectories in the runtime 3D reducer's output, axis limits zoomed to each trajectory's own neighborhood rather than the full sample

Figure 5.7.: LERP/SLERP trajectories of three representative pairs, in the runtime 2D and 3D reducer output

trajectories for pairs 4 and 5 remain close to a common path in all three dimensions. This shows that lower-dimensional projections can hide structures that become visible only when additional dimensions are considered. However, the present results do not establish whether the observed trajectories represent genuine local structure preserved by the reducer or instead result from its limited accuracy when projecting unseen points.

Within this experiment, source-prompt similarity was the main factor determining the magnitude of change between consecutive trajectory steps. By comparison, the choice between LERP and SLERP had only a minor effect on both perceptual and embedding-based measures, with both methods producing visually and quantitatively similar generated images across all three similarity tiers, despite their trajectories sometimes taking visibly different paths through the projected embedding space.

These results do not establish that interpolation geometry is generally irrelevant to image generation. They show only that no meaningful difference between LERP and SLERP could be detected under the conditions examined here. The findings also place SLERP's usual theoretical motivation in context: although SLERP is generally motivated by better-preserved vector norms compared to LERP's straight-line blend, this theoretical advantage did not

translate into measurably or perceptually smoother trajectories in the evaluated system and at the tested scale.

5.4. Direction transfer with Delta / Direction Walk

Section 4.8 introduced Direction Walk as the transfer of a direction defined by two anchor prompts to an independent starting point. Unlike interpolation, the operation is not restricted to the interval between the anchors. Values of α may extend across $[-2, 4]$, allowing the direction to be applied in reverse or extrapolated beyond a complete transfer. The present section evaluates whether such a direction produces a meaningful and consistent transformation when applied to unrelated starting contexts across this extended range.

Three directions were selected for the experiment. Each was applied to two independent starting prompts C. The first represented a near context with clear conceptual overlap with the anchor prompts A and B. The second represented a far context without an obvious shared subject. All source prompts originate from `gustavosta_sd_prompts_v2_two_pass` and were loaded through a small companion dataset constructed in the same manner as in the previous section.

Each of the six walks was evaluated using the same sequence of direction strengths: -2, -1, 0, 1, 2, 3, 4. All runs used the shared generation configuration defined at the beginning of this chapter. Each walk therefore produced seven generated images and seven corresponding compact embeddings. The step at $\alpha = 0$ represents the unmodified starting prompt C.

Two complementary forms of comparison were calculated from the saved images and embeddings, using the LPIPS and cosine-similarity measures introduced earlier. Consecutive-step comparisons evaluate each point against its immediate predecessor and therefore describe local continuity along the walk. Baseline-relative comparisons instead evaluate every step against the unmodified starting state at $\alpha = 0$. They measure how strongly the result departs from its original context as the direction strength increases. This second perspective is particularly relevant for Direction Walk because it directly tests whether a direction derived from one prompt pair induces a systematic transformation when transferred to a different starting context.

How far a point moves in compact embedding space does not reliably predict how much the generated image changes perceptually, as shown in Table 5.3. For all six walks, the minimum baseline cosine similarity occurs at $\alpha = 4$: since the tested schedule reaches further forward (to $\alpha = 4$) than in reverse (to $\alpha = -2$), this is simply the largest-magnitude step tested, not a special property of that particular alpha. However, the maximum baseline LPIPS value, which identifies the greatest perceptual deviation from the starting image, occurs at the same step in only two cases. In the remaining four walks, perceptual change peaks earlier.

Table 5.3.: Consecutive-step and baseline-relative summary per direction and starting context. Baseline values report the single largest drift from the $\alpha = 0$ step (LPIPS) or the single closest departure from it (cosine similarity), together with the alpha at which it occurred

Direction	Context	Consec. LPIPS mean	Consec. LPIPS max	Baseline LPIPS max (α)	Baseline cosine-sim. min (α)
1 (cyberpunk styling)	near	0.636	0.797	0.820 (α=3)	0.756 (α=4)
1 (cyberpunk styling)	far	0.574	0.597	0.606 (α=4)	0.837 (α=4)
2 (shoes → jacket)	near	0.565	0.707	0.846 (α=-2)	0.895 (α=4)
2 (shoes → jacket)	far	0.543	0.642	0.710 (α=2)	0.919 (α=4)
3 (identity swap)	near	0.579	0.643	0.664 (α=4)	0.746 (α=4)
3 (identity swap)	far	0.647	0.696	0.790 (α=3)	0.748 (α=4)

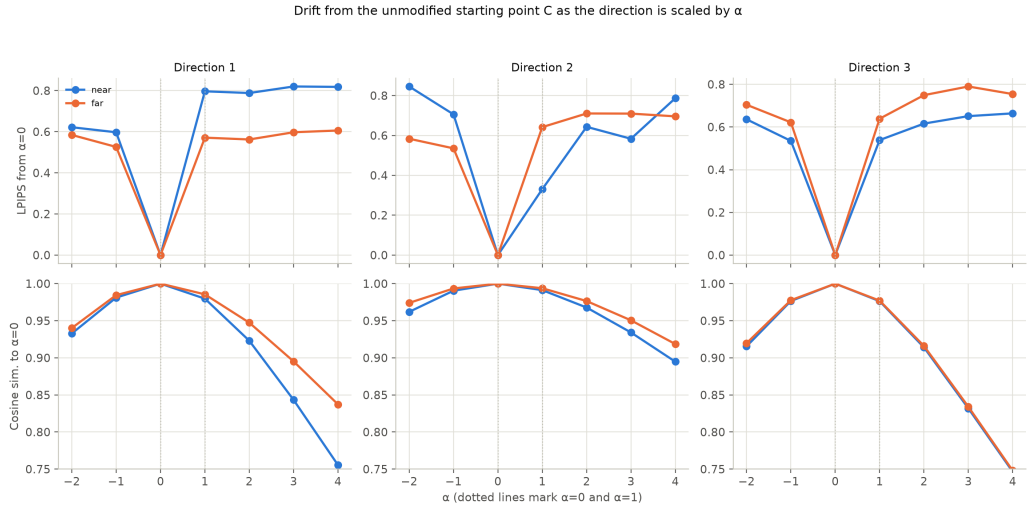


Figure 5.8.: Baseline-relative LPIPS and cosine similarity to the $\alpha = 0$ step, across the full alpha schedule, near and far context

The clearest example is the near context of Direction 2, where the highest LPIPS value occurs at $\alpha = -2$, although Figure 5.8 shows the compact embedding remains relatively close to its baseline there, with a cosine similarity of approximately 0.96.

The baseline cosine-similarity curves in Figure 5.8 are approximately symmetric around $\alpha = 0$ for equal movement in opposite directions: moving away from C by the same amount in either direction changes the result's angle from its baseline by a similarly small or large amount, so applying the same direction vector with opposite signs produces near-identical cosine similarity to the baseline. Baseline LPIPS does not exhibit a comparable symmetry. Its values rise and fall unevenly across the schedule, reflecting the mismatch between the points of maximum embedding displacement and maximum perceptual change observed in the summary table.

Together, these measures distinguish the geometric behavior of the operation from its visual outputs. Movement in compact embedding space follows the expected directional pattern and is approximately symmetric for equal positive and negative values of α. The generated images, however, do not respond proportionally or symmetrically to this movement. Their

greatest perceptual deviation may occur before the largest embedding displacement or even at the opposite end of the tested range. This suggests that the diffusion model maps similar geometric displacements to visual changes in a nonlinear and context-dependent manner.

The largest local perceptual transition, measured as the maximum LPIPS value between consecutive steps and shown in Figure 5.9, also varies across the α schedule and does not consistently align between the near and far contexts of the same direction. For Direction 1, the near context exhibits its strongest transition between $\alpha = 0$ and $\alpha = 1$, whereas the far context reaches a smaller maximum at the opposite end of the schedule, between $\alpha = 3$ and $\alpha = 4$. For Direction 2, both contexts peak relatively early: the near context between $\alpha = -2$ and $\alpha = -1$, and the far context between $\alpha = 0$ and $\alpha = 1$. Direction 3 shows another pattern, with the near context peaking between $\alpha = -2$ and $\alpha = -1$ and the far context between $\alpha = 1$ and $\alpha = 2$.

Across the six walks, no single interval consistently represents the most abrupt transition. Even when the same direction is applied, its near and far contexts may reach their maximum perceptual change in widely separated parts of the schedule. This contrasts with the LERP and SLERP results in Section 5.3, where the strongest transition occurred within the same interior interval for both methods applied to a given prompt pair. The variability observed here is consistent with the qualitative results discussed below: depending on the starting context, a transferred direction may produce a clear transformation, have only a limited visible effect, or introduce an unrelated subject.

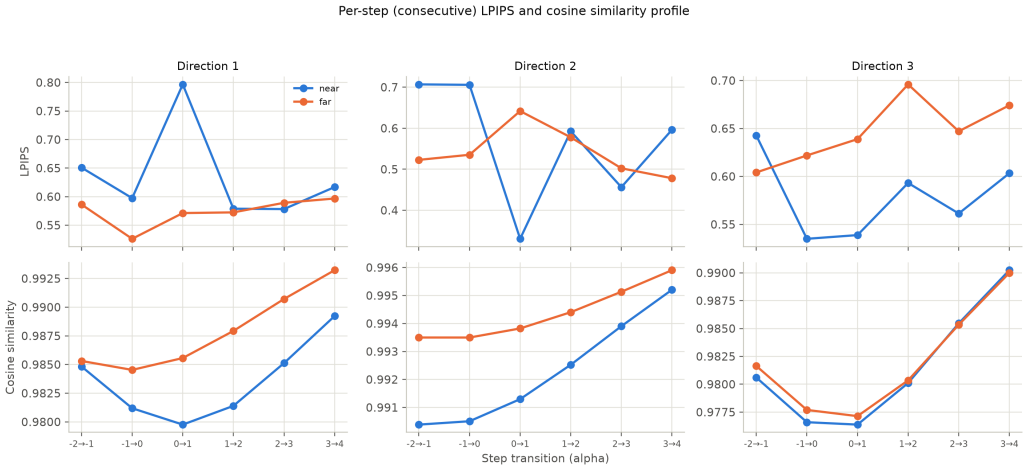


Figure 5.9.: Per-step (consecutive) LPIPS and cosine similarity profile across the six step transitions, near and far context

A structured qualitative review of the endpoints and the highest-LPIPS transition in each of the six walks supports this interpretation. Direction 2, defined by the transformation from shoes to a jacket (Figure 5.11), produces the most consistently coherent behavior. In both contexts, the original product remains recognizable across the full range, while increasing

α introduces progressively stronger weathering, patchwork, and armor-like detailing. The subject itself is not replaced, although some steps place the product on a humanoid model.

Direction 1, representing cyberpunk styling (Figure 5.10), transfers successfully to the near context. As α increases, the fox acquires increasingly pronounced cybernetic components and neon-lit details while remaining identifiable. In the far context, however, the seaside village is replaced by an unrelated humanoid figure at $\alpha = 3$ and $\alpha = 4$, rather than being transformed into a cyberpunk interpretation of the same scene. A possible explanation is that the village provides no clear visual subject to which cybernetic components can be attached, leading the model to introduce a person who can carry the transferred attribute.

Direction 3, representing an identity transformation (Figure 5.12), also transfers clearly to its near context, which contains another named person. Skin tone and styling change in the expected direction, while the original sorceress or warrior composition remains recognizable throughout the walk. The far context, an umbrella, cannot express the identity-related component of the direction, and no real person appears at any value of α. Nevertheless, the object becomes progressively more ornate, jeweled, and gold-toned as α increases. These changes reflect stylistic qualities shared by the two anchor prompts, such as ornamentation and intricacy, even though the identity attribute itself has no compatible subject on which to act. In this case, the conditioning of the starting prompt preserves the original object while allowing only semantically compatible aspects of the transferred direction to appear.

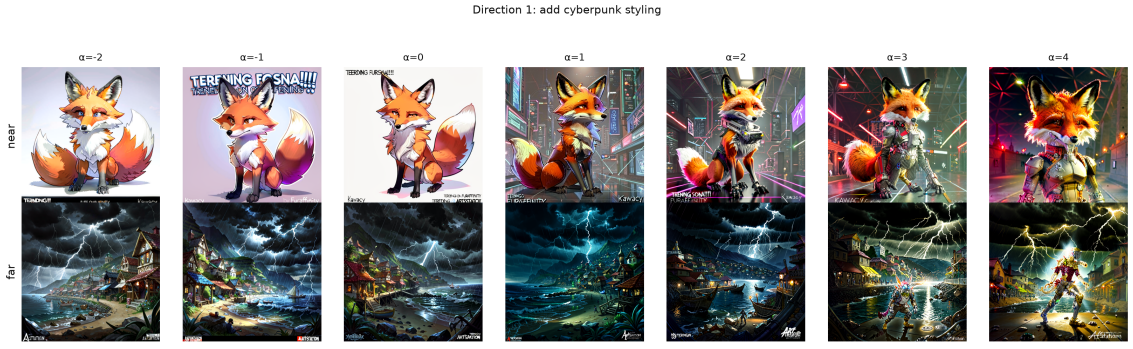


Figure 5.10.: Direction 1 (add cyberpunk styling), near and far context across the full alpha schedule. A: "a fox fursona, trending on artstation, by kawacy, furry art, digital art". B: "a fox fursona, trending on artstation, by kawacy, furry art, digital art, cyberpunk, high quality, backlighting". Near C: "an anthropomorphic fox, fursona!!!! trending on furaffinity, by kawacy, trending on artstation". Far C: "a seaside village, detailed, fantasy style, stormy day, trending on artstation"

Finally, each direction's A/B anchors and both walks were projected into `gustavosta_sd_-prompts_v2_two_pass`'s own runtime reducer, the same one used in the previous section. The 2D figure plots this against all 61.746 of the dataset's own canonical positions for

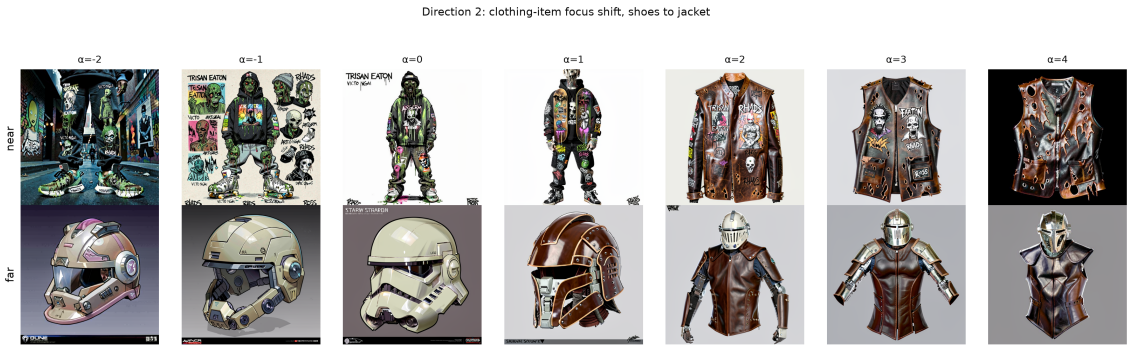


Figure 5.11.: Direction 2 (shoes to jacket focus shift), near and far context across the full alpha schedule. A: "zombie jordan shoes female hiphop streetwear drip, tristan eaton, victo ngai, artgerm, rhads, ross draws". B: "zombie leather jacket full body hiphop streetwear drip, tristan eaton, victo ngai, artgerm, rhads, ross draws". Near C: "zombie skateboard full body hiphop streetwear drip, tristan eaton, victo ngai, artgerm, rhads, ross draws". Far C: "new stormtrooper/clone trooper helmet design, concept art"

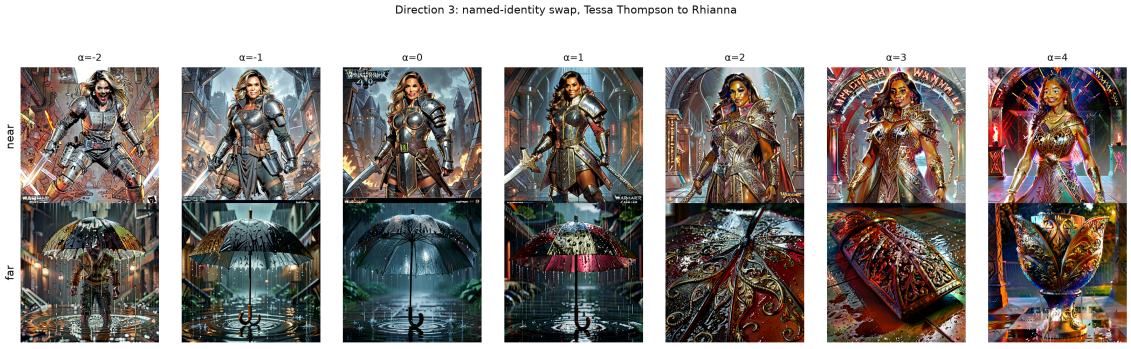
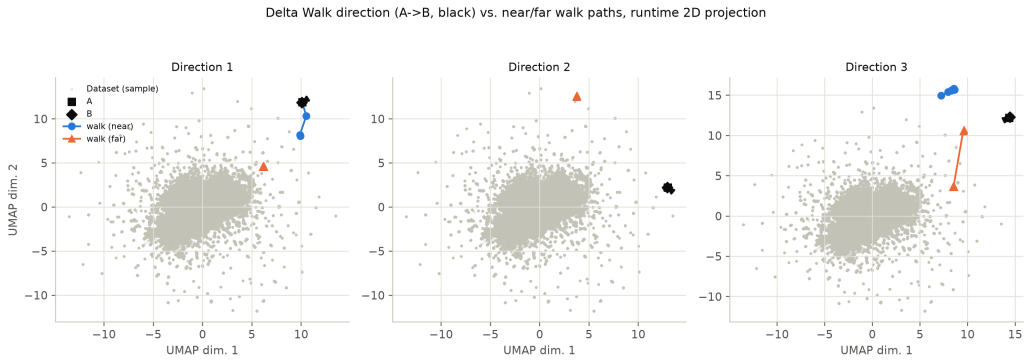
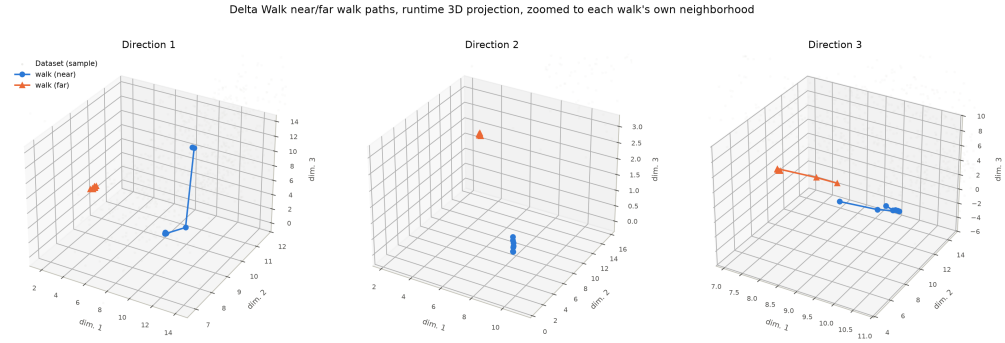


Figure 5.12.: Direction 3 (named-identity swap), near and far context across the full alpha schedule. A: "Tessa Thompson as a sorceress, passionate, seductive, sweaty, intricate dressed in ornate blue robes and staff, insane, intricate,". B: "Rhianna as a sorceress, passionate, seductive, sweaty, intricate dressed in ornate blue robes and staff, intricate,". Near C: "Sofia Vergara as a battle sister from Warhammer 40k, highly detailed, intricate, concept art, artstation". Far C: "An umbrella, extreme detailed, wet, photorealistic, octane render, 8k, by greg rutkowski and studio ghibli"

context and, new for this section, the A-to-B direction itself drawn as a reference arrow. The 3D figure instead zooms into each walk's own neighborhood.



(a) The A→B direction (black arrow) and both walks' projected positions



(b) The same three directions, axis limits zoomed to each walk's own neighborhood

Figure 5.13.: Delta/Direction Walk anchors and both walks' projected positions, runtime 2D and 3D reducer output

In the two-dimensional projection, both anchor prompts and the trajectories generated from them lie outside the main density region of the dataset, reproducing the pattern observed for the interpolation trajectories in Section 5.3. Even at $\alpha = 4$, the seven projected points of each walk cover only a short distance relative to both the corresponding A-to-B direction and the surrounding point cloud. The three-dimensional view in Figure 5.13b makes this limited displacement more visible. Each trajectory remains close to its position at $\alpha = 0$, even in cases where Figure 5.8 shows substantial perceptual change in the generated images. Large image-level changes therefore do not necessarily require a similarly large displacement in the projected compact embedding space.

Figure 5.13a and Figure 5.13b further show that the near and far contexts of the same direction occupy distinct neighborhoods rather than different positions within a shared local region. For all three directions, the projected points of each walk remain more closely grouped with their own starting context than with the other context to which the same direction was applied. This pattern also holds for the near context of Direction 1, which exhibits the greatest internal spread among the six walks: its points still form a recognizable

local group that remains separate from the corresponding far-context trajectory. Neither the projected path length of an individual walk nor the separation between the near- and far-context trajectories provides a direct measure of semantic transfer quality. This reinforces the mismatch between projected geometry and perceptual content observed earlier in this section and in Section 5.3. Whether a transferred direction remains semantically meaningful must therefore be assessed from the generated images rather than inferred from the projection alone.

Within this experimental setup, Direction Walk produces approximately smooth and symmetric movement in compact embedding space across the tested α range. The corresponding perceptual changes, however, are neither symmetric nor consistently strongest at the most extreme extrapolation values. The results further show that the semantic structure of the starting context can weaken, redirect, or absorb a transferred direction instead of expressing it as a direct attribute change.

For users applying Direction Walk as a semantic-editing method, neither the extrapolation strength nor the geometric symmetry of the operation provides a reliable indication of how strongly or predictably the edit will appear in the generated image. The outcome depends largely on whether the starting context is compatible with the transferred attribute. Contexts that share conceptual similarities with the original anchor pair are more likely to express the direction as a coherent and controllable edit, whereas unrelated contexts may weaken the effect, distort the result, or replace the original subject entirely.

5.5. Semantic Axis Projection

Section 4.8 introduced Semantic Axis Projection, in which users select two groups of points representing opposite poles of a semantic contrast. All loaded points are then assigned a score along the primary axis connecting these groups. The operation may additionally derive a second axis from the residual variation not explained by the first. As emphasized in Section 4.8, these axes are exploratory and depend on the selected examples and dataset. They should therefore be understood as approximations of semantic structure rather than universal semantic measures. Building on this limitation, the present section addresses two narrower questions: whether a primary axis based on only a few example points still applies to unseen points sharing the same theme, and whether the automatically derived second axis has a meaningful interpretation.

Two semantic contrasts were constructed using the canonical clustering of `gustavosta_sd_prompts_v2_two_pass` as an independent, dataset-specific reference for thematic membership. The target clusters were selected by inspecting representative prompts from the largest clusters. The first contrast is a semantically near pair consisting of cyberpunk prompts from

cluster 70, containing 531 members, and steampunk prompts from cluster 23, containing 338 members. These clusters represent visually related "-punk" aesthetics. The second is a semantically far pair consisting of dragon and monster prompts from cluster 99, containing 356 members, and fairytale architecture and interior prompts from cluster 109, containing 633 members. These clusters represent largely unrelated subject categories.

The generalization of the primary axis is evaluated using two complementary measures. For each target cluster, 20 members are selected randomly as anchors, while all remaining members are withheld from axis construction. The first measure evaluates whether the primary axis continues to separate these unseen samples, tested using the Mann-Whitney U test implemented by `scipy.stats.mannwhitneyu`. The null hypothesis states that a randomly selected held-out Group B member is no more likely to receive a higher score than a randomly selected held-out Group A member, meaning that thematic membership provides no information about a point's position on the axis. The alternative states that held-out members of Group B receive systematically higher scores than those of Group A, meaning that the axis correctly separates the two groups' held-out members by their true thematic membership, not only by the anchor points used to build it. The test compares every possible pair of one held-out Group A point and one held-out Group B point, and measures how often the Group B point scores higher. This proportion is equivalent to the area under the ROC curve (AUC): a value of 1 means perfect separation, while a value near 0.5 means the ranking is no better than chance. Unlike a fixed-threshold classification measure, this comparison remains informative when the score distributions are skewed or contain outliers. [23]

This result is not intended to show that the original HDBSCAN clusters are surprising or independently validated. Their assignments already derive from the geometry of the compact representations through the clustering pipeline described in Section 4.6. Instead, the test asks whether a single direction estimated from only 20 examples per side preserves enough of that higher-dimensional thematic structure to order previously unseen cluster members correctly.

Support-band precision provides a second test of primary-axis generalization. Rather than considering the complete score distribution, it evaluates the extreme regions that the operation exposes for downstream analysis, including contrastive axis-term extraction. Even an axis that separates the target groups correctly would have limited practical value if its extreme bands were dominated by unrelated samples.

For this evaluation, the 20 anchors from each target cluster are supplied to the axis-construction procedure together with all held-out members of both clusters and a background sample of 4.000 points drawn randomly from neither target cluster. Anchors, held-out members, and background samples are therefore scored and ranked jointly, resulting in approximately 4.900 to 5.000 loaded points for each semantic contrast. At this scale, the

support-band rule introduced in Section 4.8, 10 percent of the loaded points, constrained to a minimum of 24 and a maximum of 80, selects the maximum of 80 points at each pole.

Including unrelated background samples gives non-target points a realistic opportunity to enter the extreme regions of the axis. Support-band precision is therefore defined as the proportion of the 80 points at a given pole that belong to the cluster represented by that pole. Points from the opposing target cluster and from the unrelated background sample count as incorrect assignments. Consequently, values below 1 represent contamination of the axis extremes.

The interpretability of the automatically derived second axis cannot be assessed through the same held-out procedure. Unlike the primary axis, the second axis is not defined by independently specified semantic groups. Its apparent poles are discovered from the residual variation itself, meaning that evaluating the axis against those same extremes would be circular. Its interpretation is therefore examined qualitatively after the corresponding axis results are presented later in this section.

Table 5.4.: Held-out separation and support-band precision per axis pair. Anchor count is 20 per cluster throughout; held-out counts are the remainder of each cluster. Precision is followed by the number of background/unrelated points among the 80 band members

Pair	Side A (n heldout)	Side B (n heldout)	Held-out AUC	Band-A prec. (bg./80)	Band-B prec. (bg./80)	Axis-2 var.
1: cyberpunk vs. steampunk	steampunk (318)	cyberpunk (511)	0.999	0.863 (11/80)	1.000 (0/80)	0.132
2: creatures vs. fairytale architecture	creatures (336)	architecture (613)	1.000	0.900 (8/80)	0.975 (2/80)	0.120

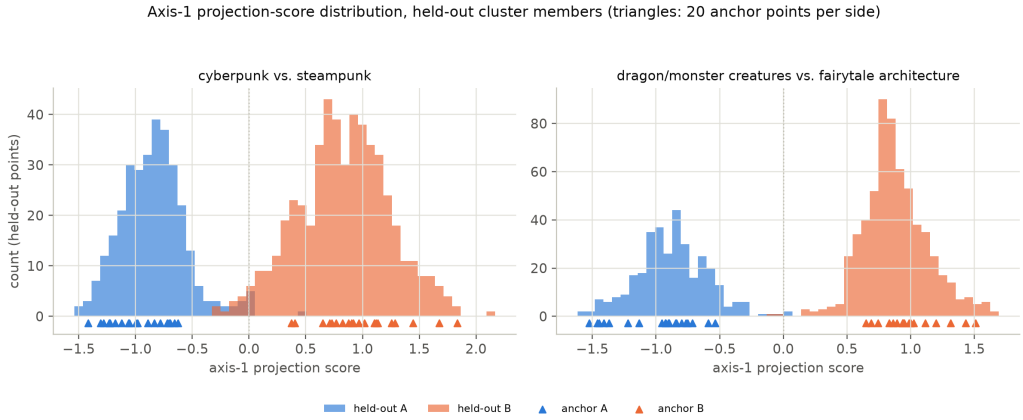


Figure 5.14.: Axis-1 projection-score distribution for held-out cluster members; triangles mark the 20 anchor points per side

The near-perfect AUC values indicate that the primary axis generalizes strongly from only 20 anchor examples per side. As shown in Figure 5.14, the held-out samples from Groups A and B form almost completely separate score distributions. For pair 1, only a narrow overlap near the center of the axis prevents the AUC from reaching 1.000, resulting instead in 0.999. For pair 2, no visible overlap remains.

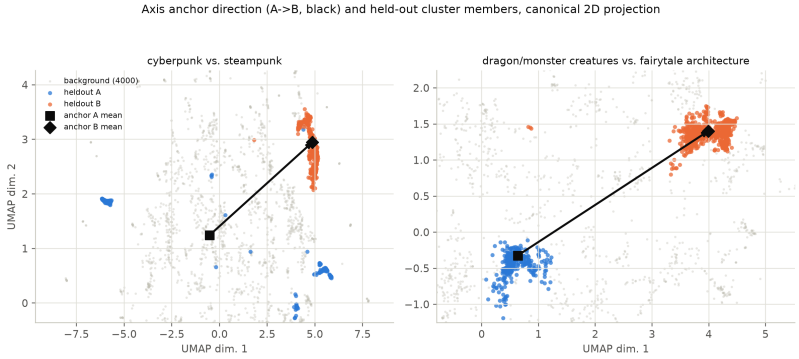
This statistical separation is supported by the axis terms extracted independently from the support bands. For pair 1, terms such as `steampunk`, `artstation`, and `trending` contrast with `cyberpunk`, `art`, and `concept`. For pair 2, `dragon`, `artstation concept`, and `art smooth` contrast with `medieval`, `artstation`, and `trending`. These terms reflect the themes for which the source clusters were selected, even though those themes were not supplied explicitly to the operation. The axis therefore does more than separate two groups numerically: it also recovers a semantic contrast that is recognizable from their content.

Support-band precision is lower than AUC in three of the four evaluated bands, and effectively matches it in the fourth. This does not contradict the AUC results, because the two measures evaluate different properties. AUC considers only the relative ordering of the two held-out target groups. Support-band precision is stricter because the extreme points exposed for downstream analysis must also compete with 4.000 unrelated background samples for the 80 available positions at each pole. Consequently, an axis may separate the two target themes almost perfectly while still admitting unrelated points into its extreme regions. This is most visible for cluster 23, whose support band contains 11 background samples (Table 5.4), compared with 0, 2, and 8 in the other three bands.

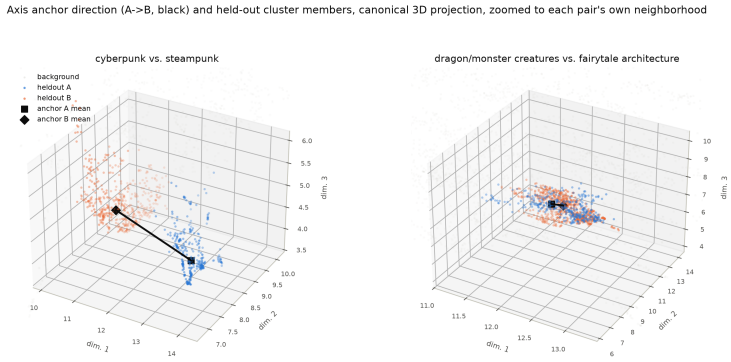
Taken together, these three measures answer the first question posed at the start of this section: a small, user-realistic anchor sample generalizes almost perfectly to the underlying theme, with only minor contamination among the specific extreme points.

The primary-axis score is computed in the compact embedding space rather than in the nonlinear UMAP display space. Its separation is therefore not guaranteed to remain visually apparent after projection. Figure 5.15a and Figure 5.15b show that it nevertheless remains visible for both pairs: held-out members of Groups A and B occupy distinct regions on opposite sides of the arrow connecting the anchor means. The three-dimensional view reveals an additional detail for pair 2. Although its groups appear clearly separated in two dimensions, some points overlap along their shared boundary when the third dimension is included. This suggests that part of the apparent 2D separation results from projection. The partial overlap for pair 2 does not conflict with its AUC of 1.000, because AUC is calculated from scores in the original compact space rather than from the reduced display coordinates. Overall, the thematic distinction remains visible after projection, although with some loss of clarity for pair 2.

For each pair, an automatically derived second axis was computed independently from the residual variation left after removing the primary axis. Both instances exceeded the 5 percent reporting threshold introduced in Section 4.8, explaining 13.2 percent and 12.0 percent of the residual variance for pair 1 and pair 2 respectively, but their interpretation is less direct than that of the primary contrast. For pair 1, one extreme contains city and landscape prompts such as "steampunk city ... trending on artstation" and "good day, morning, landscape ... steampunk, forest, wood, lost world ...", while the other is dominated by named-character



(a) Axis anchor direction (black arrow, between the two groups' 20-point anchor means) against held-out cluster members (blue/orange) and a 4.000-point background sample



(b) The same two pairs in the 3D projection, axis limits zoomed to each pair's own held-out neighborhood

Figure 5.15.: Axis anchor direction and held-out cluster members, canonical 2D and 3D projection

portraits such as "Loba from Apex Legends ... digital illustration portrait" or "Monica Bellucci as an angel", which could be interpreted as a scene-versus-portrait distinction. Pair 2 is less coherent, because one side combines "organic cyborg" and "organic mecha" prompts with an unrelated "city made of pages of paper" scene, whereas the opposite side again contains mainly named-character portraits. The three axis terms the operation derives for axis-2 ("detailed digital / style / full" vs. "steampunk / sharp / intricate" for pair 1 and "anime / detailed digital / trending artstation" vs. "focus / concept / sharp" for pair 2) are noticeably less informative than reading the underlying prompts directly, since single-word document-frequency terms cannot capture a composition-level distinction fully. The second axis should therefore be understood as simply the strongest remaining direction of variation after removing the primary contrast, not necessarily as another thematically related semantic dimension. This means, axis-2 is a much less reliable indicator of thematic structure than the primary axis, whose interpretation must be approached with more caution. Improving its interpretability, for example through additional constraints on the discovered direction or alternative axis-discovery techniques, remains an open direction for future work.

To summarize, a primary axis constructed from a small random set of anchors generalizes almost perfectly to unseen members of the same clusters, and its extracted axis terms also recover the intended thematic distinction. By contrast, the automatically discovered second axis satisfies the numerical reporting criterion but is substantially harder to interpret. Its meaning becomes noticeable mainly through inspection of the extreme prompts, revealing a gap between the structure detected by the operation and the explanation presented to the user.

5.6. Consistency Check

Section 4.8 introduced Consistency Check as an operation that generates prompt variants at a configurable strength and evaluates their deviation from the base prompt using a consistency score and predicted cluster assignments. Because the operation changes both its generation mechanism and its LLM style tier as strength increases, this section examines whether the resulting scores remain continuous and well calibrated or whether these internal transitions introduce discontinuities.

Three base prompts of 6, 14, and 45 words, drawn from `imagenet1k_labels_classic` and `gustavosta_sd_prompts_v2_two_pass`, were evaluated at seven variation strengths ranging (0.05, 0.2, 0.3, 0.35, 0.5, 0.7, 0.9), covering both generation mechanisms and all three style tiers. Five variants were generated per base prompt. Cluster stability, defined as the proportion of variants assigned to the same predicted cluster as the base prompt, was added as a second measure to examine the operation's stability from a different angle. Due to hardware constraints, Qwen2.5-3B-Instruct was used for paraphrase generation instead of the default Qwen2.5-7B-Instruct, which may have reduced the quality of the generated variants.

As described in Section 4.8, lower consistency scores indicate that the generated variants remain close to the base prompt, while higher values indicate greater drift. Although variation strength can be configured continuously, Figure 5.16 shows that its practical effect is considerably more coarse. Below the mechanism boundary at 0.35, the dropout-count formula rounds to the removal of a single word for the short base prompt at strengths 0.05, 0.2, and 0.3. These settings therefore produce identical variants despite their different numerical values. At strengths of 0.7 and 0.9, all base prompts receive the same "substantial change" instruction. Combined with deterministic decoding, this results in identical LLM outputs at both settings.

The transition at 0.35 is also discontinuous. For the medium-length base prompt, the consistency score decreases from approximately 0.31 to 0.14 when the operation switches to the mechanism intended to produce stronger paraphrases. In addition, four of the twelve

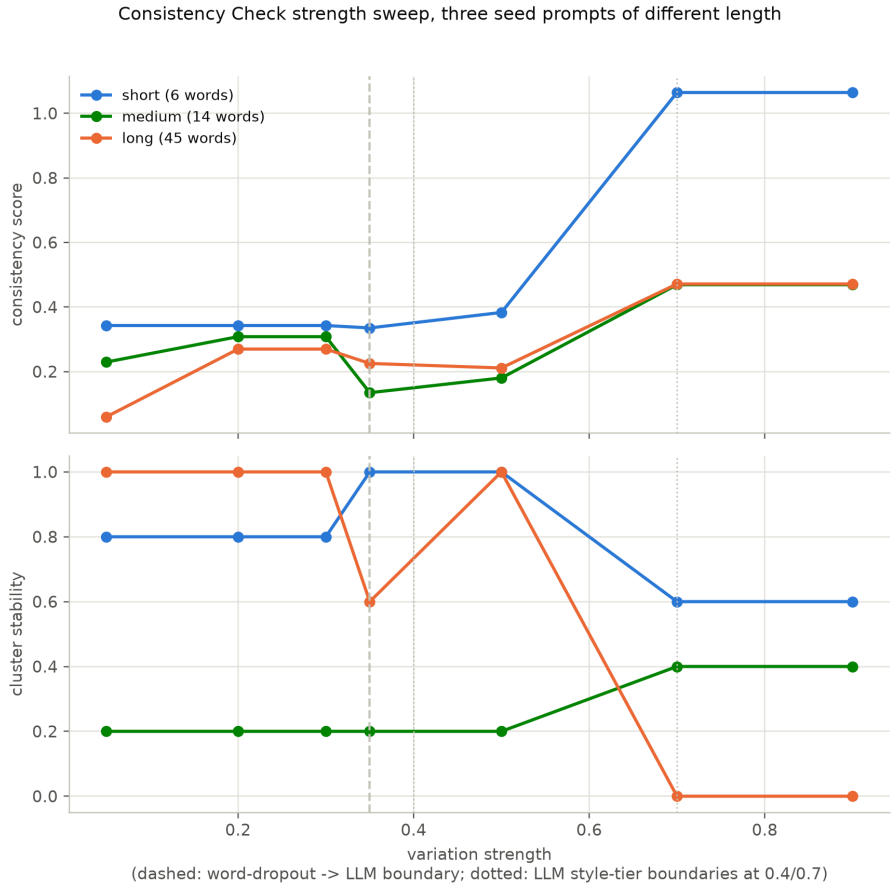


Figure 5.16.: Consistency score and cluster stability across the strength sweep. Dashed line: word-dropout to LLM-paraphrase boundary. Dotted lines: LLM style-tier boundaries

LLM-based runs did not return the required number of distinct variants and therefore invoked the base-prompt-copy padding fallback described in Section 4.8. Since a copied base prompt has a cosine similarity of 1.0 by construction, this fallback artificially lowers the consistency score and increases cluster stability. This effect is attributable partly to the use of the smaller substituted language model rather than to the variation mechanism alone.

Cluster stability has a less systematic pattern and does not reliably follow the consistency score: although both are intended to reflect how far the variants have drifted from the base prompt, a rising or falling consistency score does not reliably correspond to a matching change in cluster stability. For the medium-length base prompt, stability remains at 0.20 through strength 0.5 even though the consistency score changes substantially. At each of these strengths, no more than one of the five variants remains in the base prompt's cluster 134, while the others are assigned mainly to the related cluster 27, as illustrated in Figure 5.17. Real semantic drift therefore does not necessarily produce corresponding changes in cluster stability.

The long base prompt exhibits the opposite pattern. Its cluster stability rises to 1.00 at strength 0.5 before falling to 0.00 at strengths 0.7 and 0.9, a change too large to be explained solely by the coarse resolution of five variants. This behavior is explained by the cluster-prediction mechanism classifying all variants as noise, rather than reassigning them to a different cluster. The consistency score and cluster assignment should therefore be understood as structurally different measures: one captures continuous displacement in embedding space, while the other records discrete movement across learned cluster boundaries. They may consequently diverge rather than change in parallel.

Figure 5.17.: Base prompt "medium" (Dwayne Johnson lightsaber) rendered at its unmodified base prompt and three of its measured variants, spanning word dropout, LLM minor rewording, and LLM aggressive paraphrase

The qualitative example in Figure 5.17 supports both findings identified above: the coarse response of the consistency score and its limited correspondence with cluster stability. The figure renders the base prompt, "dwayne johnson holding a purple lightsaber wearing brown jedi robes, concept art, ultra realistic", together with three previously measured variants, so each variant's visual outcome can be compared directly against its own measured similarity to the base prompt. At strength 0.2, the word-dropout variant "holding a purple lightsaber wearing brown jedi robes, art, ultra realistic" removes the name "Dwayne Johnson" and eliminates the actor's likeness, although the consistency score remains on the same plateau as the other word-dropout conditions. At strength 0.35, the LLM minor-rewording variant "purple lightsaber in hand for dwayne johnson, brown robes, jedi style, ultra detailed concept art" preserves the name and recognizable identity while mainly altering the visual style. Nevertheless, the variant is reassigned from cluster 134 ("Hyperrealistic Celebrity Art") to cluster 27 ("Star Wars Concept Art"), suggesting that the cluster prediction is influenced more strongly by the lightsaber and Jedi-related content than by the celebrity reference. At strengths 0.7 and 0.9, the most aggressive paraphrase, "purple-hued lightsaber wielder in brown robes, futuristic concept, ultra-realistic art", removes the name, introduces an unrelated figure and setting, and remains in cluster 27. These examples show that neither the consistency score nor the cluster assignment alone provides a complete account of semantic stability.

Within this experiment, variation strength behaves more discretely than its continuous range suggests. Different structural causes produce this effect in the two generation mechanisms. As a result, even a small change in the numerical consistency score can coincide with a substantial shift in identity, subject, or scene. The findings are also specific to the use of Qwen2.5-3B-Instruct and may differ with another paraphrasing model. Differences across the three base prompts further indicate that prompt length and content influence plateau width and cluster behavior alongside the selected strength.

Consistency Check should therefore be interpreted as an exploratory estimate of stability rather than a definitive measure. Its coarse behavior follows directly from the current design, which combines integer word-dropout counts with a small number of discrete LLM style tiers. These choices make the operation simple and predictable, but limit fine-grained control. Future work could introduce more gradual dropout weighting or additional style levels to provide a more continuous response across the strength range.

5.7. Latent Probe

Section 4.8 introduced Latent Probe as an operation that estimates a target embedding from a user-selected position in the projected space. The estimate, denoted q, is obtained by combining the compact embeddings of nearby reference points using inverse-distance weighting. It is then used for cluster prediction, nearest-prompt retrieval, and, optionally, an LLM-based search for a new prompt whose embedding lies close to q. Section 4.8 explicitly states that this is an exploratory approximation, not a claim of an exact recovery. The present section tests that claim directly by examining whether local geometry in the projection retains enough usable semantic information about the underlying prompt representations to support these tasks. Two held-out tests are used: Test A examines whether probing a point's true projected position reconstructs its original compact embedding, while Test B evaluates whether the LLM-based search can generate a new prompt that remains semantically close to the held-out source prompt.

Test A is performed on two datasets with substantially different levels of prompt diversity. The first uses `imagenet1k_labels_classic` and reuses the 800/200 split introduced in Section 5.2. The split is applied directly to the dataset's existing coordinate and embedding arrays, with only the 800 training points made available as reference samples for inverse estimation. No projection model is refitted. Consequently, the stored coordinates of the 200 held-out points remain valid within the same coordinate system, avoiding the cross-fit alignment problem that required Procrustes analysis in Section 5.2. The second dataset is `gustavosta_sd_prompts_v2_two_pass`, the more diverse natural-language corpus used in the image-generation experiments. Here, 200 prompts are held out from the 61.746-point dataset, leaving 61.546 visible reference points.

For each held-out sample, the inverse-estimation procedure is evaluated at that sample's stored two- or three-dimensional position. The held-out point itself is excluded from the visible reference pool, and the operation estimates q from the surrounding points. Recovery accuracy is measured by the cosine similarity between q and the held-out sample's true compact embedding. Cluster recovery is measured by whether the cluster predicted from q matches the held-out sample's canonical cluster label.

Because cosine-similarity values depend strongly on the dataset's overall embedding distribution, the results of both tests are interpreted relative to a dataset-specific background baseline, namely the mean cosine similarity between randomly paired prompts from the same dataset. Comparing the recovery values against this baseline makes it possible to judge them as a genuine signal rather than an artifact of how similar prompts in that dataset already tend to be by chance. For each corpus, 2.000 index pairs are drawn independently with replacement, meaning each draw is made from the full dataset again, so the same prompt can be selected more than once across different pairs from the full compact embedding matrix. Pairs containing the same sample twice are discarded, leaving 1.996 valid pairs for `imagenet1k_labels_classic` and 2.000 for `gustavosta_sd_prompts_v2_two_pass`.

The resulting mean background similarity is 0.970 for ImageNet-1k, with a standard deviation of 0.018. This high value is partly explained by the shared prompt template "a photo of X." For the more varied natural-language dataset, the mean is lower at 0.901, with a standard deviation of 0.030. The precision of this kind of estimate depends on sample size rather than on what fraction of the dataset was sampled, so both baselines are equally reliable despite the datasets' very different sizes. The corresponding standard errors are approximately 0.0004 and 0.0007, both far smaller than the differences the following analysis relies on. Conceptually, this follows the same background-reference principle as the `mean inter` value used by Consistency Check in Section 4.8, although here the baseline is used only for interpretation and does not affect the operation itself.

Table 5.5.: Test A held-out recovery, aggregated over 200 points per dataset

Dataset		**Recovery cosine (mean / median / SD)**	**Cluster match rate**
ImageNet-1k	2D probe	0.992 / 0.993 / 0.005	0.96
ImageNet-1k	3D probe	0.993 / 0.994 / 0.004	0.98
ImageNet-1k	random-pair baseline	0.970 / 0.975 / 0.018	–
gustavosta	2D probe	0.974 / 0.977 / 0.017	0.79
gustavosta	3D probe	0.977 / 0.981 / 0.016	0.84
gustavosta	random-pair baseline	0.901 / 0.905 / 0.030	–

Table 5.5 and Figure 5.18 show that recovery exceeds the random-pair baseline in both datasets, although by different margins. For ImageNet-1k, the mean recovery lies approximately 1.2-1.3 baseline standard deviations above the baseline mean, compared with about 2.4-2.5 standard deviations for `gustavosta_sd_prompts_v2_two_pass`. The histograms further clarify how consistently individual points stay above the baseline by dataset. For gus-

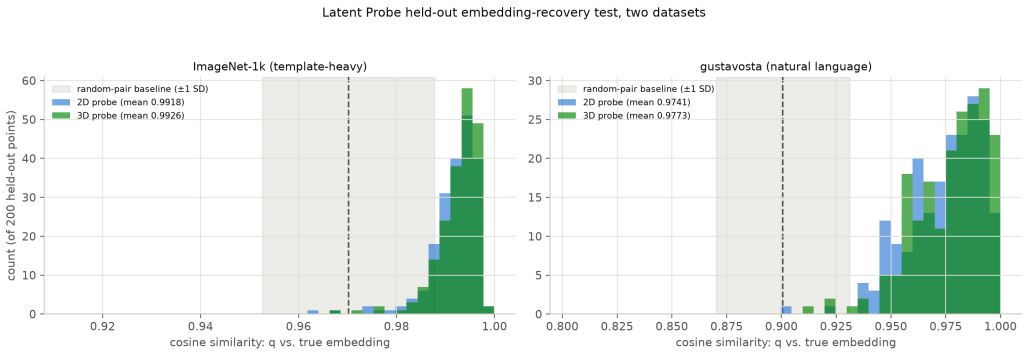


Figure 5.18.: Test A held-out recovery-cosine distribution against each dataset's own random-pair baseline band (±1 SD)

tavosta, very few of the 200 held-out samples fall inside the baseline range, meaning nearly every individual point is itself a genuinely strong recovery. For ImageNet-1k, a modest minority of points, 11 to 14 percent, still fall inside the range even though the large majority sits clearly above it, consistent with its narrower separation margin already noted above.

The clearer separation on the more diverse dataset is partly explained by the different baseline distributions. ImageNet-1k embeddings are already concentrated close to a cosine similarity of 1.0 because all prompts share the same template, as mentioned before, leaving relatively little room for recovery scores to separate from random pairs. The broader and lower baseline of the natural-language dataset makes the local-geometry signal easier to distinguish from chance. Greater prompt diversity therefore does not weaken recovery in this experiment, instead, it produces a more informative reference scale.

Recovery differs only marginally between the two- and three-dimensional probes, increasing from 0.992 to 0.993 for ImageNet-1k and from 0.974 to 0.977 for gustavosta. These differences are smaller than the variation within the recovery distributions and should not be interpreted as a meaningful advantage of the 3D representation.

Cluster recovery shows a different pattern. ImageNet-1k achieves cluster-match rates of 96 percent in 2D and 98 percent in 3D, whereas gustavosta reaches 79 percent and 84 percent, respectively. This lower result reflects a more demanding assignment problem: the natural-language dataset contains 141 clusters, with approximately 54 percent of its samples labeled as noise, while ImageNet-1k contains only about three clusters. It does not indicate weaker embedding recovery, since the same q vectors still exceed their dataset-specific baseline by the larger margin. Several of the eight ImageNet-1k mismatches in 2D involve held-out samples whose canonical labels are noise, while q is assigned to a nearby cluster instead. This is consistent with noise samples lying near cluster boundaries rather than indicating a distinct failure of the inverse-estimation procedure.

For Test B, six held-out ImageNet-1k samples were selected from the Test A results: the three with the lowest and the three with the highest recovery cosine. This allows reconstruction performance to be compared with the cases already identified as difficult or easy to recover. For each sample, the estimated vector q from Test A was reused as the target of the LLM-based reconstruction search. The highest-ranked neighbors of q served as seed prompts, while mid-ranked and distant points from the visible pool were used as noise and negative examples. The search followed the plugin's runtime configuration, using 15 seeds, 5 negative examples, 5 noise examples, and up to 10 refinement iterations or a 5-minute time limit.

Candidate prompts were generated and iteratively refined towards q. Reconstruction quality was evaluated using two cosine-similarity measures: the similarity between the reconstructed prompt's embedding and q, which indicates how closely the search reached its internal target, and the similarity between the reconstructed embedding and the held-out prompt's true embedding, which tests whether approaching q also recovers the original representation.

Table 5.6.: Test B reconstruction for six held-out points (three with the lowest, three with the highest Test A recovery cosine)

True prompt	**Reconstructed prompt**	**Recovery cos. (Test A)**	**cos. to q**	**cos. to true original**
a photo of gasmask, respirator, gas helmet	a photo of antique trolleybus, horse trolley, horse tramcar	0.963	0.986	0.956
a photo of tailed frog, bell toad, ribbed toad, tailed toad, Ascaphus trui	a photo of weathered ship's wheel	0.968	0.950	0.907
a photo of pick, plectrum, plectron	a photo of electric fan, desk fan, table fan	0.974	0.989	0.964
a photo of pillow	a photo of a hen with fluffy feathers	0.998	0.989	0.988
a photo of volleyball	a photo of a basketball hoop with players	0.998	0.989	0.989
a photo of baseball	a photo of a radiator with colorful flames	0.998	0.990	0.988

Table 5.6 reflects the same division observed in Test A. The three points with the lowest recovery cosine also yield the lowest similarities between reconstructed and original embeddings, ranging from 0.907 to 0.964, all below the ImageNet-1k random-pair baseline of 0.970. In contrast, the three strongest recoveries achieve similarities of 0.988-0.989, clearly above baseline. Despite this numerical difference, none of the six reconstructed prompts describes the same object as the original. For example, "a photo of pillow" becomes "a photo of a hen with fluffy feathers," while only "a photo of volleyball" and "a photo of a basketball hoop with players" retains a loose categorical relation. High cosine similarity therefore does not imply close topical correspondence: proximity in the compact embedding space and similarity in described content are related, but not equivalent. All six searches converged within the first beam, as no subsequent refinement improved the initial candidates beyond the configured threshold.

Test B is limited to six deliberately selected ImageNet-1k samples and was not repeated on `gustavosta_sd_prompts_v2_two_pass`, since its qualitative conclusion, close embedding similarity does not imply close described content, is a property of the reconstruction search itself rather than one expected to depend strongly on which dataset supplies the target. The selected points were chosen to span the recovery range observed in Test A rather than to form a representative random sample. In addition, the plugin's default paraphrasing model was replaced because of the 8 GB GPU-memory constraint described in Section 5.6. The exact wording of what got reconstructed is therefore specific to this particular LLM, because different models have different word choices, biases, and creative tendencies, and a different model would very plausibly phrase things differently or even land on entirely different candidate objects altogether.

Overall, the estimated target vector recovers a held-out point's embedding and cluster context more accurately than the corresponding random-pair baseline, with the stronger relative separation occurring on the more diverse dataset. The reconstruction search also follows the point-specific pattern predicted by Test A: better estimates of q lead to embeddings closer to the original. Answering this section's opening question, local geometry in the projection therefore does carry usable semantic information, in the sense of recovering an embedding and cluster context close to the original, but it does not preserve or encode which specific object or topic that embedding actually describes. This supports the interpretation of Latent Probe as an exploratory navigation tool rather than a reconstruction method.

Chapter 6.

Discussion

The preceding chapter provided a systematic, evidence-driven analysis of the operations implemented in NELS. What follows is a more reflective look at that analysis, critically asking what it actually means for the questions this thesis set out to answer. In short, this thesis asked how far latent spaces can be explored systematically without touching the underlying model, which structural and perceptual patterns and instabilities emerge along the way, and where the limits of interpretability and controllability of these latent spaces lie.

A recurring pattern across several operations was that outcomes depended more strongly on the semantic relationship between the inputs than on the geometry of the transformation itself. Interpolation provides a clear example. Although LERP and SLERP follow fundamentally different paths, one linear, the other curved to preserve vector length, they produced nearly indistinguishable images even when their embedding-space trajectories visibly diverged. Trajectory geometry therefore offered little reliable insight into the resulting visual transition. Consequently, source-prompt similarity was a much stronger indicator of whether the change appeared gradual or abrupt. Direction Walk showed the same dependency from another perspective. A direction could be transferred smoothly through embedding space, but it produced a coherent visual edit only when the new starting context was semantically compatible with that direction. Neither the magnitude nor the symmetry of the transfer predicted this outcome. Overall, the observed structural and perceptual behavior was governed less by the mechanics of the operation than by the semantic compatibility between the existing content and the transformation applied to it.

Latent Probe revealed a second, more fundamental limitation: proximity in the compact representation does not necessarily correspond to semantic similarity. Local geometry in the projected space contained enough information to recover a point's embedding and cluster context above chance, supporting the feasibility of systematic exploration without modifying the underlying model. However, reconstructing that position as a prompt often produced entirely different content, even when the reconstructed embedding remained close to the target. Semantic Axis Projection showed a related limitation. A user-defined primary axis

generalized reliably to unseen prompts from the same themes, whereas the automatically derived second axis was rarely a clear, interpretable contrast and instead reflected the strongest residual variation after removing informtaion of first axis. Systematic exploration is therefore possible, but primarily as an approximate structural analysis rather than as a precise or self-sufficient reconstruction of meaning.

With Consistency Check, however, this thesis was not able to determine whether the underlying latent space is actually stable during navigation. The operation's advertised continuous variation-strength control behaved, in practice, like a much coarser switch, and its two intended stability signals frequently disagreed with each other rather than confirming the same picture. This does not mean that navigation through this space is itself unstable, only that the specific technique built here to measure it could not reliably show one way or the other.

The third question, concerning the limits of interpretability and controllability of these latent spaces, can already be answered from the patterns described above. Direction Walk and Latent Probe both show that this system's behavior is genuinely difficult to predict and to control from an operation's own parameters alone, whether that means anticipating how a transferred direction will be expressed or assuming that a recovered position also carries the original meaning. What remains unresolved is how strongly ordinary users would perceive these limitations in practice. The qualitative assessments in Chapter 5 reflect the author's interpretation rather than evaluations by independent participants. Thus, a dedicated user study is the logical next step to test whether these automatically measured limits also hold up against real human perception.

These findings should be interpreted strictly within the scope of the evaluation. They are based on a single diffusion model, one text encoder, and a fixed generation configuration, while several experiments used deliberately selected samples rather than randomly sampled cases. The conclusions are therefore supported under the tested conditions but should not be generalized into a broader theory of diffusion latent-space behavior.

Chapter 7.

Conclusion and Future Work

This thesis presented the design, implementation, and evaluation of NELS, a ComfyUI plugin for exploring latent spaces in generative diffusion models. The work covered the complete exploration pipeline, including dataset construction, embedding projection, clustering, and a range of navigation operations. Their empirical evaluation examined structural and perceptual behavior, stability, interpretability, and controllability. The results show that latent-space navigation can be made systematic and inspectable, while also revealing that several operations behave less predictably and reliably than their design initially suggests.

These findings point to several directions for future work. The most immediate is a controlled user study examining whether the limitations identified through automated evaluation are also perceived by users and whether the plugin provides an effective environment for latent-space exploration. Further work could refine the coarse variation-strength control of Consistency Check, improve the interpretability of the automatically derived second axis in Semantic Axis Projection through additional constraints or alternative discovery methods, investigate other dimensionality-reduction techniques for more effective visualization, and repeat the evaluation across different diffusion models and larger prompt datasets.

Taken together, NELS provides both a working exploration tool and a first empirical account of how latent-space navigation actually behaves, laying the groundwork for the deeper investigations into latent spaces that future work should pursue.

The complete source code is publicly available in the project repository [16].

Appendix A.

Supplemental Information

A.1. DeepWiki Research Examples

Representative examples from the DeepWiki-assisted research conducted during this thesis, condensed and shortened from the full tool interactions.

A.1.1. ComfyUI

Building an extension without modifying ComfyUI's source

Query: *I want to build my own extension for ComfyUI without modifying ComfyUI source code. Is that possible and how can I do that?*

Response: *ComfyUI supports building extensions without modifying its core source code through its custom node system. Two approaches exist: a newer approach based on a* `ComfyExtension` *class and a* `comfy_entrypoint` *function, and an older, still widely used approach based on a* `NODE_CLASS_MAPPINGS` *dictionary. Extensions are placed in the* `custom_nodes/` *directory; during startup, ComfyUI scans this directory and loads each module, checking for* `NODE_CLASS_MAPPINGS` *or* `comfy_entrypoint`*, and optionally registers a web directory so the extension can also serve its own frontend files.*

Real-time backend-to-frontend communication

Query: *Once a workflow is running, how does the backend inform the frontend about progress or results in real time? Could a custom node package hook into it or add its own server routes?*

Response: *The backend keeps a WebSocket connection open at a dedicated endpoint and uses it to send real-time progress updates and results to the frontend, including which node is currently executing and streamed preview images during sampling. Custom node packages can hook into this same progress-reporting system rather than opening a separate connection, and can also register their own HTTP routes on the same server process, either by serving static frontend files from a declared web directory or by adding custom endpoints directly.*

A.1.2. GitNexus

Interacting with the graph

Query: *What can a user actually do with this graph? Click, hover, select, filter, zoom, search? Is there a consistent set of interactions for browsing it?*

Response: *GitNexus provides a consistent set of interactions across its Web UI: clicking a node selects it and opens a code panel, while clicking empty canvas space clears the selection; hovering over a node shows a tooltip with its name. Zoom and pan are supported through dedicated buttons, mouse drag, and the scroll wheel, and selecting a node reveals a "focus" button that animates the camera to center on it. Filtering is available by node type, by edge/relationship type, and by hop-distance from a selected node, and a header search bar performs real-time filtering by name with keyboard navigation between results.*

Graph structure: nodes and edges

Query: *How does GitNexus represent a codebase structure as a graph? What are the nodes and what are the edges/relationships between them?*

Response: *GitNexus represents a codebase as a property graph: nodes range from structural entities (files, folders, packages) to code entities (functions, methods, classes, interfaces, variables) to higher-level, automatically derived abstractions such as detected functional communities and traced execution-flow processes. Relationships between these nodes are typed as well, covering structural containment, function calls and imports, inheritance and interface implementation, and step-by-step participation in an execution-flow process. This typed structure is what lets the graph express relationships, such as which functions call which, or which symbols belong to the same functional area, directly as edges rather than something the user has to infer from a file listing.*

Rendering technology

Query: *What technology or library does GitNexus use to render and interact with this graph?*

Response: *GitNexus uses Sigma.js for WebGL-accelerated graph rendering and Graphology for the underlying graph data structure and layout algorithms. Three layout modes are available: a default force-directed layout (Force), a hierarchical layout for folder/file relationships (Tree), and a layout that clusters nodes by community membership (Circles).*

A.2. NELS Graph Panel Specification

The initial written specification that guided the AI-assisted implementation of the NELS graph-panel frontend (Section 4.4):

```
# NELS Graph Panel

## Architecture context

- ComfyUI's frontend extension mechanism is JavaScript/TypeScript-
based
  - Register the panel through ComfyUI's existing extension API (`
  app.registerExtension`)
  - Do not modify ComfyUI core
- DeepWiki analysis of the ComfyUI codebase (`Comfy-Org/ComfyUI`):
  - Custom nodes are discovered by scanning the `custom_nodes`
  directory for a `NODE_CLASS_MAPPINGS` export
  - A package can additionally export `WEB_DIRECTORY` to have its
  own JS/CSS served as static assets and auto-discovered by the
  frontend
  - Use this mechanism rather than a separate build/deploy path
- Official ComfyUI docs on sidebar tabs:
  - has dedicated `app.extensionManager.registerSidebarTab({ id,
  icon, title, type: "custom", render })` API for adding a
  persistent, dedicated panel to the ComfyUI
  - `render(element)` receives a plain DOM element to mount into
  - Use this (not a custom node widget) as the actual mounting point
   for the graph panel
  - Mount our own component tree into the provided element
- ComfyUI's own `PromptServer`:
  - Already runs an aiohttp server and exposes a WebSocket channel
  used for node-status and progress events
  - Reachable from the frontend via `api.addEventListener(eventName,
   handler)`
  - Reuse this existing channel with our own custom event name (e.g.
   `nels.session_event`) for pushing session/session-result updates
  - Do not open a separate WebSocket connection
- DeepWiki analysis of GitNexus (`abhigyanpatwari/GitNexus`):
  - Graph-based exploration UI over a large structured dataset (a
  knowledge graph of code relationships)
  - Rendered with Sigma.js + Graphology
  - User browses by selecting/hovering/filtering nodes in a spatial
  layout rather than scrolling a list or table
```

```
  - Adopt this general interaction paradigm, and Sigma.js +
  Graphology as the 2D rendering library, for exploring prompt
  embeddings
- Each dataset already stores both a 2D and a 3D UMAP projection of
the same points
  - Offer both a 2D and a 3D view of the loaded graph rather than
  committing to only one
  - Let the user pick whichever is easier to read for a given
  dataset or task

## Tech stack

- Frontend language: TypeScript, built with Vite into a single-file
bundle
  - The CSS also needs to be injected programmatically at runtime
  rather than shipped as a separate stylesheet, since ComfyUI will
  not auto-load one for a dynamically rendered tab
- Component/state framework: Preact with `@preact/signals`
  - No Redux/store framework
  - State should live in a handful of plain modules, each owning one
   concern, exporting signals and setter functions directly
- 2D graph rendering: Sigma.js, on top of graphology for the
underlying graph data structure
- 3D graph rendering: Three.js
- The interaction vocabulary must be identical across the 2D and 3D
renderer wherever practical, even though the underlying rendering
technology differs
- 2D and 3D should never both be mounted at the same time
  - Switching view modes should fully tear down one renderer and
  build the other

## State architecture

Split state into focused modules rather than one global store:

- **Graph/dataset state**: the currently loaded set of points and
their display coordinates, independent of which operation is active
  - Support both a 2D and a 3D coordinate projection of the same
  underlying data
  - A single view-mode switch re-derives the active display
  coordinates
  - Cache the resolved display coordinates per dataset configuration
  - Reuse the cached coordinates instead of trusting a freshly
  arrived snapshot when the configuration hasn't actually changed
```

```
  - Goal: routine reloads don't needlessly reposition every point (
  and don't require an expensive recomputation) unless the user
  explicitly asks for a fresh layout or the underlying configuration
   changed
  - Raw dataset coordinates are not display coordinates; map them
  into a shared, fixed display range before rendering
    - Derive the mapping from the bounding range of the currently
    loaded points, using one uniform scale factor across all axes
    rather than normalizing each axis independently
    - Goal: preserve the relative proportions of the projection (a
    cluster twice as wide as it is tall stays that shape) instead of
     stretching every axis to fill the same range
    - This mapping must be invertible, since operations in the
    backend requires raw coords rather than display position ->
    convert into raw coordinate space before sending it to the
    backend
- **Operation state**: the current operation type, its configuration
/settings (alpha schedules, neighbor counts, etc.), and the last
computed result from the backend
  - Treat this as its own concern, separate from the raw point data
- **Selection/hover state**: which point ids are currently selected,
 and which one (if any) is hovered
  - Keep independent of the operation and dataset state, so that
  selecting points doesn't require touching operation config, and
  vice versa
- **Session/sync state**: discovering and loading sessions from the
backend, applying incoming state, and pushing local UI changes back
to the backend
  - This is the orchestration layer
  - It should be the only place that talks to the API client and to
  the live event channel
- **Event/status log state**: a bounded log of backend events and a
single current status string
  - Keep separate from everything else so that logging never becomes
   a bottleneck for the rest of the UI
- **Misc**: Avoid direct imports between the renderer modules (2D/3D
) and the session/orchestration module in either direction
  - If the orchestration layer needs to reach "whichever renderer is
   currently active," introduce a small indirection layer for that
  - Do not import renderer code directly into session logic, or
  session logic into renderer code

## Backend communication
```

```
- All request/response calls (loading a session, pushing UI state,
requesting a fresh preview, ...) go through a single thin REST
client
  - The client proxies through ComfyUI's own fetch helper, not a
  separate HTTP client library
- All live push updates (new results, progress during long-running
operations, newly available data) arrive through the shared
WebSocket event channel described above
  - Dispatch by event type in one central place
- Define the full data contract (points, sessions, UI state,
operation results) as shared TypeScript types
  - These types should mirror the backend's own data model field-for
  -field, so the two sides can be kept in sync by inspection rather
  than by convention
- Panel interactions (changing the selection, editing operation
config, ...) never trigger a computation directly
  - Each interaction only pushes an incremental state update to the
  backend
  - The actual operation runs only when the user queues the ComfyUI
  graph itself, against a frozen snapshot of the interface state
  taken at that moment
  - Goal: further UI interaction while an operation is running never
   changes the inputs of that already-queued run

## Panel structure

- Register a single dedicated, resizable side panel (not a node-body
 widget) inside ComfyUI's own UI
- Provide a toggle between a graph view and a logs view
- Layout:
  - A main graph area (the 2D/3D canvas plus overlays)
  - A sidebar containing the operation configuration, the current
  result, and current session info
  - Each sidebar section lives in its own collapsible card, so the
  user can hide sections they aren't currently using
  - The sidebar is resizable by dragging its edge
  - The sidebar remembers its width across sessions (e.g. via local
  storage)
- The graph area needs supporting overlays beyond the raw point
cloud:
  - A point-count indicator
  - A cluster legend and cluster hulls
  - An optional legend display placeholder; fillable with custom
  content by the active operation
  - A hover tooltip
```

```
  - A drag-select rectangle overlay
  - All of these are layered on top of the render surface without
  interfering with it
- Provide a way to switch between multiple known sessions/datasets
  - dropdown selector in the header
  - Each session/dataset is a self-contained set of points and
  clusters
  - Switching the active session via the dropdown alone is enough to
   load whatever is already known for that session; it does not
  require pressing reload/refresh
- Provide a separate reload/refresh control that re-syncs the list
of known sessions and forces a fresh dataset preview for the
currently active session, bypassing the coordinate cache described
above so the layout actually updates
  - This is for when the user changed the dataset loader's
  configuration (path, sample count, seed, ...) and wants to see the
   update without queuing the full downstream pipeline
  - Outside of this explicit action, if a new snapshot arrives for
  the same session (e.g. from a live update) and the dataset
  configuration hasn't actually changed, keep using the cached
  layout instead of recomputing it - see the caching behavior
  described in State architecture above
- Provide a way to export the current view as a single, self-
contained, shareable file
  - Include an image of the current view
  - Include a summary of the loaded dataset, clusters, and current
  operation result

## Interaction model

Must behave the same in both the 2D and 3D view wherever the
underlying input technology allows it:

- Click selects a point, replacing the current selection
- Shift-, ctrl-, or meta-click adds or removes a point from the
current selection instead of replacing it
- Shift-drag on empty background opens a rectangular marquee
selection
  - Releasing it selects everything inside the rectangle
  - Treat a very small drag as a plain click rather than a selection
  , so accidental tiny drags don't clear an existing selection
  unintentionally
- Hovering a point shows a tooltip with that point's data (id, text,
 cluster, coordinates)
```

```
- Right-clicking a point opens a small context menu (at minimum, a
way to copy the point's prompt text)
- Clicking empty background clears the current selection
  - Exception: if the active operation is in a special placement
  mode (see Latent Probe below), the click instead places a marker
  at that position
- In the 3D view specifically, dragging the camera (orbit/pan/zoom)
must not conflict with shift-drag selection
  - Disable camera controls for the duration of a shift-drag and re-
  enable them afterward

## Panel-level controls and interactions

Beyond the graph canvas itself, the surrounding panel chrome needs
its own set of controls:

- **Header**:
  - A session selector (switch which loaded dataset/session is
  active)
  - An adjacent reload/refresh control
  - A 2D/3D view toggle
  - An animate on/off toggle
  - A status indicator showing whether the LLM used for labeling/
  annotation is currently loaded, with a way to unload it on demand
  to free resources
  - An export action
- **Operation card**:
  - A dropdown to choose which navigation operation is currently
  active, which then reveals only that operation's own controls
  - A "clear" action that resets the current selection, result, and
  any in-progress operation-specific state together
  - A "fit view" action that re-frames the camera/viewport to the
  currently loaded points
- **Cluster legend**:
  - Collapsible open/closed
  - A toggle for showing/hiding cluster outline hulls
  - Clicking a cluster entry selects every point in that cluster and
   filters/dims the rest
  - Clicking the same entry again clears that filter
- **Result card**, once an operation has produced a result:
  - Clicking any individual listed result item (a neighbor hit, a
  ranked axis-projection entry, a generated variant, a role-labeled
  source point) selects the corresponding point on the canvas,
  always replacing the current selection rather than adding to it
```

```
  - For trajectory-producing operations, provide a per-step table
  where clicking a step highlights that specific generated point on
  the canvas
  - For trajectory-producing operations, provide a toggle for
  whether generated points should snap onto their real anchor points
  - For trajectory-producing operations, provide a toggle for
  showing or hiding the generated points/overlay on the canvas at
  all
  - For semantic axis projection specifically, provide a toggle that
   visually highlights just the projection path/trajectory, dimming
  everything else in the graph
- All sidebar sections (operation configuration, result, session
info) should be individually collapsible, so the user can hide the
ones they aren't currently using

## Operation results in the graph

- Do not render operation outputs (trajectory steps, generated
variants, nearest-neighbor hits) in a separate results view
- Render them as additional points inside the same graph instead
  - Visually distinguish them from real dataset points (different
  texture/marker, a highlighted ring or outline, and a subtle idle
  animation) so they read as "generated" rather than "loaded"
  - Keep them in the same coordinate space as real points
  - Make them selectable/hoverable/right-clickable through the same
  interaction code path as real points, not a separate one
- Where a result implies an ordered path between points (an
interpolation, a directed walk), draw a connecting line or curve
between the relevant points/results
- Where a result is a set of independent items with no implied order
 (e.g. generated variants for a consistency check), do not draw a
connecting path between them
- Where a result is an unordered set of per-result matches to a
query point (e.g. nearest-neighbor hits), draw a dashed connection
from each result back to its query point instead
  - Use the dashed style specifically to distinguish this from the
  solid ordered-path case above
- Provide a way to snap a generated point visually onto its real "
anchor" point at the extremes of a schedule (e.g. the very start/end
 of an interpolation)
  - Goal: it's clear which real point a generated sequence begins or
   ends at, even while idle animation is active

## Interpretability affordances
```

```
- Cluster legend:
  - List every detected cluster with a color swatch, a
  representative sample prompt, and its point count
  - Clicking an entry filters/highlights the view to that cluster
  - Clicking again clears the filter
  - Support toggling a visual outline (hull) around each cluster's
  spatial extent
  - Also show, per cluster, how many of the active operation's
  result variants (e.g. from a Consistency Check) belong to it
    - Include clusters that have no members among the currently
    loaded points, since a variant can belong to a cluster the
    loaded sample doesn't otherwise represent
- Axis legend (for operations that produce a labeled axis/direction)
:
  - Recolor every loaded point along a gradient based on its
  projection score onto the axis
  - Show the two human-readable poles of the axis next to the
  relevant score range
  - Show an optional LLM-generated name for the axis if the user
  requested one
  - Once a second (orthogonal) axis is available, its own gradient
  and pole labels replace the primary axis's when the user switches
  to it (see the color-mode toggle under Semantic axis projection
  below)
  - Goal: the axis is never shown as a bare, unlabeled number

## Per-operation configuration UI

A dropdown lets the user choose which navigation operation is
currently active; only that operation's own controls should be shown
 below it, each with only the controls that operation actually needs
:

- **Inspection (no-op)**:
  - Controls: none operation-specific
  - Requires: a selection of one or more points to inspect
- **Nearest neighbor**:
  - Controls: a neighbor-count control
  - Requires: at least one selected point
- **Linear interpolation (lerp) / spherical interpolation (slerp)**:
  - Controls: an editable alpha schedule (the interpolation steps
  between the two endpoints) with quick-fill presets (e.g. evenly
  spaced 3/5/10 steps in [0,1])
  - Controls: an opt-in toggle for whether each generated trajectory
   step should also look up its nearest real dataset prompt
```

```
  - Requires: exactly two selected points
- **Direction/delta walk**:
  - Controls: its own alpha-schedule presets in the range [-2,4] (e.
  g. negative, forward, and full-range presets)
  - Controls: show the three selected points with their role labels
  (direction-from / direction-to / walk-start) once all three are
  selected
  - Controls: the same opt-in "nearest real dataset prompt per step"
   toggle as lerp/slerp
  - Requires: exactly three selected points
- **Semantic axis projection**:
  - Controls: two disjoint point groups ("Group A" / "Group B"),
  each with an "add selected points to this group" action and a "
  clear group" action, keeping the two groups mutually exclusive
  - Controls: an optional "compute a human-readable label for this
  axis" toggle (note in the UI that enabling it adds extra
  processing time)
  - Controls: once a second (orthogonal) axis is available, a toggle
   to switch which axis's scores currently drive the point coloring/
  display
  - Requires: at least one point in each group
- **Consistency check**:
  - Controls: a variant-count control and a variation-strength
  control, with inline guidance distinguishing a "light word-level
  edit" regime at low strength from a "full LLM paraphrase" regime
  at high strength
  - Requires: exactly one selected point (the seed)
- **Latent-space probing**:
  - Controls: a "place a probe point" mode that lets the user click
  a position directly on the graph instead of selecting existing
  points
  - Controls: once placed, let the user fine-tune the probe position
   via direct numeric coordinate input instead of only re-clicking
  - Controls: an optional "reconstruct text for this position"
  toggle that unlocks further settings (a maximum search time, a
  maximum number of refinement iterations, and counts for how many
  nearby/neutral/dissimilar existing points to use as context)
  - Requires: a placed probe position rather than a point selection

Across all operations:

- Show a clear, specific warning whenever the current selection
doesn't satisfy what the active operation requires, rather than only
 failing silently when the operation is run
```

```
- Document the selection interaction vocabulary (click / modifier-
click / shift-drag / empty-click) directly in the UI as in-app help,
 not only in external documentation

## Known limitation to label, not engineer around

The graph should support a purely cosmetic idle animation (a slight,
 continuous jitter of point positions) to make the view feel alive
and make orientation easier, but this means displayed positions are
not always pixel-exact while it runs

- Provide a toggle to disable it
- Label its effect directly in the UI (e.g. "animate ON - positions
not exact") so a smooth-looking view is never mistaken for a precise
 one
```

A.3. LLM System Prompts

The system-prompt templates referenced in Sections 4.6 and 4.8, transcribed from the plugin source. Curly-brace placeholders (e.g. `{style}`) mark values interpolated at call time. The accompanying user message for each call is described briefly below its title, since it consists mainly of the reference prompts, axis terms, or term-profile data already described in the main text.

A.3.1. Dataset annotation (Section 4.6)

Term-profile-based cluster labeling:
(`label_cluster_from_profile()`, `core/operations/cluster_annotation.py`)

User content supplies the term profile (most frequent terms, most distinctive terms, key phrases) and a small set of representative example prompts, and, once other clusters have already been labeled, the list of labels already assigned so far to avoid duplicates.

System prompt: *You label semantic clusters`{ctx}`. You are given the most frequent terms, most distinctive terms, key phrases, and a few raw examples. Use all of them to infer the shared theme and output a single short label (2-5 words). Example - frequent: tench, goldfish, shark | distinctive: aquatic, marine | phrases: great white, tiger shark -> Aquatic Animals. Example - frequent: tabby, persian, siamese | distinctive: feline, domestic | phrases: persian cat -> Domestic Cats. Output ONLY the label. One line, no punctuation at the end.*

Raw-example cluster labeling (no-prefix fallback):
(`label_cluster()`, `core/operations/cluster_annotation.py`)

User content supplies up to 20 representative example prompts directly, plus the same term profile when available. This is the fallback path used when the dataset has no detectable shared prompt prefix, since the prefix is what the term-profile extraction relies on to strip templated text before counting terms.

System prompt: *You label semantic clusters of image generation prompts`{ctx}`. Output a single short label (2-5 words) capturing the shared theme. Example: 'a sports car racing', 'a blue sedan on a highway' -> Sports Cars. Example: 'a medieval castle at dusk', 'stone fortress walls' -> Medieval Architecture. Output ONLY the label. One line, no punctuation at the end.*

Refinement round:
(`refine_labels()`, `core/operations/cluster_annotation.py`)

User content lists every cluster's current label together with its frequent and distinctive terms, one line per cluster.

System prompt: *You refine semantic cluster labels`{ctx}`. Each line shows a cluster's current label, its most frequent terms, and its most distinctive terms. Improve labels that are too generic or overlap with another cluster - use the distinctive terms to differentiate. Reply with ONLY the revised labels, one per line, in the same order. Format: 'Cluster <id>: <label>'. Do not change labels that are already specific and unique.*

A.3.2. Semantic Axis Projection (Section 4.8)

Candidate label generation:
(`generate_axis_label_candidates()`, `core/operations/axis_projection.py`)

User content supplies the contrastive axis terms for both sides of the axis and their example prompts, plus, when labeling the second axis, the already-accepted first-axis label to exclude.

System prompt: *You name semantic axes in an image prompt embedding space. The keywords summarize what is distinctive on each side of the axis; the example prompts give tone and context. Reply with short, vivid phrases in the format 'X to Y' describing the concrete visual or thematic concept that changes along the axis. Never use generic scales like 'low to high' or 'simple to complex' unless they are grounded in the actual content. Return exactly `{n}` different candidate labels as a numbered list, one per line. Each line must contain only one 'X to Y' phrase.*

Candidate selection and refinement:
(`critique_and_refine_axis_label()`, `core/operations/axis_projection.py`)

User content supplies the same contrastive axis terms together with the numbered list of candidates from the previous step. The bracketed exclusion line is appended only when labeling the second axis, carrying the already-accepted first-axis label forward so it cannot be reused.

System prompt: *You pick the best of several candidate axis labels for an image prompt embedding space. Judge by: (1) precision to the keywords, (2) vividness, (3) distinctness of both sides. You may lightly rewrite the winning candidate to sharpen it, but keep the 'X to Y' format and keep it grounded in the keywords. Output ONLY the final phrase, nothing else. [Already used for a different axis (do not reuse): `{exclude_label}`]*

A.3.3. Consistency Check (Section 4.8)

LLM paraphrasing:
(`paraphrase()`, `core/operations/consistency_check.py`)

User content is the base prompt itself. `{style}` is one of "minor rewording, synonyms only" (strength below 0.4), "moderate rephrasing, vary structure" (0.4 to below 0.7), or "aggressive paraphrase, change phrasing substantially" (0.7 and above); `{max_variant_words}` is the base prompt's own word count plus 5; the bracketed avoid-list is appended once earlier variants exist within the same call.

System prompt: *You paraphrase short image generation prompts. Style: `{style}`. Keep the paraphrase to at most `{max_variant_words}` words. Output exactly ONE paraphrase: a single line, no numbering, no extra text. [Avoid these already used paraphrases: `{avoid_hint}`]*

A.3.4. Latent Probe (Section 4.8)

Initial candidate generation:
(`probe_synthesize()`, `core/operations/latent_probe.py`)

User content supplies the nearby "seed" reference prompts, labeled "write something like these", and, if available, the negative reference prompts, labeled "do not write anything like these".

System prompt: *You synthesize image generation prompts. Given reference prompts near a target concept and prompts far from it, write new prompts that match the target concept. Output exactly `{count}` prompts, one per line, no numbering, no explanations. Do not copy*

phrases from the references. Each prompt must be 5-20 words and a complete self-contained image description.

Refinement iteration:
(`probe_suggest_iter()`, `core/operations/latent_probe.py`)

User content supplies the current ranked candidate beam with similarity scores, the same negative reference prompts, and a trailing instruction: *Write `{count}` new image prompts that push further in the direction of the top-scoring candidates and avoid the themes of the negatives. Use different vocabulary and phrasing - do not copy from the ranked list above.*

System prompt: *You write short image generation prompts. Output only the prompts themselves - one per line, no numbering, no explanations, no analysis. Every line must be a self-contained image prompt of 5-20 words.*

List of Figures

2.1. Illustration of the forward noising and reverse denoising process in diffusion models, from [62], p. 6 . . . 4
2.2. UMAP embedding of random RGBA colors into 2 dimensions; nearby points share similar colors, from [42] . . . 7
2.3. The same random-color data embedded into 3 dimensions instead, from [42] . . . 7

4.1. The NELS graph panel in 2D view: 1500 points colored by cluster, the cluster sidebar with legend, and the operation controls . . . 23
4.2. The same session in 3D view . . . 23
4.3. Images generated along the primary axis and the automatically discovered second axis with the LLM-generated axis labels and the three contrastive axis terms 45
4.4. Latent Probe text-reconstruction search, retained snapshots in order, with cosine similarity to the estimated target q and the reconstructed prompt text below each image . . . 51

5.1. Held-out points' ground-truth position (gray circles) against each strategy's Procrustes-aligned runtime-predicted position (colored triangles) . . . 57
5.2. Mean step-to-step LPIPS and cosine similarity across the three similarity tiers . 59
5.3. Per-step LPIPS and cosine similarity profile across the four step transitions, averaged over the two pairs per tier . . . 60
5.4. Representative trajectory, high-similarity tier (pair 2): forest fox to cyberpunk-styled fox . . . 61
5.5. Representative trajectory, medium-similarity tier (pair 4): grizzly bear monster to nurse portrait . . . 61
5.6. Representative trajectory, low-similarity tier (pair 5): garden gnome in a medieval tavern to spacecraft . . . 62
5.7. LERP/SLERP trajectories of three representative pairs, in the runtime 2D and 3D reducer output . . . 63
5.8. Baseline-relative LPIPS and cosine similarity to the $\alpha = 0$ step, across the full alpha schedule, near and far context . . . 65
5.9. Per-step (consecutive) LPIPS and cosine similarity profile across the six step transitions, near and far context . . . 66
5.10. Direction 1 (add cyberpunk styling), near and far context across the full alpha schedule . . . 67

5.11. Direction 2 (shoes to jacket focus shift), near and far context across the full alpha schedule 68
5.12. Direction 3 (named-identity swap), near and far context across the full alpha schedule 68
5.13. Delta/Direction Walk anchors and both walks' projected positions, runtime 2D and 3D reducer output 69
5.14. Axis-1 projection-score distribution for held-out cluster members; triangles mark the 20 anchor points per side 72
5.15. Axis anchor direction and held-out cluster members, canonical 2D and 3D projection 74
5.16. Consistency score and cluster stability across the strength sweep. Dashed line: word-dropout to LLM-paraphrase boundary. Dotted lines: LLM style-tier boundaries 76
5.17. Base prompt "medium" (Dwayne Johnson lightsaber) rendered at its unmodified base prompt and three of its measured variants, spanning word dropout, LLM minor rewording, and LLM aggressive paraphrase 77
5.18. Test A held-out recovery-cosine distribution against each dataset's own random-pair baseline band (±1 SD) 80

List of Tables

4.1. Consistency Check variants for the shared base prompt (imagenet1k_labels_-classic), three returned variants per representative strength setting 47

5.1. Held-out accuracy per strategy. Neighbor recall is reported as runtime-model / ground-truth-model 56
5.2. Mean, worst-step LPIPS, cosine similarity per similarity tier and operation, averaged over the two pairs per tier 59
5.3. Consecutive-step and baseline-relative summary per direction and starting context 65
5.4. Held-out separation and support-band precision per axis pair 72
5.5. Test A held-out recovery, aggregated over 200 points per dataset 79
5.6. Test B reconstruction for six held-out points (three with the lowest, three with the highest Test A recovery cosine) 81

Bibliography

[1] Charu C. Aggarwal, Alexander Hinneburg, and Daniel A. Keim. "On the Surprising Behavior of Distance Metrics in High Dimensional Space". In *Database Theory — ICDT 2001*, volume 1973 of *Lecture Notes in Computer Science*, pages 420–434, Berlin, Heidelberg, 2001. Springer. DOI: 10.1007/3-540-44503-X_27 (Accessed 16 July 2026).

[2] Birch-san. "PyTorch implementation of spherical linear interpolation (GitHub Gist)". https://gist.github.com/Birch-san/230ac46f99ec411ed5907b0a3d728efa (Accessed 14 July 2026).

[3] Black Forest Labs. "FLUX.2 [klein]: Towards Interactive Visual Intelligence". https://bfl.ai/blog/flux2-klein-towards-interactive-visual-intelligence (Accessed 15 July 2026).

[4] Tomas Cabezon Pedroso and Daragh Byrne. "Browsing the Latent Space: A New Approach to Interactive Design Exploration for Volumetric Generative Systems". In *Proceedings of the 15th Conference on Creativity and Cognition*, pages 330–333, New York, NY, USA, 2023. Association for Computing Machinery. DOI: 10.1145/3591196.3596815 (Accessed 16 June 2026).

[5] Ricardo J. G. B. Campello, Davoud Moulavi, Arthur Zimek, and Jörg Sander. "Hierarchical Density Estimates for Data Clustering, Visualization, and Outlier Detection". *ACM Transactions on Knowledge Discovery from Data (TKDD)*, 10(1):5:1–5:51, 2015. DOI: 10.1145/2733381 (Accessed 10 July 2026).

[6] CaptionEmporium. "conceptual-captions-cc12m-llavanext (Hugging Face dataset)". https://huggingface.co/datasets/CaptionEmporium/conceptual-captions-cc12m-llavanext (Accessed 14 May 2026).

[7] Soravit Changpinyo, Piyush Sharma, Nan Ding, and Radu Soricut. "Conceptual 12M: Pushing Web-Scale Image-Text Pre-Training To Recognize Long-Tail Visual Concepts", 2021. https://arxiv.org/abs/2102.08981v2 (Accessed 19 July 2026).

[8] Comfy-Org. "Comfy-Org FLUX.2 Klein 4B (Hugging Face model repository)". https://huggingface.co/Comfy-Org/vae-text-encorder-for-flux-klein-4b (Accessed 19 July 2026).

[9] Comfy-Org. "ComfyUI (GitHub repository)". https://github.com/Comfy-Org/ComfyUI (Accessed 19 July 2026).

[10] Comfy-Org. "Desktop (Installation)". https://docs.comfy.org/installation/desktop/windows (Accessed 19 July 2026).

[11] Comfy-Org. "Getting Started (Custom Nodes Walkthrough)". https://docs.comfy.org/custom-nodes/walkthrough (Accessed 15 July 2026).

[12] Comfy-Org. "Lifecycle (Custom Nodes Backend)". https://docs.comfy.org/custom-nodes/backend/lifecycle (Accessed 15 July 2026).

[13] Comfy-Org. "Overview (Custom Nodes Overview)". https://docs.comfy.org/custom-nodes/overview (Accessed 15 July 2026).

[14] Comfy-Org. "Portable (Installation, Windows)". https://docs.comfy.org/installation/comfyui_portable_windows (Accessed 19 July 2026).

[15] daspartho. "stable-diffusion-prompts (Hugging Face dataset)". https://huggingface.co/datasets/daspartho/stable-diffusion-prompts (Accessed 14 May 2026).

[16] davidg-h. "NeLs (GitHub repository)". https://github.com/davidg-h/NeLs (Accessed 30 July 2026).

[17] DeepWiki. "DeepWiki". https://deepwiki.com/ (Accessed 19 July 2026).

[18] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. "The Faiss library", 2025. DOI: 10.48550/arXiv.2401.08281 (Accessed 16 July 2026).

[19] Elizabeth DuPre. "3. Procrustes alignment (fmralign tutorials)", 2021. https://neurodatascience.github.io/fmralign-tutorials/1-2-procrustes.html (Accessed 25 July 2026).

[20] Hugging Face. "Datasets". https://huggingface.co/docs/datasets (Accessed 19 July 2026).

[21] Hugging Face. "Transformers". https://huggingface.co/docs/transformers (Accessed 19 July 2026).

[22] Rohit Gandikota, Joanna Materzynska, Tingrui Zhou, Antonio Torralba, and David Bau. "Concept Sliders: LoRA Adaptors for Precise Control in Diffusion Models", 2023. DOI: 10.48550/arXiv.2311.12092 (Accessed 8 April 2026).

[23] GeeksforGeeks. "Mann-Whitney U test". https://www.geeksforgeeks.org/machine-learning/mann-whitney-u-test-2/ (Accessed 27 July 2026).

[24] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. "Generative Adversarial Networks", 2014. DOI: 10.48550/arXiv.1406.2661 (Accessed 9 July 2026).

[25] Jiayi Guo, Xingqian Xu, Yifan Pu, Zanlin Ni, Chaofei Wang, Manushree Vasu, Shiji Song, Gao Huang, and Humphrey Shi. "Smooth Diffusion: Crafting Smooth Latent Spaces in Diffusion Models", 2023. DOI: 10.48550/arXiv.2312.04410 (Accessed 8 July 2026).

[26] Gustavosta. "Stable-Diffusion-Prompts (Hugging Face dataset)". https://huggingface.co/datasets/Gustavosta/Stable-Diffusion-Prompts (Accessed 14 May 2026).

[27] René Haas, Inbar Huberman-Spiegelglas, Rotem Mulayoff, Stella Graßhof, Sami S. Brandt, and Tomer Michaeli. "Discovering Interpretable Directions in the Semantic Latent Space of Diffusion Models", 2024. DOI: 10.48550/arXiv.2303.11073 (Accessed 20 June 2026).

[28] Jack Hessel, Ari Holtzman, Maxwell Forbes, Ronan Le Bras, and Yejin Choi. "CLIPScore: A Reference-free Evaluation Metric for Image Captioning", 2022. DOI: 10.48550/arXiv.2104.08718 (Accessed 26 June 2026).

[29] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium", 2017. DOI: 10.48550/arXiv.1706.08500 (Accessed 8 July 2026).

[30] Irina Higgins, Loic Matthey, Xavier Glorot, Arka Pal, Benigno Uria, Charles Blundell, Shakir Mohamed, and Alexander Lerchner. "Early Visual Concept Learning with Unsupervised Deep Learning", 2016. DOI: 10.48550/arXiv.1606.05579 (Accessed 28 June 2026).

[31] Jonathan Ho, Ajay Jain, and Pieter Abbeel. "Denoising Diffusion Probabilistic Models", 2020. DOI: 10.48550/arXiv.2006.11239 (Accessed 5 July 2026).

[32] Lawrence Hubert and Phipps Arabie. "Comparing partitions". *Journal of Classification*, 2(1):193–218, 1985. DOI: 10.1007/BF01908075 (Accessed 8 July 2026).

[33] ILSVRC. "imagenet-1k (Hugging Face dataset)". https://huggingface.co/datasets/ILSVRC/imagenet-1k (Accessed 14 May 2026).

[34] Alexis Jacomy and Guillaume Plique. "Sigma.js". https://www.sigmajs.org/ (Accessed 18 July 2026).

[35] jxie. "coco_captions (Hugging Face dataset)". https://huggingface.co/datasets/jxie/coco_captions (Accessed 14 May 2026).

[36] Minsuk Kahng, Nikhil Thorat, Duen Horng Chau, Fernanda Viégas, and Martin Wattenberg. "GAN Lab: Understanding Complex Deep Generative Models using Interactive Visual Experimentation". *IEEE Transactions on Visualization and Computer Graphics*, 25(1):310–320, 2019. DOI: 10.1109/TVCG.2018.2864500 (Accessed 2 July 2026).

[37] Andrej Karpathy. "hacky stablediffusion code for generating videos (GitHub Gist)". https://gist.github.com/karpathy/00103b0037c5aaea32fe1da1af553355 (Accessed 9 July 2026).

[38] Diederik P. Kingma and Max Welling. "Auto-Encoding Variational Bayes", 2014. DOI: 10.48550/arXiv.1312.6114 (Accessed 18 June 2026).

[39] Mingi Kwon, Jaeseok Jeong, and Youngjung Uh. "Diffusion Models already have a Semantic Latent Space", 2023. DOI: 10.48550/arXiv.2210.10960 (Accessed 20 July 2026).

[40] Junsung Lee, Minsoo Kang, and Bohyung Han. "Diffusion-Based Image-to-Image Translation by Noise Correction via Prompt Interpolation", 2024. DOI: 10.48550/arXiv.2409.08077 (Accessed 15 July 2026).

[41] Justin Matejka, Michael Glueck, Erin Bradner, Ali Hashemi, Tovi Grossman, and George Fitzmaurice. "Dream Lens: Exploration and Visualization of Large-Scale Generative Design Datasets". In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, pages 1–12, New York, NY, USA, 2018. Association for Computing Machinery. DOI: 10.1145/3173574.3173943 (Accessed 3 July 2026).

[42] Leland McInnes. "Basic UMAP Parameters (umap-learn documentation)". https://umap-learn.readthedocs.io/en/latest/parameters.html (Accessed 28 July 2026).

[43] Leland McInnes. "Using UMAP for Clustering (umap-learn documentation)". https://umap-learn.readthedocs.io/en/latest/clustering.html (Accessed 16 July 2026).

[44] Leland McInnes, John Healy, and James Melville. "UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction", 2020. DOI: 10.48550/arXiv.1802.03426 (Accessed 17 July 2026).

[45] Byeonghu Na, Minsang Park, Gyuwon Sim, Donghyeok Shin, HeeSun Bae, Mina Kang, Se Jung Kwon, Wanmo Kang, and Il-Chul Moon. "Diffusion Adaptive Text Embedding for Text-to-Image Diffusion Models", 2025. DOI: 10.48550/arXiv.2510.23974 (Accessed 28 June 2026).

[46] Mayu Otani, Riku Togashi, Yu Sawai, Ryosuke Ishigami, Yuta Nakashima, Esa Rahtu, Janne Heikkilä, and Shin'ichi Satoh. "Toward Verifiable and Reproducible Human Evaluation for Text-to-Image Generation", 2023. DOI: 10.48550/arXiv.2304.01816 (Accessed 15 July 2026).

[47] Yong-Hyun Park, Mingi Kwon, Jaewoong Choi, Junghyo Jo, and Youngjung Uh. "Understanding the Latent Space of Diffusion Models through the Lens of Riemannian Geometry", 2023. DOI: 10.48550/arXiv.2307.12868 (Accessed 22 April 2026).

[48] Abhigyan Patwari. "GitNexus: The Zero-Server Code Intelligence Engine". https://github.com/abhigyanpatwari/GitNexus (Accessed 10 June 2026).

[49] Guillaume Plique. "Graphology: a robust and multipurpose Graph object for JavaScript". DOI: 10.5281/zenodo.5681257 (Accessed 20 July 2026).

[50] Preact contributors. "Preact". https://preactjs.com/ (Accessed 19 July 2026).

[51] Preact contributors. "Signals (Preact Guide)". https://preactjs.com/guide/v10/signals/ (Accessed 19 July 2026).

[52] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. "Learning Transferable Visual Models From Natural Language Supervision", 2021. DOI: 10.48550/arXiv.2103.00020 (Accessed 13 July 2026).

[53] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. "High-Resolution Image Synthesis with Latent Diffusion Models", 2022. DOI: 10.48550/arXiv.2112.10752 (Accessed 20 July 2026).

[54] Tim Sainburg, Leland McInnes, and Timothy Q. Gentner. "Parametric UMAP embeddings for representation and semi-supervised learning", 2021. DOI: 10.48550/arXiv.2009.12981 (Accessed 8 July 2026).

[55] Gerard Salton and Chris Buckley. "Term Weighting Approaches in Automatic Text Retrieval". Technical report, Cornell University, 1987. https://hdl.handle.net/1813/6721 (Accessed 16 July 2026).

[56] Dvir Samuel, Rami Ben-Ari, Nir Darshan, Haggai Maron, and Gal Chechik. "Norm-guided latent space exploration for text-to-image generation", 2023. DOI: 10.48550/arXiv.2306.08687 (Accessed 17 June 2026).

[57] Yujun Shen, Ceyuan Yang, Xiaoou Tang, and Bolei Zhou. "InterFaceGAN: Interpreting the Disentangled Face Representation Learned by GANs", 2020. DOI: 10.48550/arXiv.2005.09635 (Accessed 9 July 2026).

[58] Yujun Shen and Bolei Zhou. "Closed-Form Factorization of Latent Semantics in GANs", 2021. DOI: 10.48550/arXiv.2007.06600 (Accessed 15 June 2026).

[59] Yuying Tang, Ningning Zhang, Mariana Ciancia, and Zhigang Wang. "Exploring the Impact of AI-generated Image Tools on Professional and Non-professional Users in the Art and Design Fields", 2024. DOI: 10.48550/arXiv.2406.10640 (Accessed 15 July 2026).

[60] Three.js contributors. "Three.js". https://threejs.org/ (Accessed 19 July 2026).

[61] Jiazheng Xu, Xiao Liu, Yuchen Wu, Yuxuan Tong, Qinkai Li, Ming Ding, Jie Tang, and Yuxiao Dong. "ImageReward: Learning and Evaluating Human Preferences for Text-to-Image Generation", 2023. DOI: 10.48550/arXiv.2304.05977 (Accessed 8 July 2026).

[62] Ling Yang, Zhilong Zhang, Yang Song, Shenda Hong, Runsheng Xu, Yue Zhao, Wentao Zhang, Bin Cui, and Ming-Hsuan Yang. "Diffusion Models: A Comprehensive Survey of Methods and Applications", 2025. DOI: 10.48550/arXiv.2209.00796 (Accessed 27 July 2026).

[63] Hu Yu, Hao Luo, Fan Wang, and Feng Zhao. "Uncovering the Text Embedding in Text-to-Image Diffusion Models", 2024. DOI: 10.48550/arXiv.2404.01154 (Accessed 28 June 2026).

[64] E. Zhixuan Zeng, Yuhao Chen, and Alexander Wong. "Decoding Diffusion: A Scalable Framework for Unsupervised Analysis of Latent Space Biases and Representations Using Natural Language Prompts", 2024. DOI: 10.48550/arXiv.2410.21314 (Accessed 6 July 2026).

[65] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric", 2018. DOI: 10.48550/arXiv.1801.03924 (Accessed 27 June 2026).

[66] Zhihua Zhong and Xuanyang Huang. "Latent Diffusion : Multi-Dimension Stable Diffusion Latent Space Explorer", 2025. DOI: 10.48550/arXiv.2509.22038 (Accessed 11 July 2026).

Glossary

ARI Adjusted Rand Index. i

AUC Area Under the ROC Curve. i

BLIP Bootstrapping Language-Image Pre-training. i

CLIP Contrastive Language-Image Pre-training. i

DATE Diffusion Adaptive Text Embedding. i

FID Fréchet Inception Distance. i

FP4 4-bit Floating Point. i

FP8 8-bit Floating Point. i

GAN Generative Adversarial Network. i

HDBSCAN Hierarchical Density-Based Spatial Clustering of Applications with Noise. i

kNN k-Nearest Neighbor. i

LERP Linear Interpolation. i

LPIPS Learned Perceptual Image Patch Similarity. i

MLP Multi-Layer Perceptron. i

NELS Navigation and Exploration of Latent Spaces. i

ROC Receiver Operating Characteristic. i

SLERP Spherical Linear Interpolation. i

SVD Singular Value Decomposition. i

TF-IDF Term Frequency-Inverse Document Frequency. i

UMAP Uniform Manifold Approximation and Projection. i